



שם בית הספר: עתיד יוענה ז' בוטינסקי

שם העבודה: luz

שם התלמיד: אריאל חלף

ת.ז התלמיד: 331774752

שם המנחה: רימה גרוסמן

שם החלופה: טלפונים חכמים באנדרואיד בשפה java

תאריך הגשה: 18.5.2025



דרישות חובה עבור 5 יח"ל		
3 יח"ל אם לא קיים	שימוש באחד מהנושאים הבאים : AlarmManager + Notification Job Scheduler הורדת נתוני מידע (לא תמונות/ מוזיקה) מהאינטרנט (API של אתר) ושימוש בנתונים שהורדו (דוגמה : שווי מטבע חוץ, רשימת בתי עסק באזור...) Service כתיבת מחלקת שימוש ב - AcvtivityForResult בפורמט החדש שימוש ב ContentProvider (לא כולל שימוש ב- Intenet מרומז) יצירת אנימציה (לא כולל שימוש במחלקת אנימציה) הערה : תלמיד שבחר ביותר מנושא אחד מהרשימה, יוכל לבחור הרחבה אחת בלבד מסעיף 9	6
3 יח"ל אם לא קיים	אחסון וטיפול בנתונים (חובה שמירה ושליפה) לפחות בדרך אחת מהדרכים המפרטות : יש להשתמש בפרויקט בבסיס נתונים (SQLite, MySQL, Firebase, Firestore וכדומה) למען הסר הספק, שימוש ב- SharedPreference , קובץ טקסט, קובץ XML אינו נחשב שימוש בבסיס נתונים. שימוש בבסיס נתונים מרוחק (כגון MySQL, Firestore, Firebase) לכתיבה וגם לקריאה יחשב בנוסף גם כנושא מתקדם אחד.	7
3 יח"ל אם לא קיים	יש לבחור 2 נושאים מתקדמים : אפשרות (1) לבחור שני נושאים (או יותר) מסעיף 9. אפשרות (2) לבחור לפחות נושא אחד מסעיף 9 ושני נושאים (או יותר) מסעיף 10	8
	חיישנים, SurfaceView , ירושה ממחלקת View , אפליקציית רב משתתפים , Fragment , GoogleMaps (לפחות הצגת מיקום ע"י כתובת, נקודות אורך רוחב), מאזיני תנועה, Threads, Bluetooth, NFC , פיתוח ושימוש ב API בצד שרת (אפשר בכל שפת תכנות), בינה מלאכותית, רשתות מחשבים, למידת מכונה, או טכנולוגיה חדשה (כגון עיבוד תמונה)	9
	שימוש של שני נושאים ומעלה ייחשב כנושא מתקדם אחד (סעיף 9). Broadcast Receiver Share Preference Camera & Gallery Async Task Handler Microphone GPS Count Down Timer TextToSpeech SpeechToText	10



תוכן העניינים

3	תוכן העניינים
8	מבוא
8	הרקע לפרויקט
8	תהליך המחקר
9	אתגרים מרכזיים
10	תיאור תחום הידע:
10	אובייקטים נחוצים:
13	סוגי נתונים:
14	ייצוג מידע
15	תיאור פעולות על המידע
17	מבנה/ ארכיטקטורת הפרויקט
17	מסך התחברות-
18	מסך הרשמה-
19	מסכי ניווט (תפריט)-
20	מסך ניווט שני + toolbar –
21	מסך הבית-
22	מסך תפיסת חדר על פי מערכת יומית של החדר –
23	מסך המפה –
24	המסך הראשי של החדרים-
25	מסך תפיסת חדר על פי מערכת
26	מסך החדרים שלי-
27	מסך חיפוש מתקדם-
28	מסך חשבון המשתמש-
29	מסך (PopUp) שינוי אימייל-
30	מסך ניהול הודעות המערכת –
31	מסך ניהול החדרים –
32	מסך יצירת חדר חדש-
33	תרשים היררכיית המסכים והמעברים ביניהם-



34	תרשים UML
39	מימוש הפרויקט
39	Adapters-
39	HourScheduleAdapter-
40	RoomsAdapter
41	RoomViewHolder
42	ScheduleAdapter
44	ScheduleViewHolder
44	HeaderViewHolder
45	SchedualTeachAdapter
47	SchedualViewHolder
47	HeaderViewHolder
48	UserRoomsAdapter
49	UserRoomsViewHolder
50	Data modules
50	Rooms Class
52	Users Class
54	Booking Class
55	Fragments
55	AccountFragment
58	AvailableRoomsFragment
60	DailyScheduleFragment
63	HomeFragment
64	MainRoomsFragment
66	ManageRoomsFragment
68	ManageSchoolNotificationsFragment
69	MapFragment
70	ScaleListener
71	SchoolUpdatesFragment
73	Activities
73	AddRoomActivity



75	AdvSearchActivity
77	InfoAndBookRoomActivity
79	LoginActivity
81	MainActivity
83	MyRoomsActivity
84	RegisterActivity
86	Notification Management
86	NotificationRecivier
87	SetNoti
88	Utilities
88	GetDay
88	GetEmail
89	GetFirstHours
89	GetHour
90	GetIsStaff
91	GetIsT
91	GetName
92	GetProfilePic
93	More
93	PrefManager
93	RoomAdapterInterface
94	מדריך למשתמש
96	רפלקציה
97	בבליוגרפיה
98	נספחים
98	Adapters
98	HourScheduleAdapter
101	RoomsAdapter
104	ScheduleAdapter
108	ScheduleTeachAdapter
112	UserRoomsAdapter



117	Rooms
122	User&Booking
128	Fragments
128	AccountFragment
138	AvailableRoomsFragment
144	DailyScheduleFragment
151	HomeFragment
153	MainRoomsFragment
161	ManageRoomsFragment
166	ManageSchoolNotificationsFragment
171	MapFragment
175	SchoolUpdatesFragment
180	Activities
180	AddRoomActivity
183	AdvSearchActivity
188	InfoAndBookRoomActivity
194	LoginActivity
199	MainActivity
205	MyRoomsActivity
208	RegisterActivity
214	Utilities
214	GetDay
216	GetEmail
218	GetFirstHours
221	GetHour
224	GetIsStaff
226	GetIsT
228	GetName
230	GetProfilePic
233	More
233	PrefManager



235 NotificationReciver
237 SetNoti
241 RoomAdapterInterface
241 (סוף סוף) סוף



מבוא

הרקע לפרויקט

שם הפרויקט: luz

תיאור:

הפרויקט מזהה חדרים ריקים בבית הספר תוך הצלבה עם מערכת השעות של החדרים ורשת מצלמות Ali, הפרויקט גם מאפשר למורים לקבוע חדר חדש במידה והחדר שהם משובצים אליו לא מתאים או שחלק מהאביזרים בו קלוקים. בנוסף לכך, בעתיד יהיו קבוצות למידה שגם מורים יוכלו לקבוע פרטני וגם תלמידים יוכלו ללמוד יחד.

קהל היעד:

נתינת גישה לאפליקציה לבתי ספר תמורת תשלום חודשי, משתמשי האפליקציה יהיו מורים, תלמידים ומזכירי בית הספר.

הסיבות לבחירת הנושא:

בבית ספרנו קיימת הפסקה ארוכה של כ-45 דקות, בזמן ההפסקה הזו, התלמידים נשארים בחוץ כרצונם, אך בתקופות גשומות, קרות או חמות לא נעים להישאר בחוץ ורק אם הם מוצאים חדר ריק אז נכנסים אליו, בחלק מהמקרים אף מחפשים חדר לכל משך ההפסקה. בנוסף קרו הרבה מקרים שבהם אנו נמצאים בשיעור והמקורן או הרמקולים לא עובדים, אי אפשר להשתמש במחשב או שהמחשב עצמו תקול וכתוצאה מכך מערך השיעור נהרס וחייבים לעשות משהו אחר, פחות מעניין ומושך את התלמידים ובכך אנו מאבדים עניין ולומדים באופן לא יעיל. הפרויקט נעשה במסגרת עבודה עם מגמות אחרות בבית ספרנו, במסגרת "מודיס", אני וחברייט בחרנו בפרויקט בגלל הצרכים הנ"ל.

תהליך המחקר

מחקר על תחום ידע:

כחלק מבניית הפרויקט, הייתי חייב ללמוד על המון דברים שמהווים בסיס וחלק חשוב מהפרויקט שלי, לדוגמה, Firebase Auth, Firestore, למדתי אותם דרך המנחה שלי וכשהיו לי בעיות, פניתי גם למורה וגם ניסיתי להסתדר לבד, דרך האינטרנט ו-AI. בנוסף לכך, למדתי המון על פעולות אסינכרוניות, על פרגמנטים, על ממשקים וכו'. כחלק מתהליך הלמידה השתמשתי במגוון מקורות מידע, וביניהם יוטיוב, Stack overflow, מדריכים שונים באינטרנט שהמנחה הפנתה אותנו אליו וסוגי AI שונים.

בדיקת אפליקציות קיימות בשוק & המצב הקיים בשוק:

בשוק הישראלי, האפליקציה הקרובה ביותר לאפליקציה שבניתי היא משוב, שגם היא לא עונה ולו על צורך אחד מהצרכים שהאפליקציה שלי עונה עליהם. אך בשוק הבינלאומי, קיימת אפליקציה אשר עונה על הצורך של סדר היום אשר קיים אצלי רק למורים, וגם האפליקציה הזו לא דומה בכל שאר האספקטים, ולכן, פחות או יותר, האפליקציה שלי תהיה היחידה בשוק, כמובן לתקופה מסוימת עד שיגלו את הקונספט. בשוק קיים צורך לאפליקציה חדשה, רעננה אשר תכניס מימד חדש של ניהול בית הספר



שתשפר את יעילות ואת איכות חיי התלמידים והמורים בזמן בית הספר. הצורך מתבטא בבית הספר שבו אני לומד, כאשר בהפסקות ובשיעורים חופשיים התלמידים מחפשים חדרים להיות בהם או כאשר בתחילת השיעור, המורים מנסים להראות סרטון ולא יכולים. בנוסף לכך, פניתי לכמה מורים בבית ספרי, הסברתי להם על האפליקציה ושאלתי אותם אם הם ישתמשו בה, ורובם ענו שכן.

אתגרים מרכזיים

בעיות אשר התמודדתי איתן במהלך בניית הפרויקט:

היו כמה בעיות בפרויקט אצלי, ובניהן, לחץ בזמנים, עבודה עם סביבה שאינה מוכרת לי (בהתחלה), סידור הפרויקט והפיכתו ליותר יעיל, עבודה עם מבני הנתונים ובחירתם באופן המתאים ביותר למשתנים שאני רוצה לעבוד איתם ובנוסף ההתחשבות בצרכים של המשתמש, ולא רק אם זה עובד אז זה בסדר היו מאתגרים מכיוון שהייתי צריך ממש להיכנס לנעליים של המשתמש ולראות מה נוח ומה פחות ולשנות את האפליקציה לפי הדברים הללו. גם ההתראה על זה שהחדר מתפנה לא עבדה ולקח לי זמן לסדר את זה.

הצרכים שהפרויקט עונה עליהם & הפתרון שהפרויקט בא לתת:

בבתי הספר הגדולים ברחבי הארץ יש בעיה, קיימים המון חדרים והמון כיתות, והחדרים לא מנוצלים באופן מלא, חלק מהחדרים נוחים יותר מאחרים, ולפעמים בחדר מסוים יש דברים שלא עובדים כמו המקרן, המזגן הרמקולים או המחשב עצמו, או אפילו שהחדר לא יכול להכיל את הכיתה במלואה, בעיות אלו משבשות את מהלך השיעור או אפילו יכולות להביא לביטולו לחלוטין, לכן קיים צורך ממשי להחלפת החדר בזריזות, ללא הצורך ללכת למזכירה, ולקוות שהיא שם ולא עסוקה בדבר אחר או לעבור חדר חדר ולראות אם הוא פנוי, צריך פתרון אוטומטי, קל, נוח ומהיר. האפליקציה מביאה את רשימת החדרים הפנויים בזה הרגע ומאפשרת לקבוע שיעור בחדר זה מעכשיו לעכשיו, בנוסף קיימת האופציה לחפש חדר לפי שעות מסוימות, כך שכמעט בוודאות יהיה ניתן למצוא חדר באופן אוטומטי ונוח. בנוסף לכך, בימים חמים או גשומים, התלמידים לא יכולים להיות בחוץ, ולכן האפליקציה מאפשרת לתלמידים לראות איזה חדרים פנויים וללכת אליהם, חשוב לציין כי לתלמידים אין יכולות לקבוע חדר, וגישת התלמידים לאפליקציה תלויה במדיניות בית הספר. האפליקציה גם מהווה מקום לעדכן את תלמידי בית הספר באירועים שונים כגון: "קיימת בגרות במתמטיקה לשכבה י"א בחדרים 26-32 נא לא להפריע", "מתקיים טורניר כדורסל נגד בית הספר נווה נחום באולם ספורט, ולכן שיעורי הספורט עברו למגרש" כך, ברגע שהתלמידים יכנסו לאפליקציה הם יעודכנו בהודעות מטעם בית הספר וכך יהיו פחות אי הבנות והפרעות במבחנים ובאירועים שונים.

פתרונות לבעיה:

הפתרונות שמצאנו לבעיה מסורבלים ומסובכים כגון ללכת ולבדוק את החדרים, להתקשר למורים או למזכירה ולקוות שהיא תענה. או בקיצור להשתמש במערכת ולחפש באופן ידני, לא יעיל ושמבזבז את זמן השיעור. הבנו שצריך פתרון אחר, שונה וחדשני.



תיאור תחום הידע:

אובייקטים נחוצים:

בפרויקט קיימים כמה אובייקטי מפתח אשר משומשים לאורך כל הפרויקט וגם קיימים אובייקטים מקומיים אשר נחוצים לפעולות ספציפיות בלבד, עם השפעה מועטה על האפליקציה בכלליות.

משתמשים:

לאפליקציה קיימים כמה סוגי משתמשים אשר נותנות למשתמש גישות שונות בהתאם לסיווג המשתמש שלו בסיס המשתמש הוא משתמש רגיל, בעל שם, כיתה, סיסמא, מייל ותמונת פרופיל, כולם נשמרים בענן, כסטרינגים (תמונת פרופיל דרך ביטמפ). בנוסף קיימים שדות המייצגים את הרשאת הגישה של המשתמש.

- משתמש רגיל- בעל הגישות הבסיסיות ביותר, עם יכולת לראות את החדרים הפנויים בזה הרגע, רשימה של החדרים הקיימים בבית הספר ואת הזמינות שלהם. בנוסף לכך, המשתמש הרגיל יוכל לראות הודעות של בית הספר ואת מפת בית הספר (להחליט אם להוסיף מפה או לא).

- מורה- בעל כל הגישות של המשתמש הרגיל ובנוסף לכך בעל הגישה לראות את המערכת של החדר, לקבוע שיעור בחדר מסוים בזמן מסוים, מקבל את האופציה לראות את החדרים שקבע ולנהל אותם ויכול לחפש חדרים באופן מתקדם (לפי יום ושעה).

- מזכיר - מקבל את אותם הגישות של תלמיד ובנוסף לכך מקבל את האופציה לנהל את המערכת הרגילה לכל חדר.

משתני User:

- אימייל - המייל של המשתמש (טקסט)
- שם משתמש - השם של המשתמש (טקסט)
- האם הוא מורה (משתנה בוליאני)
- האם הוא מזכיר (משתנה בוליאני)
- כיתה (במידה ואין כיתה נשאר ריק)
- רשימה מסוג Bookings ששומרת את החדרים שהמשתמש תפס.



הרשמות:

לכל משתמש יש רשימה של החדרים שהוא תפס ובאיזה שעות הם, כדי שיוכל למחוק אותם.

משתני ה- Booking :

- מספר חדר (string)
- יום בשבוע (string)
- השעות של החדר (רשימה)

חדרים:

לכל חדר יש כמה משתנים, אשר עוזרים לאפיונו ומשמשים לתפיסתו.

המשתנים:

- המצב בזה הרגע לפי מצלמה, בשיתוף עם מגמת אלקטרוניקה, קיימת מצלמה אשר בודקת אם יש כיתה בחדר כדי שיהיה עוד מקור מידע, ולא נסתמך על המערכת בלבד, משתנה זה משונה ע"י המכשור של אלקטרוניקה והאפליקציה משתמשת בו ומציגה אותו למשתמשים.

- מספר חדר - המספר של החדר הממוספר לפי המערכת של בית הספר (המספר מוכנס ע"י המזכיר)

- טמפרטורה- גם, בשיתוף עם מגמת אלקטרוניקה, בכיתות יהיה חיישן טמפרטורה וכך יהיה ניתן לדעת אם נעים בכיתה בימים חמים או קרים, הטמפרטורה תועלה לענן ומשם הישר לאפליקציה ולמשתמשים.

- מערכת קבועה - לכל חדר יש מערכת קבועה של שיעורים המתרחשים בו. המערכת היא טבלה של 9 שעות (שורה) ו-5 ימים (טורים), בכל ערך רשום אם החדר תפוס או פנוי בשעה זו.
- מערכת משתנה - לכל חדר יש מערכת נוספת אשר משתנה כל שבוע, המערכת נתונה לשינויי המורים והצוות, בכל שבוע חדש המערכת "מתאפסת" למערכת הקבועה. המערכת המשתנה מיוצגת באותה הדרך שהמערכת הקבועה מיוצגת.

סוגי טבלאות:

1. טבלת הרשמה- שומרת את האימייל והסיסמא
2. טבלת משתמש- שומרת אובייקט "user" המכיל את כל הנתונים של המשתמש ואיזה סוג משתמש הוא
3. טבלת חדרים- טבלה השומרת את כל החדרים
4. טבלת נתונים משותפים לבית הספר- טבלה ובתוכה עדכונים שרוצים להעביר לתלמידים ולמורים וסיסמה משותפת למורים כדי שיוכלו להזדהות כמורים.



טבלת הרשמה:

Search by email address, phone number, or user UID					Add user		
Identifier	Providers	Created ↓	Signed In	User UID			
ariellimudim12@gmail....		Jan 15, 2025	May 6, 2025	E9dPHOWyggdKTGke5BCyQ9I...			
					Rows per page: 50	1 - 1 of 1	< >

מייל דרך התחברות User id

טבלת משתמש:

(default)	users	YzGkf1EmggQDQwF1VX4DpxrSg82
+ Start collection	+ Add document	+ Start collection
Rooms	YzGkf1EmggQDQwF1VX4DpxrSg82 >	+ Add field
SharedData		bookedRooms: [] (array) +
users >		class1: "
		email: "ariellimudim12@gmail.com"
		isT: true
		name: "Ariel"
		profilePicture: "
		staff: false

מאגרי הנתונים (במקרה הנוכחי- מאגר המשתמשים)	רשימה של היוזרים (לפי Id)	טבלה אשר מכילה את נתוני המשתמש- שם, מייל, כיתה, רשימה של החדרים שהוא תפס, האם הוא מורה האם הוא חלק מצוות בית הספר צוות ותמונת פרופיל
---	---------------------------	--

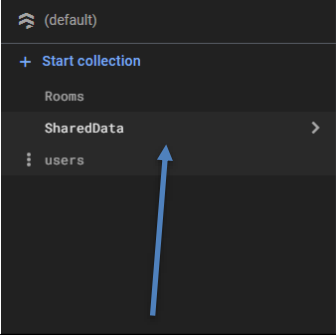
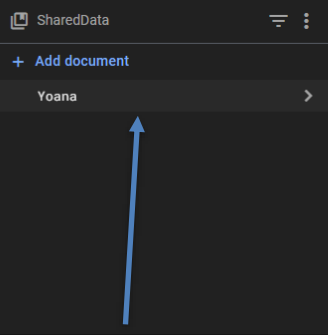
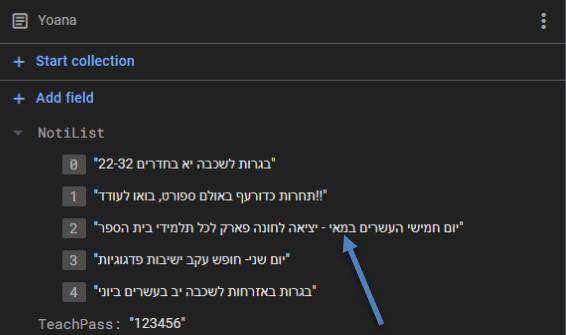
טבלת חדרים:

(default)	Rooms	1
+ Start collection	+ Add document	+ Start collection
Rooms >	1 >	+ Add field
SharedData	2	isOccupied: false
users	3	roomNum: "1"
	4	schedule: {1: ["Available", "Availab...]
	5	temp: "25"
	6	weekSched: {1: ["Available", "Availab...]

מאגרי הנתונים (במקרה הנוכחי- מאגר החדרים)	רשימה של החדרים(לפי מספר חדר)	טבלה אשר מכילה את נתוני החדר- מספר החדר, המערכת השבועית שלו, טמפרטורה בחדר ומערכת קבועה
---	-------------------------------	---



טבלת נתונים משותפים:

		
מאגרי הנתונים (במקרה הנוכחי- מאגר הנתונים המשותפים)	הנתונים המשותפים (הודעות בית הספר וסיסמת הזדהות למורים)	טבלה המכילה את הודעות המערכת ואת סיסמת ההזדהות של המורים

סוגי נתונים:

Array	Boolean	Map	String
NotiList	isOccupied	schedule	roomNum
bookedRooms Array במקרה פה זה של Booking	isT	weekSched	temp
	staff		TeachPass
			class1
			email
			name
			profilePicture



ייצוג מידע

`Schedule&weekSched` – בחרתי בייצוג של מפה שבתוכה יש רשימה. ערכי המפה הם שעות היום וערכי הרשימה הם ימי השבוע. בחרתי בייצוג זה כי הוא נוח מאוד ומתאים לעבודה עם פיירבייס (בניגוד לרשימה בתוך רשימה). בנוסף הבחירה במפה וסטוס כטקסט מאפשרת לשדרג את התפיסות בצורה יותר נוחה (נגיד לשים את השם של המורה שתפס במקום המילה "Occupied"). קשר בין נתונים אלו – כל תחילת שבוע המערכת השבועית "מתאפסת" בחזרה למערכת הקבועה כך שהשינויים מתבצעים על המערכת השבועית.

`isOccupied` – בחרתי בייצוג בוליאני זה כדי שיהיה נוח לתקשר גם עם החלק של אלקטרוניקה, וכי המשתנה לא צריך יותר מתפוס או פנוי (כי משתנה זה מתייחס למצב בזה הרגע).

`IsT&Staff` – בחרתי בייצוג זה כי זה ייצוג פשוט וקל להבנה ולא צריך יותר מזה כדי לדעת אם המשתמש מורה\חלק מצוות בית ספר.

`NotiList` – בחרתי ברשימה פשוטה שבכל ערך יש `String` מכיוון שזה כל מה שצריך (פשוט רשימה של התראות)

`bookedRooms` – בחרתי ברשימה מטיפוס `Booking` מכיוון שהיה צורך במשתנים שונים כדי לאפיין את החדר שהמורה תפס ולאפשר למורה לנהל אותו (משתנים כגון מספר החדר, יום בשבוע ובאיזה שעות ביום הזה). רשימה מטיפוס `Booking` הינה הפתרון הכי יעיל ונוח לאחסן מידע זה.

`RoomNum & temp` – כדי להבטיח גמישות, שמירה על פורמט, ותמיכה בערכים שאינם בהכרח מספריים טהורים.

`TeachPass` – בחרתי בייצוג של טקסט כי סיסמאות חייבות להישמר כStrings מטעמי בטיחות ותמיכה לא רק במספרים

`Class1 & name & email` – נשמרו כStrings כי הם טקסטים.

`profilePicture` – נשמרה כString כי היא ממורת לBitmap ולכן בייצוג כסטרינג הוא חובה.



תיאור פעולות על המידע

פעולות על המידע ←	הוספה	מחיקה	קריאת מצב	עדכון מצב
סוגי משתמשים ↓				
משתמש רגיל (תלמיד)	יכולת להוסיף משתמשים (התלמיד יכול להירשם לאפליקציה כתלמיד), יכולת להוסיף תמונת פרופיל	יכולת למחוק תמונת פרופיל (להפוך אותה לריקה)	יכולת לקרוא את מצב החדרים (לקרוא את הנתונים במחלקה של החדר, חוץ מהמערכת הקבועה) בנוסף יכולת לקרוא את נתוני המשתמש שלו חוץ מהסיסמא. יכול לקרוא את הודעות המערכת.	יכולת לעדכן תמונת פרופיל, אימייל וסיסמא
מורה	יכולת להוסיף משתמשים (המורה יכול להירשם לאפליקציה כמורה) יכולת להוסיף תמונת פרופיל + יכולת להוסיף ערכים ל- bookedRooms	יכולת למחוק תמונת פרופיל (להפוך אותה לריקה) + יכולת למחוק ערכים מ- bookedRooms שנמצאת במשתמש שלו (לא את הרשימה עצמה)	יכולת לקרוא את מצב החדרים (לקרוא את הנתונים במחלקה של החדר, חוץ מהמערכת הקבועה) בנוסף יכולת לקרוא את נתוני המשתמש שלו חוץ מהסיסמא, יכול לקרוא את הודעות המערכת. + יכול לקרוא את החדרים שהוא תפס (bookedRooms).	יכולת לעדכן תמונת פרופיל, אימייל וסיסמא + יכול לעדכן את המערכת השבועית של החדר
צוות	יכולת להוסיף משתמשים (יכול להירשם לאפליקציה כתלמיד ויהפוך לצוות באופן ידני). יכולת להוסיף תמונת פרופיל + יכולת להוסיף חדרים (מספר ולו"ז, כל שאר הערכים נוצרים באופן	יכולת למחוק תמונת פרופיל (להפוך אותה לריקה) + יכולת למחוק הודעות (מ- NotiList) אבל לא את הרשימה עצמה	יכולת לקרוא את מצב החדרים (לקרוא את הנתונים במחלקה של החדר, חוץ מהמערכת השבועית) בנוסף יכולת לקרוא את נתוני המשתמש שלו חוץ מהסיסמא. יכול לקרוא את הודעות המערכת.	יכולת לעדכן תמונת פרופיל, אימייל וסיסמא + יכול לעדכן את המערכת הקבועה של החדר



			אוטומטי) + יכולת להוסיף הודעות מערכת (ל- NotiList)	
--	--	--	--	--

יש לציין כי מנהל המערכת מקבל את גישת המורה וגישת הצוות ולכן הוא יכול לעשות את מה שהם יכולים לעשות.



מבנה/ ארכיטקטורת הפרויקט

מסך התחברות-

המסך הראשון שהמשתמש רואה כאשר הוא נכנס לאפליקציה, בפעם הראשונה שהוא נכנס הוא רואה את זה. במידה והוא מסמן את כפתור ה"Remember Me" המסך הראשון שהוא יראה זה יהיה מסך הבית.

לוגו האפליקציה - `ImageView`

כיתוב אשר רומז למשתמשים להזין פרטים - `TextView`

הזנת אימייל וסיסמא עם כיתוב מנחה - `EditText`

כפתור התחברות לאפליקציה - `Button`

טקסט המנחה את המשתמש לעבור להרשמה במידה ואין לו חשבון - `TextView`

כפתור הרשמה לאפליקציה - `Button`

מקום לסימון "וי" על ידי המשתמש כדי שהמערכת תזכור אותו לפעם הבאה והוא לא יצטרך למלא פרטים שוב + טקסט מנחה - `CheckBox`



מסך הרשמה-

במסך הזה המשתמש מכניס פרטי התחברות
ויוצר חשבון שנשמר במאגר הנתונים

Register

טקסט מנחה להרשמה לאפליקציה-
TextView

Enter your Name

Enter your Email

Enter your Password

Enter Teacher Password

Class

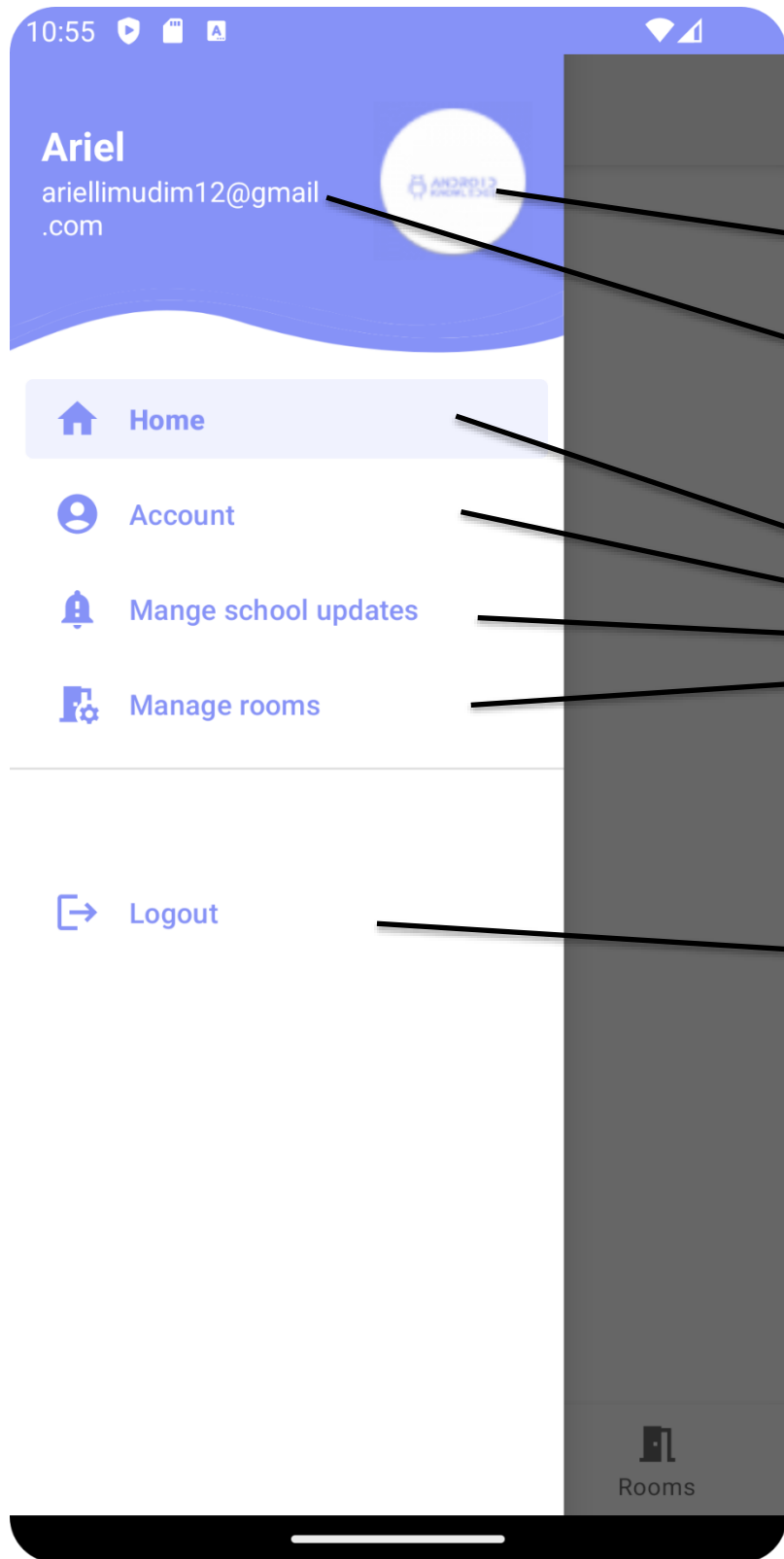
שדות להזנת פרטים להירשמות
לאפליקציה - EditText

REGISTER

כפתור הרשמה, יוצר משתמש,
שומר אותו במאגר הנתונים ומעביר
למסך הבית - Button



מסכי ניווט (תפריט)-



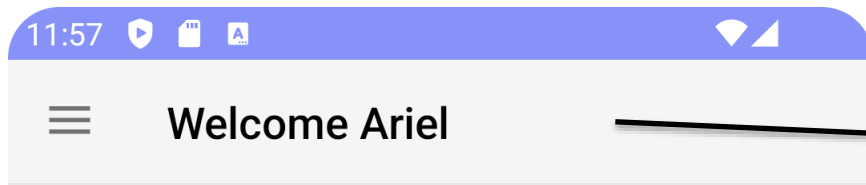
מסכים המאפשרים מעבר בין מסכים שונים של האפליקציה והתנתקות ממנה באמצעות מגירת ניווט וסרגל ניווט.

תמונת פרופיל - `ImageView`

שם ואימייל - `TextView`

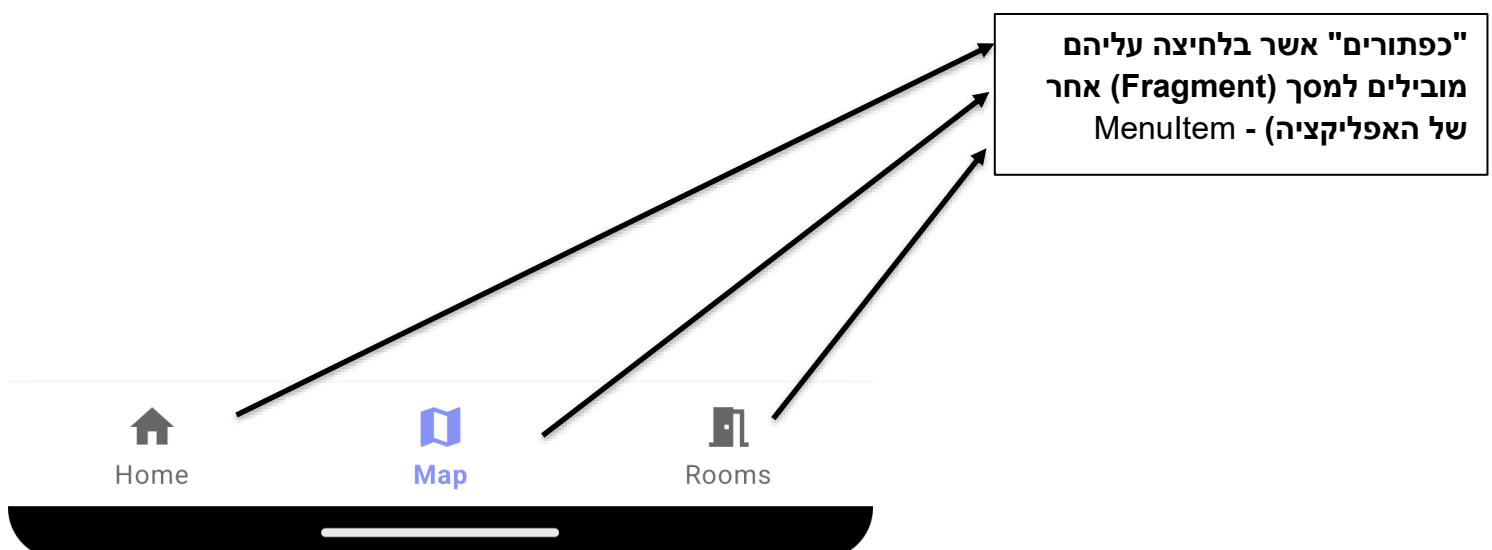
"כפתורים" אשר בלחיצה עליהם מובילים למסך (Fragment) אחר של האפליקציה) - `MenuItem`

"כפתור" אשר בלחיצה עליו מתנתק מהאפליקציה, מוחק את שמירת הנתונים ב- `SharedPreferences` (Remember me) ומעביר את המשתמש למסך ההתחברות - `MenuItem`



מסך ניווט שני + toolbar –

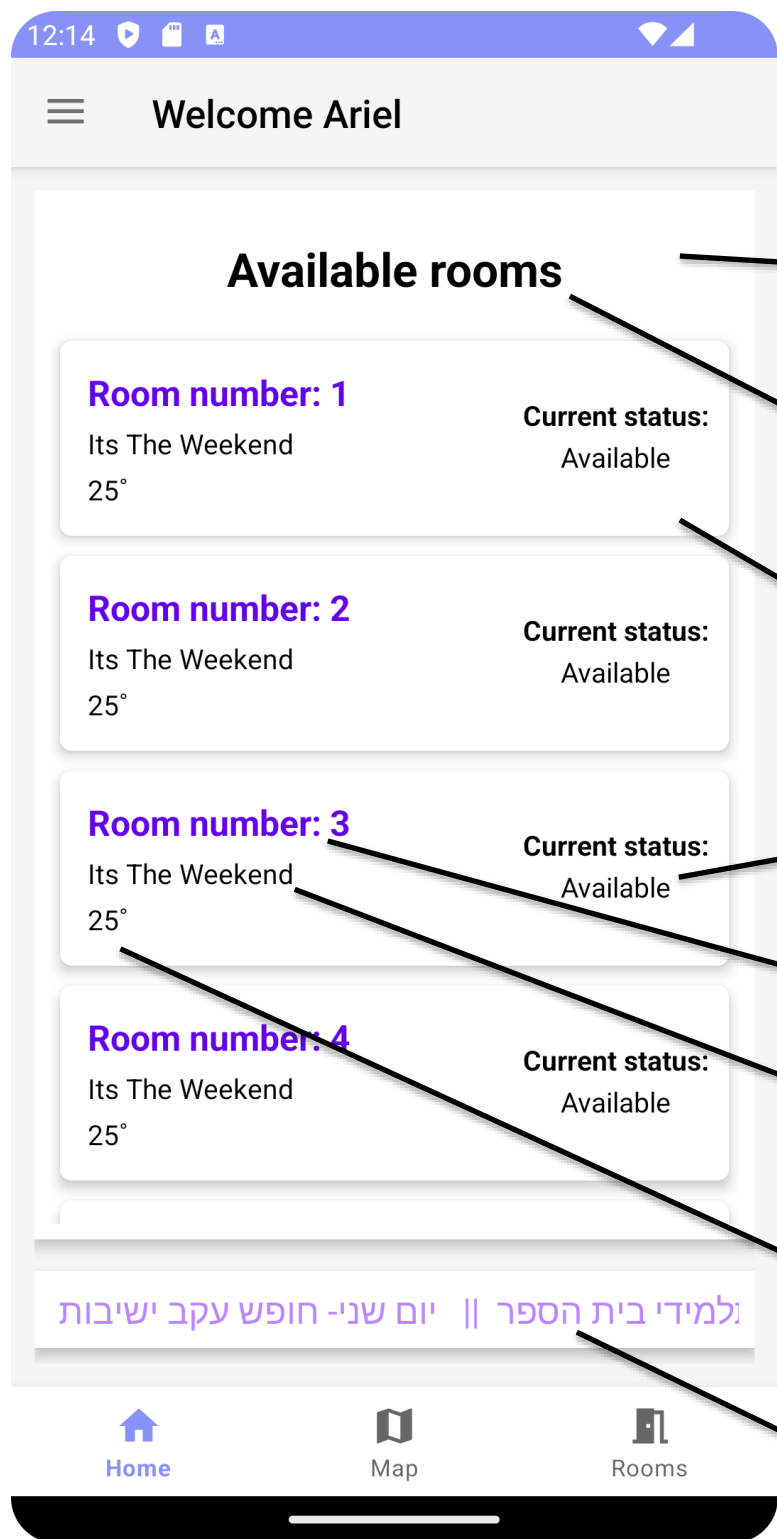
טקסט המכיל את השם של
המשתמש וברכה, בנוסף כפתור
אשר בלחיצה פותח את תפריט
הניווט - Toolbar





מסך הבית-

מסך שמכיל את החדרים הפנויים בזה הרגע
תלמיד וצוות יכולים לראות את החדרים,
בנוסף יש למטה את ההודעות של בית הספר.



מסך המכיל את החדרים הפנויים
בתוך מסך הבית -
Fragment

טקסט שמסביר על מה המשתמש
מסתכל -
TextView

תצוגה נגללת של החדרים הפנויים
בזה הרגע, בלחיצה על אחד
החדרים המשתמש יישלח למסך
תפוסת חדר על פי מערכת יומית -
RecyclerView

טקסט המכיל את המצב הנוכחי של
החדר לפי הנתון שמתקבל ממגמת
אלקטרוניקה וטקסט מנחה -
TextView

טקסט המייצג את מספר החדר -
TextView

טקסט המכיל את המצב של החדר
לפי המערכת שלו, במקרה הזה
בסוף השבוע אין מערכת -
TextView

טקסט המכיל את הטמפרטורה של
החדר לפי הנתון שמתקבל ממגמת
אלקטרוניקה -
TextView

מסך המכיל תצוגה נגללת של
הודעות המערכת -
TextView
שנמצא בתוך
Fragment



12:42

Welcome Ariel

Room number: 3 | Day: Thursday

Hour 1 Available	Hour 2 Available
Hour 3 Available	Hour 4 Available
Hour 5 Available	Hour 6 Available
Hour 7	Hour 8

Select Time

From Hour To Hour

9	1
1	2
2	3

Book the room

Home Map Rooms

מסך תפיסת חדר על פי מערכת יומית של החדר -

מסך שמכיל את המערכת היומית של החדר ונותן אופציה לקבוע את החדר לאותו היום בשעות מסוימות.

טקסטים המציגים את היום ואת מספר החדר שהמשתמש רואה את המערכת שלו - TextView

מסך המכיל תצוגה נגללת של המערכת היומית של החדר (מתוך המערכת השבועית) - RecyclerView

טקסטים המציגים את השעה ביום והאם החדר פנוי או תפוס (מתוך המערכת השבועית) - TextView

טקסטים המנחים את המשתמש לבחור שעה שבה הוא רוצה להשתמש בחדר - TextView

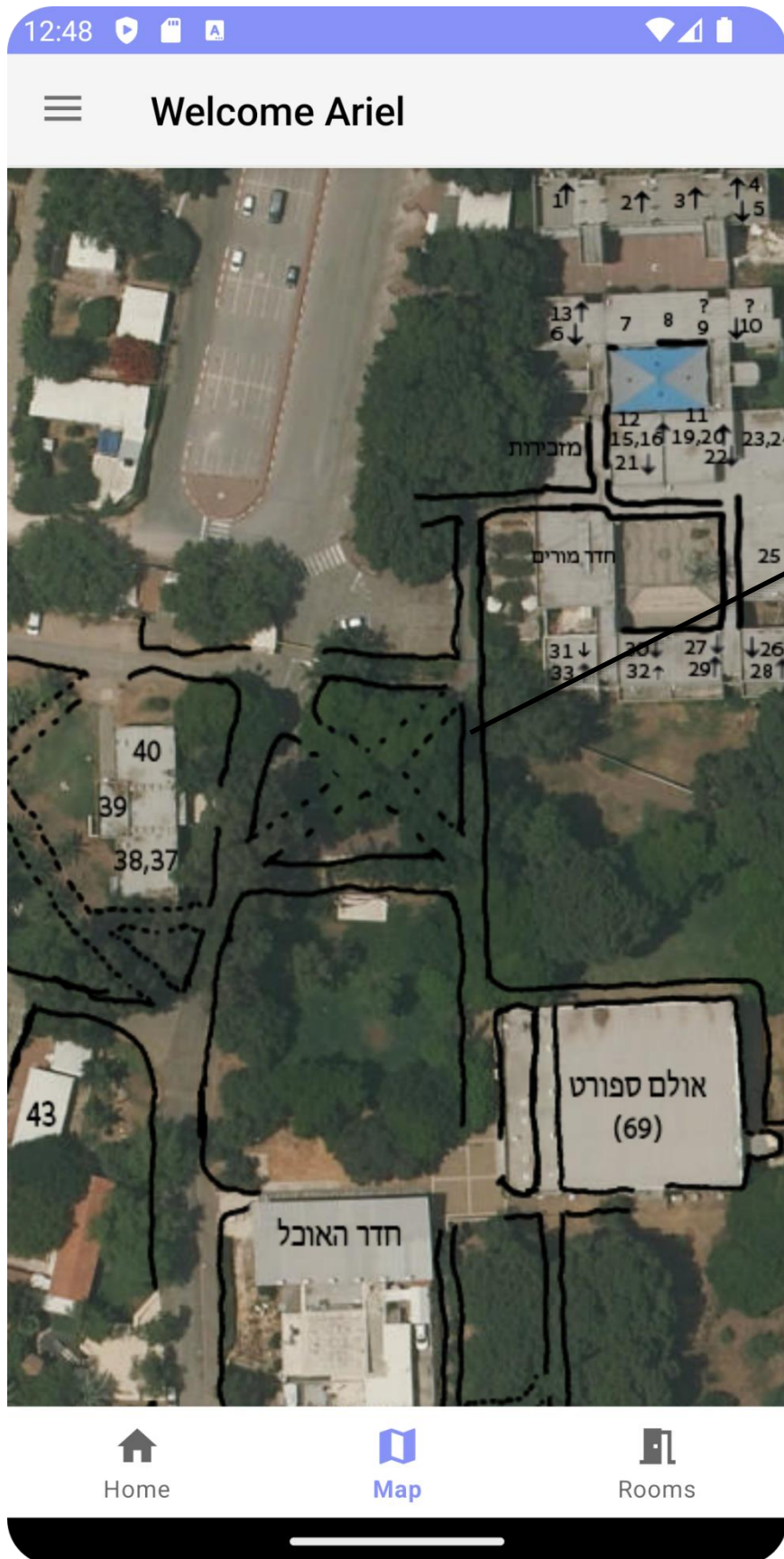
גלגלות המציגות את השעות שבהן המשתמש יכול לתפוס את החדר (אחת המציגה את השעה הראשונה והשנייה מציגה את השעה האחרונה) - NumberPicker

כפתור עם טקסט המנחה את המשתמש לתפוס את החדר, בלחיצה עליו מתעדכן מאגר הנתונים והמצב משתנה לתפוס בשעות שהמשתמש בחר, המשתמש מקבל הודעה במסך שהתפיסה הצליחה - NumberPicker

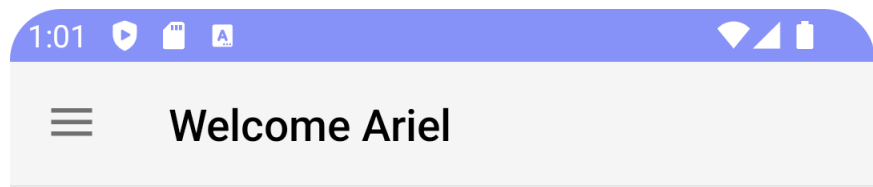


מסך המפה –

מסך המציג את מפת בית הספר,
מאפשר להגדיל אותה, להקטין אותה
ולהזיז אותה בפלאפון.



תמונה של מפה אשר אפשר להגדיל,
להקטין או להזיז אותה על המסך -
ImageView



המסך הראשי של החדרים-

מסך המציג את כל החדרים
כל תנאי ומאפשר חיפוש
לפי מספר חדר.

Rooms

כותרת המסך, מסבירה מה המשתמש
רואה - TextView

Search for rooms

שורת חיפוש עם טקסט מנחה
המאפשרת לחפש חדר לפי מספרו -
SearchView

Room number: 1

The end of the day
25°

Current status:
Available

תצוגה נגללת של החדרים הקיימים
במאגר הנתונים, בלחיצה על אחד
החדרים המשתמש יישלח למסך
תפיסת חדר על פי מערכת שבועית של
החדר - RecyclerView

Room number: 2

The end of the day
25°

Current status:
Available

טקסט המכיל את המצב הנוכחי של
החדר לפי הנתון שמתקבל ממגמת
אלקטרוניקה וטקסט מנחה - TextView

Room number: 3

The end of the day
25°

Current status:
Available

טקסט המייצג את מספר החדר - TextView

טקסט המכיל את המצב של החדר לפי
המערכת שלו, במקרה הזה בסוף היום אין
מערכת - TextView

Room number: 4

The end of the day

Current status:
A

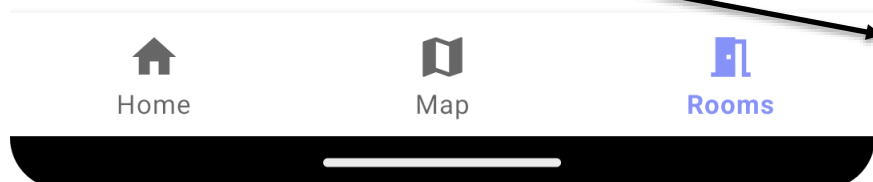
טקסט המכיל את הטמפרטורה של החדר לפי
הנתון שמתקבל ממגמת אלקטרוניקה - TextView

Advanced Search

My rooms

כפתור ששולח את המשתמש למסך החדרים שהוא
תפס כבר - Button

כפתור ששולח את המשתמש למסך חיפוש מתקדם
- Button





1:29

Room number: 1

Hour	Sunday	Monday	Tuesday	Wednesday	Thursday
1	Available	Available	Available	Available	Available
2	Available	Available	Available	Available	Available
3	Available	Available	Available	Available	Available
4	Available	Available	Available	Available	Available
5	Available	Available	Available	Available	Available

מסך תפוסת חדר על פי מערכת

שבועית של החדר-

מסך שמכיל את המערכת
השבועית של החדר ונותן אופציה
לתפוס את החדר לאורך השבוע

טקסט המציג את מספר החדר
TextView - למערכת

כותרת הטבלה, בכל עמודה
קיים טקסט המנחה מה רשום
מתחתיו - TextView

תצוגה נגללת של המערכת של
החדר, מורכבת משורות של
שעות ותפוסות החדר בהן,
בראש התצוגה קיימת כותרת
הטבלה - RecyclerView

בכל שורה יש 5 טקסטים
המייצגים את הזמינות של
החדר בימים מסוימים ובשעה
מסוימת - TextView

טקסט המייצג את השעה - TextView

טקסטים המנחים את המשתמש לתפוס חדר
ביום מסוים ובין שעות מסוימות - TextView

גלגלות המציגות את היום ואת השעות שבהן
המשתמש יכול לתפוס את החדר (אחת
המציגה את השעה הראשונה והשנייה מציגה
את השעה האחרונה) - NumberPicker

כפתור עם טקסט המנחה את המשתמש
לתפוס את החדר, בלחיצה עליו מתעדכן מאגר
הנתונים והמצב משתנה לתפוס בשעות וביום
שהמשתמש בחר, המשתמש מקבל הודעה
במסך שהתפוסה הצליחה - NumberPicker

Book the room



1:59



My Rooms

מסך החדרים שלי-

מסך המציג את רשימה כל החדרים
שהמשתמש תפס ונותן אופציה למחוק
אותם, ובכך להפוך אותם לפנויים

כותרת המסך האומרת למשתמש מה הוא
רואה - TextView

קו המפריד בין הכותרת לתוכן המסך - View

תצוגה נגללת המציגה למשתמש את החדרים
שהוא תפס - TextView

כפתור מחיקת חדר שכאשר לוחצים עליו
מופיע AlertDialog השואל את המשתמש
האם הוא בטוח שהוא רוצה למחוק את החדר,
כאשר הוא מסכים החדר הופך לפנוי במאגר
הנתונים ומופיעה הודעה שהחדר נמחק
בהצלחה אם הוא לוחץ "לא" אז הPopUp
נסגר - ImageButton

טקסט המייצג את מספר החדר - TextView

טקסט המייצג את היום בשבוע שהחדר נתפס
TextView -

טקסט את השעות שהחדר נתפס בהם (ביום
שלפי המשתנה הקודם) - TextView



3:37



Search room

מסך חיפוש מתקדם-

מסך המאפשר לחפש חדרים
לפי יום ושעה, המסך יציג את
החדרים הפנויים ביום ובשעה
שהשתמש בו

Room number: 2

The end of the day
25°

Current status:
Available

TextView - המסך

תצוגה נגללת של החדרים הקיימים
במאגר הנתונים, בלחיצה על אחד
החדרים המשתמש יישלח למסך
תפוסת חדר על פי מערכת שבועית של
החדר - RecyclerView

Room number: 3

The end of the day
25°

Current status:
Available

טקסט המכיל את המצב הנוכחי של
החדר לפי הנתון שמתקבל ממגמת
אלקטרוניקה וטקסט מנחה - TextView

Room number: 5

The end of the day
25°

Current status:
Available

טקסט המייצג את מספר החדר - TextView

טקסט המכיל את המצב של החדר לפי
המערכת שלו, במקרה הזה בסוף היום אין
מערכת - TextView

Room number: 6

Current status:

טקסט המכיל את הטמפרטורה של החדר לפי
הנתון שמתקבל ממגמת אלקטרוניקה - TextView

Select Day and Time

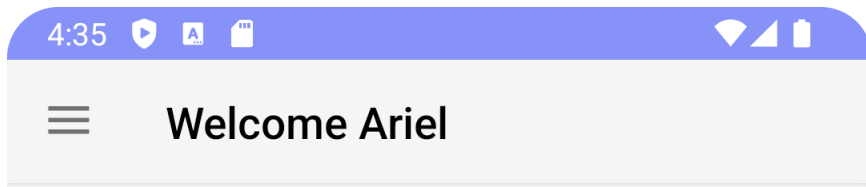
Day	From Hour	To Hour
Thursday	9	1
Sunday	1	2
Monday	2	3

טקסטים המנחים את המשתמש לחפש חדר
ביום מסוים ובין שעות מסוימות - TextView

גלגלות המציגות את היום ואת השעות שבהן
המשתמש יכול לחפש את החדר (אחת
המציגה את השעה הראשונה והשנייה מציגה
את השעה האחרונה) - NumberPicker

כפתור עם טקסט המנחה את המשתמש
לחפש חדר, בלחיצה עליו מתעדכנת הרשימה
ומופיעים החדרים הפנויים בשעות שנבחרו
ביום שנבחר - NumberPicker

Search

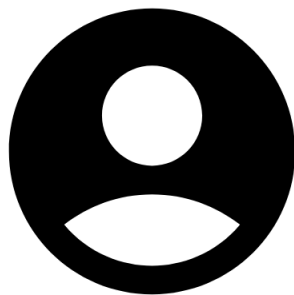


מסך חשבון המשתמש-

מסך שבו המשתמש יכול לצפות בכל הפרטים שלו חוץ מהסיסמא. בנוסף המשתמש יכול לערוך חלק מפרטיו.

My Account

TextView - המסך



ImageView - תמונת הפרופיל

Change Profile Picture

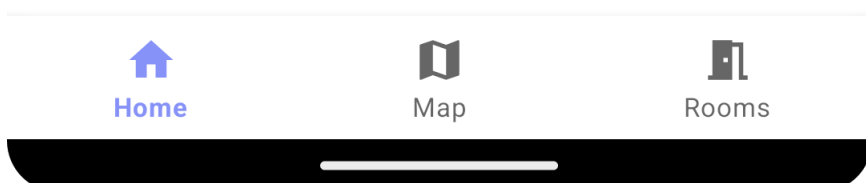
כפתור אשר בלחיצה עליו מוחק את תמונת הפרופיל ממאגר הנתונים
ImageButton -

טקסט אשר בלחיצה עליו פותח אופציה לבחור תמונת פרופיל מהגלריה או לצלם אחת, מעדכן את התמונה במאגר הנתונים -
ImageButton

טקסטים המנחים מה יהיו השדות שאחריהם
TextView -

TextView - הנתונים של המשתמש

כפתורי עריכה שכאשר לוחצים על הראשון והשלישי מופיעה הודעה שצריך לדבר עם ההנהלה כדי לערוך ובשני מופיע PopUp לשינוי אימייל, ברביעי AlertDialog ששואל אם המשתמש בטוח שהוא רוצה לשנות סיסמא, במידה וכן אז נשלח אימייל לשינוי ובמידה ולא אז זה נסגר -
ImageButton





5:04



Welcome Ariel

מסך (PopUp) שינוי אימייל-

מסך המאפשר למשתמש לשנות את
המייל שלו.

My Account

Update Email

New Email Address

Current Password

UPDATE EMAIL

CANCEL

כותרת המסך המנחה מה המשתמש
עושה במסך הזה - TextView

שדה להכנסת המייל החדש עם
טקסט מנחה - EditText

שדה להכנסת הסיסמה עם טקסט
מנחה - EditText

כפתור עם טקסט מנחה אשר בלחיצה
עליו שולח אימייל אימות , מתנתק
מהאפליקציה ועובר למסך
ההתחברות - Button

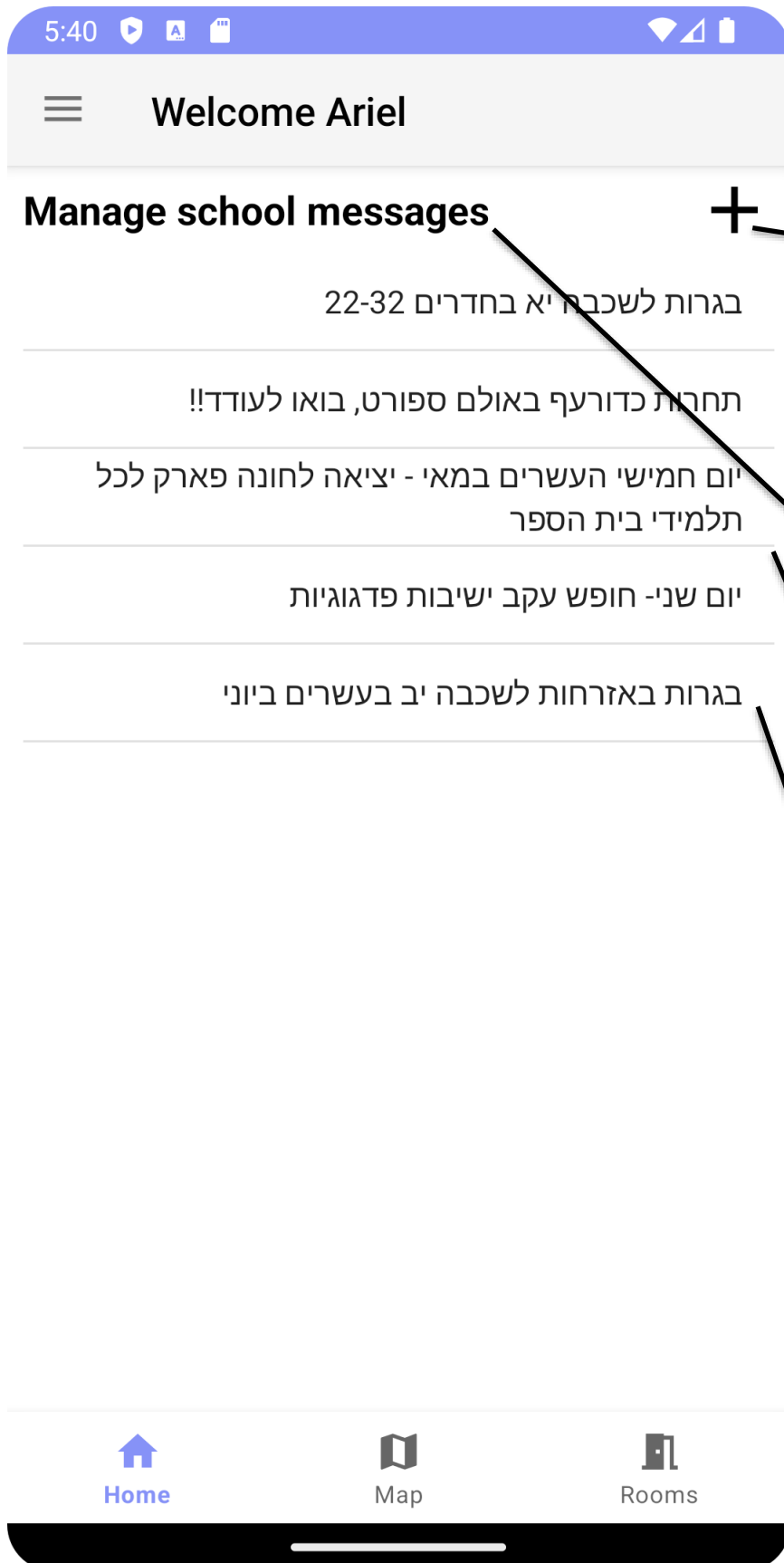
כפתור ביטול, בלחיצה סוגר את
החלון הקופץ - Button

Password: *****

Home

Map

Rooms



מסך ניהול הודעות המערכת –

מסך הפתוח רק לצוות בית הספר
במסך ניתן לראות את הודעות המערכת,
להוסיף הודעות ולמחוק אותם.

כפתור הוספת הודעה, בלחיצה על
הכפתור נפתח **AlertDialog** עם
EditText שבו רושמים את ההודעה
במידה ולוחצים על כפתור ההוספה
בדיאלוג ההודעה מתווספת למאגר
הנתונים והרשימה מתעדכנת . -
ImageButton

כותרת המסך, המצביעה על הפעולות
שעושים במסך - **TextView**

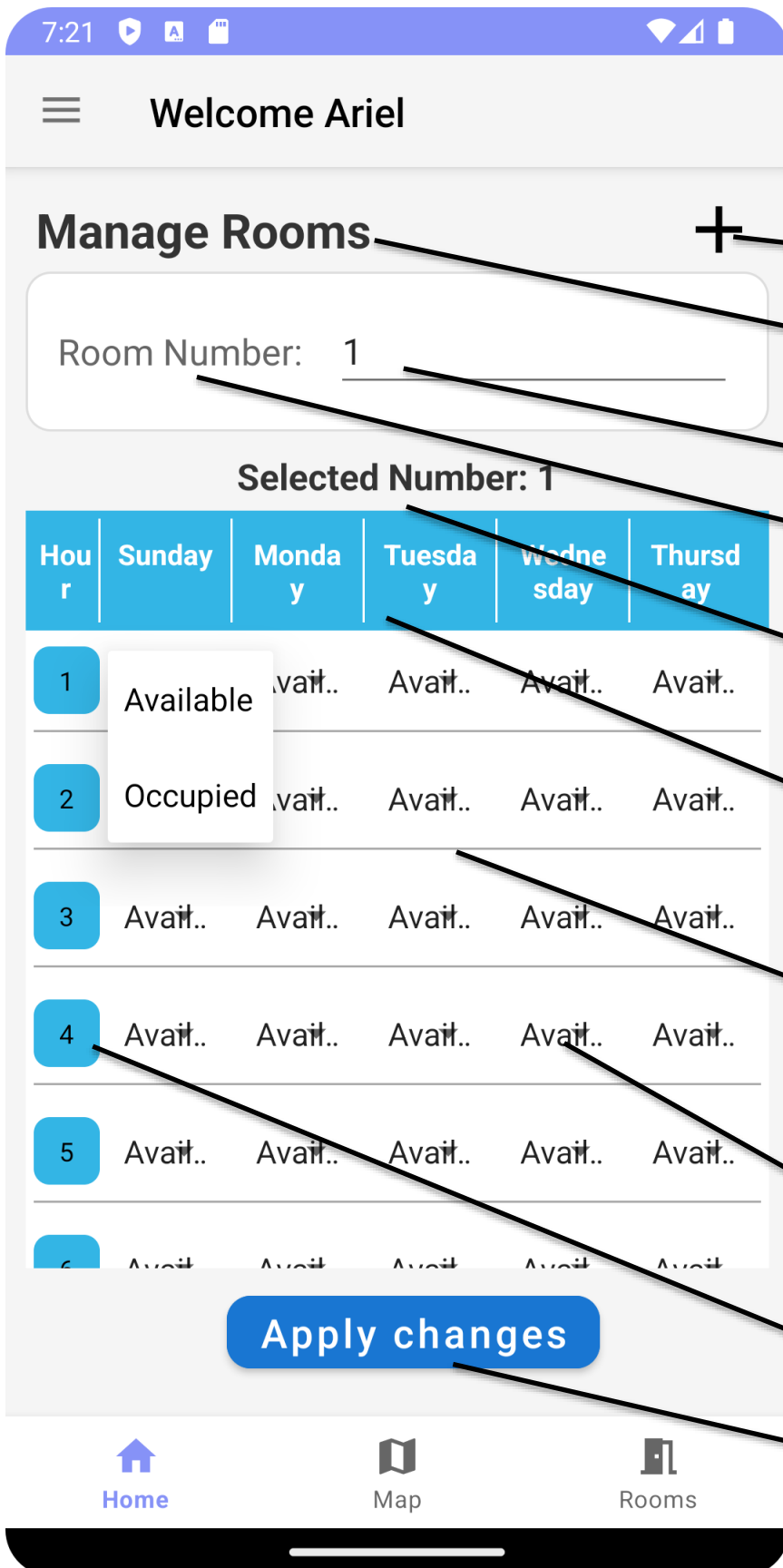
רשימה של הודעות המערכת,
בלחיצה על הודעה נפתח
AlertDialog שמאשר את מחיקת
ההודעה, במידה ומאשרים את
המחיקה ההודעה נמחקת ממאגר
הנתונים והרשימה מתעדכנת -
ListView

הודעת מערכת - **TextView**



מסך ניהול החדרים –

במסך הזה כל מי שמוגדר כצוות יכול לנהל את המערכת הקבועה של החדרים.



כפתור הוספת חדר. מעביר את המשתמש למסך הוספת חדר - Button

כותרת המסך המצביעה על תכלית המסך - TextView

מקום להכנסת מספר חדר שהמשתמש רוצה לערוך את מערכתו - EditText

טקסט מנחה להצגת מערכת החדר - TextView

טקסט המציג את מספר החדר שנבחר - TextView

כותרת הטבלה, בכל עמודה קיים טקסט המנחה מה רשום מתחתיו - TextView

תצוגה נגללת של המערכת של החדר, מורכבת משורות של שעות ותפיסות החדר בהן, בראש התצוגה קיימת כותרת הטבלה - RecyclerView

בכל שורה יש 5 אופציות לבחירת הזמינות של החדר בימים מסוימים ובשעה מסוימת - Spinner

טקסט המייצג את השעה - TextView

כפתור השומר שינויים ומעדכן אותם במאגר הנתונים מעדכן את הרשימה במסך - Button



7:41

Create Room

Enter room number

Edit Schedule (Optional)

Hou r	Sunda y	Monda y	Tuesda y	Wedne sday	Thursd ay
----------	------------	------------	-------------	---------------	--------------

1	Available	Avai..	Avai..	Avai..	Avai..
2	Occupied	Avai..	Avai..	Avai..	Avai..
3	Avai..	Avai..	Avai..	Avai..	Avai..
4	Avai..	Avai..	Avai..	Avai..	Avai..
5	Avai..	Avai..	Avai..	Avai..	Avai..
6	Avai..	Avai..	Avai..	Avai..	Avai..
7	Avai..	Avai..	Avai..	Avai..	Avai..
8	Avai..	Avai..	Avai..	Avai..	Avai..

Create Room

מסך יצירת חדר חדש-

במסך הזה כל מי שמוגדר כצוות יכול להוסיף חדרים וליצור להם מערכת.

כותרת המסך המצביעה על תכלית המסך
TextView -

מקום לכתוב את מספר החדר
שהמשתמש רוצה להוסיף-
EditText

כיתוב מנחה לעריכת המערכת-
TextView

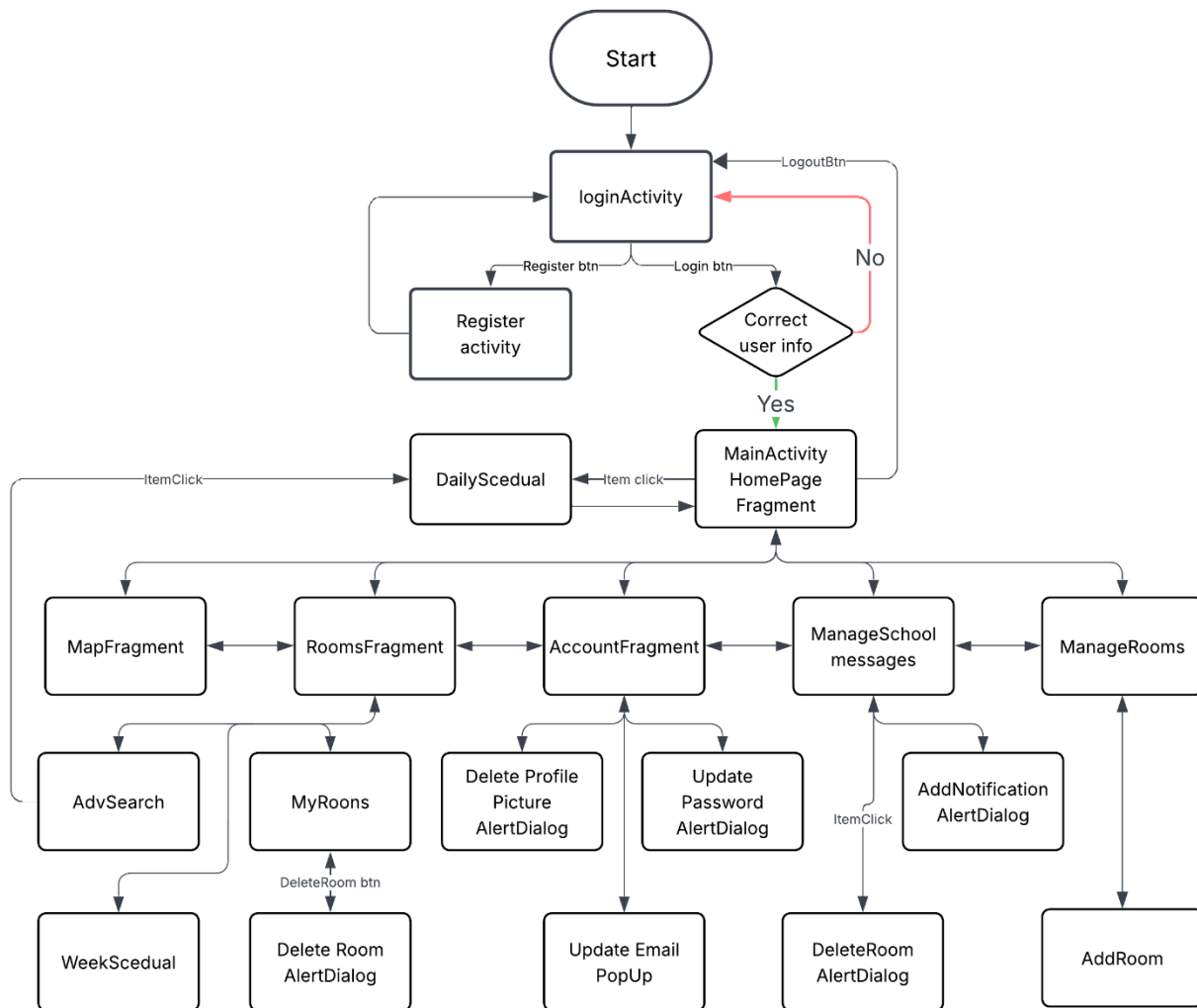
כותרת הטבלה, בכל עמודה
קיים טקסט המנחה מה רשום
מתחתיו
TextView -

תצוגה נגללת של המערכת של
החדר, מורכבת משורות של
שעות ותפיסות החדר בהן,
בראש התצוגה קיימת כותרת
הטבלה
RecyclerView -

בכל שורה יש 5 אופציות
לבחירת הזמינות של החדר
בימים מסוימים ובשעה מסוימת
Spinner -

טקסט המייצג את השעה
TextView -

כפתור היוצר חדר חדש במאגר הנתונים
Button -





תרשים UML

התמונה לא נכנסת ולכן אני אכניס את הקוד, (שמים את הקוד בPlantUML ומקבלים תרשים UML רגיל).

```
@startuml
!pragma layout smetana

package "com.example.projectarieleyalyotamyona" {

    package Adapters {
        interface RoomAdapterInterface {
            + onItemClick(index: int): void
        }

        class HourScheduleAdapter {
            - context: Context
            - scheduleList: List<String>
            + HourScheduleAdapter(context: Context, scheduleList: List<String>)
            + onCreateViewHolder(parent: ViewGroup, viewType: int):
ScheduleViewHolder
            + onBindViewHolder(holder: ScheduleViewHolder, position: int): void
            + getItemCount(): int
            + getScheduleList(): List<String>
        }

        class RoomsAdapter {
            - roomAdapterInterface: RoomAdapterInterface
            - context: Context
            - roomList: List<Rooms>
            + RoomsAdapter(context: Context, roomList: List<Rooms>,
roomAdapterInterface: RoomAdapterInterface)
            + onCreateViewHolder(parent: ViewGroup, viewType: int): RoomViewHolder
            + onBindViewHolder(holder: RoomViewHolder, position: int): void
            + getItemCount(): int
            + getRoomList(): List<Rooms>
        }

        class ScheduleAdapter {
            - context: Context
            - scheduleList: List<Map.Entry<String, List<String>>>
            - recyclerView: RecyclerView
            + ScheduleAdapter(context: Context, scheduleMap: Map<String,
List<String>>, recyclerView: RecyclerView)
            + onCreateViewHolder(parent: ViewGroup, viewType: int):
RecyclerView.ViewHolder
            + onBindViewHolder(holder: RecyclerView.ViewHolder, position: int):
void
            + getScheduleList(): List<Map.Entry<String, List<String>>>
        }

        class ScheduleTeachAdapter {
            - context: Context
            - scheduleList: List<Map.Entry<String, List<String>>>
            + ScheduleTeachAdapter(context: Context, scheduleMap: Map<String,
```



```
List<String>>)+ onCreateViewHolder(parent: ViewGroup, viewType: int):
RecyclerView.ViewHolder
+ onBindViewHolder(holder: RecyclerView.ViewHolder, position: int):
void
+ getScheduleList(): List<Map.Entry<String, List<String>>>
}

class UserRoomsAdapter {
- context: Context
- userRooms: List<User.Booking>
+ UserRoomsAdapter(context: Context, userRooms: List<User.Booking>)
+ onCreateViewHolder(parent: ViewGroup, viewType: int):
UserRoomViewHolder
+ onBindViewHolder(holder: UserRoomViewHolder, position: int): void
+ getUserRooms(): List<User.Booking>
}
}

package Data_Modules {
class Rooms {
- roomNum: String
- schedule: Map<String, List<String>>
- weekSched: Map<String, List<String>>
- temp: String
- isOccupied: boolean
+ getRoomNum(): String
+ getSchedule(): Map<String, List<String>>
+ getWeekSched(): Map<String, List<String>>
+ getTemp(): String
+ isOccupied(): boolean
+ setOccupied(status: boolean): void
}

class User {
- email: String
- name: String
- isT: Boolean
- isStaff: Boolean
- class1: String
- bookedRooms: List<Booking>
+ getEmail(): String
+ getName(): String
+ isT(): Boolean
+ isStaff(): Boolean
+ getClass1(): String
+ getBookedRooms(): List<Booking>
+ addBooking(booking: Booking): void
}

class Booking {
- roomNumber: String
- day: String
- hourStatuses: List<String>
+ getRoomNumber(): String
+ getDay(): String
+ getHourStatuses(): List<String>
}
```



```
}

Rooms --> "0..*" Booking
User --> "0..*" Booking
}

package Fragments {
    class AvailableRoomsFragment {
        - adapter: RoomsAdapter
        - roomList: List<Rooms>
        + onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
        + fetchAvailableRooms(): void
        + setupRecyclerView(): void
    }

    class MainRoomsFragment {
        - adapter: RoomsAdapter
        - roomList: List<Rooms>
        + onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
        + loadMainRooms(): void
        + setupRecyclerView(): void
    }

    class DailyScheduleFragment {
        - adapter: HourScheduleAdapter
        - selectedRoom: Rooms
        + onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
        + displaySchedule(): void
    }

    class ManageSchoolNotificationsFragment {
        - adapter: ArrayAdapter<String>
        + onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
        + loadNotifications(): void
    }

    class MapFragment {
        + onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
    }

    class AccountFragment {
        + onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
    }

    class ManageRoomsFragment {
        + onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
        + manageRoomData(): void
    }

    class SchoolUpdatesFragment {
```



```
+ onCreateView(inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?): View
+ displayUpdates(): void
}
}

package Utilities {
    class GetDay {
        + static getCurrentDay(): String
    }

    class GetEmail {
        + static getEmail(): String
    }

    class GetFirstHours {
        + static getFirstHours(): List<String>
    }

    class GetHour {
        + static getCurrentHour(): String
    }

    class GetIsStaff {
        + static check(): Boolean
    }

    class GetIsT {
        + static check(): Boolean
    }

    class GetName {
        + static getName(): String
    }
}

package Activities {
    class InfoAndBookRoomActivity {
        - scheduleAdapter: ScheduleAdapter
        - scheduleMap: Map<String, List<String>>
        - room: Rooms
        - user: User
        + onCreate(savedInstanceState: Bundle?): void
        + loadSchedule(): void
        + bookRoom(): void
        + setupUI(): void
    }

    class MainActivity {
        - user: User
        + onCreate(savedInstanceState: Bundle?): void
        + setupFragments(): void
        + loadUserData(): void
        + setupDrawer(): void
    }
}
```



```
' Relationships
RoomAdapter ..|> RoomAdapterInterface

RoomAdapter --> Rooms
HourScheduleAdapter --> Rooms
ScheduleAdapter --> Rooms
ScheduleTeachAdapter --> Rooms
UserRoomsAdapter --> User.Booking

AvailableRoomsFragment --> RoomAdapter
MainRoomsFragment --> RoomAdapter
DailyScheduleFragment --> HourScheduleAdapter
InfoAndBookRoomActivity --> ScheduleAdapter

InfoAndBookRoomActivity --> Rooms
InfoAndBookRoomActivity --> User
MainActivity --> User

MainActivity --> GetIsStaff
MainActivity --> GetName
AvailableRoomsFragment --> GetDay
DailyScheduleFragment --> GetHour
}
@enduml
```



מימוש הפרויקט

Adapters-

HourScheduleAdapter-

שם המחלקה:

HourScheduleAdapter

תפקיד המחלקה:

אדפטר עבור RecyclerView שמציג את זמינות החדרים לפי שעות ביום מסוים. כל שורה מציגה שתי שעות עם סטטוס הזמינות שלהן (פנוי/תפוס), לפי רשימה שהועברה מראש.

תכונות המחלקה:

- context

שומר את ההקשר (Context) של האפליקציה. נדרש לצורך ניפוח של תצוגות XML ולשימוש כללי במשאבים.

- scheduleList

רשימת מחרוזות שמכילה את מצב כל שעה ביום) למשל "Available", או "Occupied" במידה והרשימה שהועברה לבנאי היא null, תיווצר רשימה ריקה כדי למנוע קריסה.

ScheduleViewHolder

מחלקה פנימית שמחזיקה רכיבי תצוגה עבור שורה אחת ברשימה. כוללת ארבעה TextView שתי שעות ושני סטטוסים זמינים. כל ViewHolder מציג זוג שעות (או אחת אם אין שנייה).

הבנאי:

יוצר מופע של האדפטר עם ההקשר ורשימת שעות. אם הרשימה ריקה או null, תיווצר רשימה ריקה כדי למנוע שגיאות בזמן ריצה של RecyclerView

onCreateViewHolder:

מנפח את קובץ ה־XML שמייצג שורה בתצוגה ומחזיר מופע חדש של ScheduleViewHolder שמכיל את רכיבי התצוגה.

onBindViewHolder:

מופעל על כל שורה ב־RecyclerView מחשב את האינדקס לפי המיקום, ומכניס את השעה וסטטוס הזמינות לתוך רכיבי התצוגה. אם יש רק שעה אחת (כלומר, אין זוג שני), העמודה השנייה מוסתרת.

getItemCount:

מחזיר את מספר השורות שיש להציג ברשימה. כל שורה מציגה שני פריטים, לכן הפונקציה מחלקת את גודל הרשימה ב־2, עם עיגול כלפי מעלה כדי לטפל במקרה של מספר אי־זוגי.



RoomsAdapter

שם המחלקה: RoomsAdapter

תפקיד המחלקה: אחראית להצגת רשימת חדרים ב RecyclerView - היא מתאימה את הנתונים (כמו מספר חדר, טמפרטורה, מצב תפוסה ולוח זמנים) לכל פריט ברשימה. המחלקה כוללת גם את מנגנון האינטראקציה עם המשתמש (לחיצה על חדר) ועדכון נתונים בזמן אמת.

תכונות המחלקה:

1. context

תכונה מסוג- Context

מאחסן את ההקשר של התכונה (של המחלקה שמשתמשת באדפטר). משמש להחזרת ממשקי משתמש (Views). תחום הכרה בכל המחלקה.

2. roomList

תכונה מסוג- List<Rooms>

תחום הכרה- כל המחלקה

תפקיד- שומרת את כל החדרים שמוצגים ב RecyclerView .

שימוש- נותנת את המידע לכל Item ב RecyclerView

3. RoomAdapterInterface

תכונה מסוג- RoomAdapterInterface

תחום הכרה- המחלקה כולה

תפקיד – ממשק המאפשר אניטרקציה עם ה- Items מגדיר פונקציה onItemClick שימוש- קולט מתי המשתמש לוחץ על Item ומגדיר את הפעולות שמתבצעות בעקבות הלחיצה.

פעולות במחלקה-

1. onCreateViewHolder –

הגדרת התצוגה של ה- holder לפי הxml השייך לו (room_item).

2. onBindViewHolder –

מגדיר את פרטי הItem לפי מקומו ברשימה (מוסיף לholder)

3. getItemCount –

מקבל את כמות הItems שנמצאת בתצוגה

בתוך המחלקה הזו קיימת מחלקה נוספת, פירוט עליה בעמוד הבא.



RoomViewHolder

שם המחלקה: RoomViewHolder

תפקיד המחלקה: יצירת holder בשביל Adaptern , שיחזיק את הערכים של כל חדר

תכונות המחלקה (כולם מסוג TextView ותחום ההכרה של כולם זה כל המחלקה חוץ ממספר 5) –
1. roomNum –

תפקיד/שימוש – מקום השמים בו מאפיינים של ה- xml (מספר חדר במקרה הזה)

2. scheduleStatus –

תפקיד/שימוש – מקום השמים בו מאפיינים של ה- xml (אם החדר פנוי או תפוס בשעה הנוכחית לפי המערכת)

3. temp –

תפקיד/שימוש – מקום השמים בו מאפיינים של ה- xml (טמפרטורת החדר)

4. isOccupied –

תפקיד/שימוש – מקום השמים בו מאפיינים של ה- xml (אם החדר פנוי או תפוס לפי הנתון המתקבל ממגמת אלקטרוניקה (מצלמות ולי)

5. pos –

תכונה מסוג int

תפקיד/ שימוש – שומרת את המקום של item שנלחץ, משמש לבנות את האופציה של הלחיצה על הitem

פעולות במחלקה –

1. RoomViewHolder

יצירת משתנה המחזיק את ה- views לפי הxml

2. onClick

פעולה א-סינכרונית שמקבלת View ומאפשרת למחלקה שמשמשת בholder להגדיר מה

קורה כלוחצים על item



ScheduleAdapter

שם המחלקה: ScheduleAdapter

תפקיד המחלקה: אחראית להצגת מערכת שעות שבועית בתוך RecyclerView לפי layout

תכונות המחלקה –

1. context-

סוג- Context

תחום הכרה: משתנה מופע

תפקיד- מאחסן את ההקשר של המחלקה המשתמשת באדפטר, ומשמש להחזרת תצוגות (Views) ולגישה למשאבים.
שימוש- ניפוח התצוגות ולהגדרת מתאמים (לספינרים)

2. scheduleList –

סוג- List<Map.Entry<String, List<String>>>

תחום הכרה: משתנה מופע

תפקיד- רשימה שבכל מקום בה מיוצגת מערכת השעות כרשימת זוגות מפתח-ערך (שעה ← סטטוסים לימי השבוע)
שימוש - משמש להצגת הנתונים ב-RecyclerView (אחזור והצגה של שעות וסטטוסים לכל יום).

3. recyclerView-

תחום הכרה – משתנה מופע

תפקיד - מאפשר גישה לרכיב ה- RecyclerView שמציג את הנתונים
שימוש- נדרש במתודה getSchedulaData() כדי לשלוף את ה- ViewHolder ולקרוא ממנו את הבחירות של ה-Spinners

4. VIEW_TYPE_HEADER –

סוג- static final int

תחום הכרה – משתנה קבוע במחלקה (סטטי)

תפקיד- מזהה את סוג הפריט בתצוגה כ- header (כותרת)
שימוש- בשיטה getItemViewType() כדי לדעת איזה Layout יש להציג לכל שורה (Header לעומת Item רגיל)

5. VIEW_TYPE_ITEM –

סוג- static final int

תחום הכרה – משתנה קבוע במחלקה (סטטי)

תפקיד- מזהה את סוג הפריט בתצוגה כ- Item (שורה במערכת)
שימוש- בשיטה getItemViewType() כדי לדעת איזה Layout יש להציג לכל שורה (Item לעומת header).



פעולות במחלקה –

1. `onCreateViewHolder` –
הגדרת התצוגה של ה- `holder` לפי ה-`xml` השייך לו (`item_schedule_header`-לראש הכותרת ו-`item_schedule_rv` לכל `Item`).
2. `onBindViewHolder` –
מגדיר את פרטי ה-`Item` לפי מקומו ברשימה (מוסיף ל-`holder`)
3. `setupSpinner` –
מאתחל את הספינרים עם רשימת סטטוסים שנקבעה מראש (`R.array.room_statuses`) ושמה ערך ראשוני
4. `getItemViewType` –
מחזיר את סוג הפריט לפי מיקום, אם 0 אז זו הכותרת (`header`) אם כל מספר אחר זה שורת תוכן (`Item`)
5. `getItemCount` –
מחזירה את מספר הפריטים הכולל (ב-`RecyclerView`) מספר שעות ועוד 1 לכותרת
6. `getScheduleData` –
הפעולה מחזירה את המערכת לאחר שהמשתמש עשה את השינויים הרצויים בה. עוברת על כל ה-`holders` (חוץ מהכותרת), בודקת אם הוא טעון, במידה וכן שואבת את ערכי הספינרים. במידה ולא, שומרת את הערכים המקוריים. מחזירה `Map` של המערכת (שעה -> סטטוסים לכל יום).

במחלקה הזו יש גם מחלקות פנימיות, הסבר עליהן בעמוד הבא.



ScheduleViewHolder

שם המחלקה: ScheduleViewHolder

תפקיד המחלקה: יצירת holder בשביל Adaptern , שיחזיק את הערכים של שורה בודדת ב RecyclerView במקרה הזה- שורה אחת של מערכת השעות.

תכונות המחלקה:

כל התכונות הן בתחום הכרה של משתנה מופע.

1. hourTextView

משתנה מטיפוס TextView המכיל את השעה של ה Item

2. כל שאר התכונות הן ספינרים לכל יום (ראשון עד חמישי) הרנרשמים כ- "day"Spinner החלק שבמרכאות הוא שם היום. הספינרים מייצגים את הזמינות של החדר בכל הימים בשעה מסוימת.

פעולות המחלקה:

1. SchedualViewHolder

יצירת משתנה המחזיק את ה- views לפי הxml ומחברת בין ה-Xml לאדפטר (לערכים holders)

HeaderViewHolder

שם המחלקה: HeaderViewHolder

תפקיד המחלקה: יצירת holder בשביל Adaptern שיחזיק את הערכים של הכותרת

תכונות: ללא תכונות

פעולות במחלקה:

1. HeaderViewHolder

מחברת בין התצוגה של שורת הכותרת לבין מנגנון ה- RecyclerView שמנהל את הרשימה כולה.



SchedualTeachAdapter

שם המחלקה: ScedualTeachAdapter

תפקיד המחלקה: אחראית להצגת מערכת שעות שבועית בתוך RecyclerView לפי layout

תכונות המחלקה:

1. -context
סוג - Context
תחום הכרה: משתנה מופע
תפקיד - מאחסן את ההקשר של המחלקה המשתמשת באדפטר, ומשמש להחזרת תצוגות (Views) ולגישה למשאבים.
שימוש - ניפוח התצוגות ולהגדרת מתאמים (לספינרים)\
2. - scheduleList
סוג - List<Map.Entry<String, List<String>>>
תחום הכרה: משתנה מופע
תפקיד - רשימה שבכל מקום בה מיוצגת מערכת השעות כרשימת זוגות מפתח-ערך (שעה ← סטטוסים לימי השבוע)
שימוש - משמש להצגת הנתונים ב-RecyclerView (אחזור והצגה של שעות וסטטוסים לכל יום).
6. - VIEW_TYPE_HEADER
סוג - static final int
תחום הכרה - משתנה קבוע במחלקה (סטטי)
תפקיד - מזהה את סוג הפריט בתצוגה כ- header (כותרת)
שימוש - בשיטה getItemViewType() כדי לדעת איזה Layout יש להציג לכל שורה (Header לעומת Item רגיל)
7. - VIEW_TYPE_ITEM
סוג - static final int
תחום הכרה - משתנה קבוע במחלקה (סטטי)
תפקיד - מזהה את סוג הפריט בתצוגה כ- Item (שורה במערכת)
שימוש - בשיטה getItemViewType() כדי לדעת איזה Layout יש להציג לכל שורה (Item לעומת header).



פעולות המחלקה:

1. `onCreateViewHolder`
הגדרת התצוגה של ה- `holder` לפי הxml השייך לו (`item_schedule_header`-לראש הכותרת ו- `item_schedule_rv` לכל `Item`).
 2. `onBindViewHolder`
מגדיר את פרטי ה `Item` לפי מקומו ברשימה (מוסיף ל `holder`)
 3. `getItemViewType`
מחזיר את סוג הפריט לפי מיקום, אם 0 אז זו הכותרת (`header`) אם כל מספר אחר זה שורת תוכן (`Item`)
 4. `getItemCount`
מחזירה את מספר הפריטים הכולל (ב- `RecyclerView`) מספר שעות ועוד 1 לכותרת
- במחלקה הזו יש גם מחלקות פנימיות, הסבר עליהן בעמוד הבא.



SchedualViewHolder

שם המחלקה: SchedualVlewHolder

תפקיד המחלקה: יצירת holder בשביל Adaptern , שיחזיק את הערכים של שורה בודדת RecyclerView במקרה הזה- שורה אחת של מערכת השעות.

תכונות המחלקה:

כל התכונות הן מסוג TextView ותחום הכרתן הוא משתנה מופע.

1. hourTextView

תפקיד תכונה זו הוא לשמור את השעה במערכת של הItem

2. "day"TextView

תכונות אלו (יום בשבוע + סוג התכונה לדוגמה- sundayTextView) הן מייצגות את הזמיונות של החדר ביום מסוים בשעה מסוימת.

פעולות המחלקה:

1. SchedualVlewHolder

יצירת משתנה המחזיק את ה- views לפי הxml ומחברת בין ה-Xml לאדפטר (לערכים בholder)

HeaderViewHolder

שם המחלקה: HeaderViewHolder

תפקיד המחלקה: יצירת holder בשביל Adaptern שיחזיק את הערכים של הכותרת

תכונות: ללא תכונות

פעולות במחלקה:

1. HeaderViewHolder

מחברת בין התצוגה של שורת הכותרת לבין מנגנון ה- RecyclerView שמנהל את הרשימה כולה.



UserRoomsAdapter

שם המחלקה: UserRoomsAdapter. (במחלקה זו יש מחלקה פנימית, הסבר עליה בעמוד הבא).

תפקיד המחלקה: אחראית להצגת רשימת הזמנות החדרים של המשתמש בתוך RecyclerView

1. context -

סוג - Context

תחום הכרה: משתנה מופע

תפקיד - מאחסן את ההקשר של המחלקה המשתמשת באדפטר, ומשמש להחזרת תצוגות

(Views) ולגישה למשאבים.

שימוש - ניפוח התצוגות ולהגדרת מתאמים.

2. bookingList

סוג – List<User.Booking>

תחום הכרה: משתנה מופע

תפקיד - מאחסנת את כל ההזמנות של המשתמש

שימוש - משמשת לייצוג ההזמנות בRecyclerView

פעולות המחלקה -

1. onCreateViewHolder

הגדרת התצוגה של ה- holder לפי הxml השייך לו

2. onBindViewHolder

מגדיר את פרטי הitem לפי מקומו ברשימה (מוסיף לholder) ומגדיר את הפעולות הקורות בלחיצה על הכפתור.

3. UpdateBooking

מעדכנת את רשימת ההזמנות החדרים של המשתמש בFirestore

4. UpdateRoomSched

מעדכן את זמינות החדר ברשימת השעות של אותו חדר, כך שיחזור להיות "Available" לאחר מחיקה של תפיסת החדר ע"י המשתמש

5. getDayNumber

ממירה את השם של היום למספר

6. getItemCount

מחזירה את מספר ההזמנות ברשימה (גודל הרשימה). כלומר כמה שורות צריך להציג



UserRoomsViewHolder

שם המחלקה: UserRoomsViewHolder

תפקיד המחלקה- יצירת holder בשביל ה Adaptern , שיחזיק את הערכים של חדר ב RecyclerView.
מחזיקה את הרכיבים הגרפיים מה- item_user_rooms.xml

תכונות המחלקה (תחום הכרה של כולם- משתנה מופע) :

1. roomNum
סוג- TextView
תפקיד/שימוש – מקום השמים בו מאפיינים של ה- xml (מספר חדר במקרה הזה).
 2. status
סוג- TextView
תפקיד/שימוש- מקום השמים בו מאפיינים של ה- xml (אם החדר פנוי או תפוס).
 3. temp
סוג- TextView
תפקיד/שימוש- מקום השמים בו מאפיינים של ה- xml (טמפרטורה של החדר).
 4. trashButton
סוג- ImageButton
תפקיד/ שימוש- כפתור פח אשר בלחיצה מוחק את החדר מהרשימה של החדרים של המשתמש ומעדכן את המערכת.
- פעולות המחלקה-

1. UserRoomsViewHolder
יצירת משתנה המחזיק את ה- views לפי הxml ומחברת בין ה-Xml לאדפטר (לערכים
holder)



Data modules

Rooms Class

שם המחלקה: Rooms

תפקיד המחלקה: הגדרת חדר, שימוש לחלקים רבים בפרויקט, וביניהם אדפטרים ומחלקות.

תכונות המחלקה (תחום הכרה של כל התכונות זה משתנה מופע):

1. roomNum
סוג - String
שימוש - מאפיין את מספר החדר
2. schedule
סוג - Map<String, List<String>>
שימוש - מאפיין את המערכת הקבועה של החדר, בסדר של : שעה ← יום בשבוע(רשימה של 5 ימים וזמינות החדר)
3. weekSched
סוג - Map<String, List<String>>
שימוש - מאפיין את המערכת השבועית (המשתנה) של החדר, בסדר של : שעה ← יום בשבוע(רשימה של 5 ימים וזמינות החדר)
4. temp
סוג - String
שימוש - מאפיין את הטמפרטורה של החדר
5. isOccupied
סוג - boolean
שימוש - מאפיין את זמינות החדר בזה הרגע

פעולות המחלקה:

1. Getters לכל אובייקט
מחזירה את הערך השמור.
2. Setters לכל אובייקט
מגדירה את הערך השמור.
3. Rooms
בנאי למחלקה אשר מגדיר את החדר, הבנאי הזה מקבל את מספר החדר והמערכת.



4. Rooms

בנאי למחלקה אשר מגדיר את החדר, הבנאי הזה מקבל רק מספר חדר ובונה חדר עם זמינות מלאה.

5. toString

פעולה המחזירה String של נתוני החדר

6. deepCopy

פעולה המעתיקה את המערכת הקבועה ל-`Map<String, List<String>>` אחר

7. getDayByIndex

פעולה הממירה יום במספרים לטקסט



Users Class

שם המחלקה: Users

תפקיד המחלקה: הגדרת משתמש, היישור בפירבייס ושימוש לחלקים רבים בפרויקט, וביניהם אדפטרים ומחלקות.

תכונות המחלקה (תחום הכרה של כל התכונות זה משתנה מופע):

1. email

סוג - String

תפקיד - מאחסן את המייל של המשתמש
שימוש - תקשורת עם המשתמש, איפיון, אימות סיסמא.

2. name

סוג - String

תפקיד - מאחסן את השם של המשתמש
שימוש - תקשורת עם המשתמש, איפיון המשתמש.

3. isT

סוג - boolean

תפקיד - משתנה הקובע האם המשתמש הוא מורה או לא
שימוש - במידה והמשתמש הוא מורה אז נפתחות לו אופציות של משתמשים אחרים אין

4. isStaff

סוג - boolean

תפקיד - משתנה הקובע האם המשתמש הוא חלק מצוות בית הספר (מזכיר) או לא
שימוש - במידה והמשתמש הוא מורה אז נפתחות לו אופציות של משתמשים אחרים אין

5. class1

סוג - String

תפקיד - מאחסן את הכיתה של המשתמש, במידה ואין כיתה (מורה או צוות) השדה נשאר ריק
שימוש - איפיון המשתמש

6. bookdRooms

סוג - List<Booking>

תפקיד - מאחסן את רשימת הזמנות החדרים של המשתמש במהלך השבוע.
שימוש - משמש כדי להציג למשתמש את רשימת החדרים שהוא תפס במהלך השבוע ונותן לו את האופציה לנהל אותם.



פעולות המחלקה:

1. Getters לכל אובייקט
מחזירה את הערך השמור.
 2. Setters לכל אובייקט
מגדירה את הערך השמור.
 3. User
בנאי המגדיר את המשתמש ומקבל את כל התכונות חוץ מהאחרונה (רשימת החדרים
שהמשתמש תפס).
 4. User
בנאי ריק בשביל מאגר הנתונים (פיירבייס)
 5. toString
פעולה המחזירה String של נתוני המשתמש.
- במחלקה זו יש מחלקה פנימית, פירוט עליה בעמוד הבא.



Booking Class

שם המחלקה: Booking

תפקיד המחלקה- הגדרת אובייקט של חדר הנתפס ע"י המשתמש, היוצג כחלק מרשימה של חדרים עבור המשתמש

תכונות המחלקה (תחום הכרה של כל התכונות זה משתנה מופע):

1. roomNumber

סוג- String

תפקיד- מאפיין את מספר החדר שהמשתמש תפס

שימוש- משמש להצגת החדר שהמשתמש תפס למשתמש בRV

2. day

סוג- String

תפקיד- מאפיין את היום שהמשתמש תפס

שימוש- משמש להצגת היום שהמשתמש תפס במערכת החדר למשתמש בRV

3. hourStatuses

סוג- List<String>

תפקיד- רשימה המכילה את השעות שהמשתמש תפס בהן את החדר

שימוש- משמש להצגת השעות שהמשתמש תפס במערכת החדר למשתמש בRV

פעולות המחלקה:

1. Getters לכל אובייקט

מחזירה את הערך השמור.

2. Setters לכל אובייקט

מגדירה את הערך השמור.

3. toString

פעולה המחזירה String של נתוני המחלקה.



Fragments

AccountFragment

שם המחלקה: AccountFragment

תפקיד: אחראית על ניהול והצגת פרטי המשתמש במסך החשבון של האפליקציה

תכונות המחלקה:

1. PICK_IMAGE_REQUEST
סוג - static final int
תפקיד - משמש כקוד זיהוי (Request Code) עבור קריאה לפעולה חיצונית – בחירת תמונה מהגלריה או צילום עם מצלמה.
שימוש - כאשר המשתמש בוחר או מצלם תמונה כדי לזהות את התוצאה שחוזרת מהגלריה או המצלמה
2. profilePicture
סוג - ImageView
תפקיד - מציגה את תמונת הפרופיל של המשתמש במסך החשבון
שימוש - משמשת בכל הפעולות על תמונת הפרופיל.
3. textName
סוג - TextView
תפקיד - מציגה את השם של המשתמש שנשמר במסד הנתונים (Firestore) בתוך מסך החשבון.
שימוש - בעת טעינת פרטי המשתמש מה-Firebase.
4. textEmail
סוג - TextView
תפקיד - מציגה את מייל המשתמש שנשמר במסד הנתונים (Firestore) בתוך מסך החשבון.
שימוש - בעת טעינת פרטי המשתמש מה-Firebase.
5. textClass
סוג - TextView
תפקיד - מציגה את כיתת המשתמש שנשמרה במסד הנתונים (Firestore) בתוך מסך החשבון.
במידה ואין כיתה השדה יהיה ריק
שימוש - בעת טעינת פרטי המשתמש מה-Firebase



6. `delProfilePicture`

סוג- `ImageButton`

תפקיד- כפתור למחיקת תמונת הפרופיל של המשתמש מהמסך ומהמסד
שימוש- כאשר המשתמש לוחץ על כפתור מחיקת תמונת הפרופיל – הוא משמש להפעלת
חלונת אישור ולביצוע המחיקה ממאגר הנתונים

7. `mAuth`

סוג- `FirebaseAuth`

תפקיד- מנהלת את תהליך האימות עם `Firebase`, ומאפשרת גישה למידע על המשתמש
המחובר.
שימוש- שינוי פרטים וקבלת מידע על המשתמש.

8. `db`

סוג- `FirebaseFirestore`

תפקיד- מאפשר אינטראקציה עם `Firestore` לקריאה וכתובה של נתוני המשתמש במסד
הנתונים
שימוש- משמשת לגישה למסד הנתונים `Firestore` כדי לקרוא ולעדכן מידע של המשתמש

9. `currentUser`

סוג- `FirebaseUser`

תפקיד- מחזיק את פרטי המשתמש הנוכחי המחובר לאפליקציה
שימוש- משמש לאחסן את המשתמש המחובר כרגע, (ה- `user` שהתחבר דרך `Firebase`),
כדי לגשת למידע האישי שלו או לבצע פעולות הקשורות אליו.

פעולות המחלקה:

1. `OnCreateView`

אתחול ממשק המשתמש של הפרגמנט על פי קובץ ה- `fragment_account` הפעולה קושרת את
רכיבי ה- `UI` כמו תמונת פרופיל, טקסטים וכפתורים לקוד, מאתחלת את `Firebase Auth` ו-
`Firestore` טוענת את נתוני המשתמש מהענן, ומגדירה האזנה לפעולות מסוימות.

2. `loadUserData`

טוענת את פרטי המשתמש מתוך `Firestore` למסך – כולל שם, מייל, כיתה ותמונת פרופיל (אם
קיימת). המידע מוצג בטקסטים והתמונה עוברת המרה מ- `Base64` ל- `Bitmap` אם המשתמש
לא תלמיד – יוצג טקסט חלופי.

3. `editField`

פונקציה כללית שנקראת כשמנסים לערוך שם, כיתה וכו'. במקום לאפשר עריכה, מוצגת הודעה
למשתמש לפנות להנהלת בית הספר – כלומר, הפעולה מונעת עריכה עצמאית באפליקציה.



4. `showPopup`
מציגה חלון קופץ (`PopupWindow`) כשמשתמש רוצה לשנות כתובת מייל. בחלון יש שדה למייל חדש, שדה לסיסמה נוכחית וכפתורי אישור וביטול.
5. `updateEmail`
מתבצעת מתוך החלון הקופץ. מאמתת מחדש את המשתמש עם הסיסמה שהוזנה, ואם האימות הצליח – שולחת למייל החדש קישור לאישור. המשתמש מתנתק אוטומטית ונדרש להתחבר מחדש כדי שהשינוי ייכנס לתוקף.
6. `resetPassword`
פותחת חלונית אישור לשליחת מייל איפוס סיסמה. אם המשתמש מאשר, נשלחת אליו הודעת מייל מ- `Firebase` עם קישור לשחזור סיסמה.
7. `openImageChooser`
פותחת בוחר תמונות שמאפשר למשתמש לבחור תמונה חדשה – או דרך הגלריה, או לצלם חדשה במצלמה. השימוש נעשה באמצעות `IntentChooser`
8. `onActivityResult`
מטפלת בתוצאה שהתקבלה מבחירת תמונה. אם התמונה נבחרה בהצלחה – מציגה אותה בפרופיל, מעלה ל- `Firestore` ומעדכנת גם את התמונה הראשית ב `Navigation Drawer`
9. `decodeBase64ToImage`
פונקציה פנימית שממירה מחרוזת `Base64` לתמונה מסוג `Bitmap`. נועדה לשחזור תמונה שנשמרה בפורמט טקסטואלי במסד הנתונים.
10. `encodeImageToBase64`
הופכת תמונת `Bitmap` למחרוזת `Base64` כדי שניתן יהיה לשמור אותה במסמך המשתמש ב `FireStore`
11. `uploadImageToFirestore`
שומרת את התמונה (כ-`Base64`) במסמך המשתמש תחת השדה `profilePicture` אם הצליח – מוצגת הודעת הצלחה, אחרת הודעת שגיאה.



AvailableRoomsFragment

שם המחלקה: AvailableRoomsFragment

תפקיד המחלקה: להציג למשתמש את רשימת הכיתות הפנויות באותו הרגע, לפי המידע המתקבל ממאגר הנתונים.

תכונות המחלקה:

1. `recyclerView`
סוג- `RecyclerView`
תפקיד: רכיב תצוגה שמציג את רשימת הכיתות הפנויות בצורה גלילה.
שימוש: מקושר ל- `RecyclerView` מתוך קובץ ה- `XML` ומציג את הנתונים מה `Adapter`.
2. `adapter`
סוג- `RoomsAdapter`
תפקיד: מתאם (`Adapter`) שמקשר בין הנתונים (`roomList`) לבין תצוגת ה-`RecyclerView`.
שימוש: מנהל את הצגת כל כיתה פנויה כרשומה נפרדת בתוך הרשימה.
3. `roomList`
סוג- `List<Rooms>`
תפקיד: רשימת הכיתות הזמינות, המכילה אובייקטים מסוג `Rooms`
שימוש: מאוחסנים בה הנתונים שמגיעים מ- `Firebase` היא מתעדכנת ומתואמת מול ה-`Adapter`.
4. `progressBar`
סוג- `ProgressBar`
תפקיד: מציג חיווי טען (`loading`) בעת שליפת נתונים מ-`Firebase`.
שימוש: מוצג בזמן טעינת הכיתות ומוסתר לאחר סיום הקריאה.

פעולות המחלקה:

1. `onCreateView`
יצירת התצוגה של ה- `Fragment` מקשר בין קובץ ה- `XML` לבין הקוד, מאתחל את ה-`RecyclerView`, את רשימת הכיתות ואת ה- `ProgressBar` ומכין את האדפטר שיציג את הכיתות הפנויות.
2. `populateRoomList`
פונקציה שמביאה את כל הכיתות מ- `Firestore` ובודקת עבור כל כיתה אם היא פנויה בשעה וביום הנוכחיים. אם כן – מוסיפה אותה לרשימה שתוצג למשתמש. בנוסף, מעדכנת את האדפטר ומסתירה את ה-`ProgressBar`.



3. OnItemClickListener

מופעלת כשמורה לוחץ על חדר מהרשימה. בודקת אם המשתמש הוא מורה, ואם כן – מעבירה אותו למסך המציג את מערכת השעות היומית של אותו החדר, תוך העברת אובייקט החדר כ-

Serializable

4. onResume

פונקציה שמופעלת אוטומטית כשחוזרים למסך הזה. היא מנקה את הרשימה הקודמת של הכיתות ומרעננת אותה ע"י קריאה חוזרת ל-populateRoomList.



DailyScheduleFragment

שם המחלקה: DailyScheduleFragment

תפקיד המחלקה: להציג את לוח הזמנים היומי של כיתה שנבחרה, ולאפשר למורה להזמין את הכיתה בשעות מסוימות לפי היום הנוכחי.

תכונות המחלקה:

1. adapter

וג: HourScheduleAdapter

תפקיד: מנהל את התצוגה של לוח השעות ברשימה (RecyclerView) שימוש: משויך ל- RecyclerView כדי להציג את זמינות הכיתה לפי שעות ביום הנוכחי

2. recyclerView

וג: RecyclerView

תפקיד: מציג את השעות (1–9) של היום הנוכחי ואת הסטטוס שלהן ("Available"/"Occupied") שימוש: נטען עם הנתונים דרך ה- adapter ומעודכן לאחר כל הזמנה

3. day

וג: String

תפקיד: שומר את שם היום הנוכחי שימוש: מוצג למשתמש ומנחה את המשתמש.

4. id

וג: String

תפקיד: מזהה המשתמש המחובר לאפליקציה שימוש: משמש לעדכון פרטי ההזמנה של הכיתה ושל המשתמש

5. db

וג: FirebaseFirestore

תפקיד- מאפשר אינטראקציה עם Firestore שימוש- משמשת לגישה למסד הנתונים Firestore לעדכון פרטי ההזמנה של הכיתה ושל המשתמש



6. hourFromPicker & hourToPicker

סוג: NumberPicker

תפקיד: רכיבי בחירה לשעות התחלה וסיום להזמנה

שימוש: מאפשרים למורה לבחור טווח שעות פנוי להזמנת החדר

7. selectedRoom

סוג: Rooms

תפקיד: מייצג את החדר שנבחר לצפייה והזמנה

שימוש: משמש להצגת ועדכון הנתונים של החדר הזה ולתפיסתו.

8. bookRoomBT

סוג: Button

תפקיד: כפתור שמפעיל את תהליך ההזמנה של החדר

שימוש: בלחיצה מבצעת בדיקת זמינות, עדכון Firestore ורישום ההזמנה.

9. user

סוג: FirebaseAuth

תפקיד- מחזיק את פרטי המשתמש הנוכחי המחובר לאפליקציה

שימוש- משמש לאחסון את המשתמש המחובר כרגע, (ה־ user שהתחבר דרך Firebase),

וכדי לשייך את ההזמנה לחשבון הנכון במאגר הנתונים (לרשימת ההזמנות של המשתמש).

10. mAuth

סוג: FirebaseAuth

תפקיד- מנהלת את תהליך האימות עם Firebase, ומאפשרת גישה למידע על המשתמש

המחובר.

שימוש- באמצעותו מאוחזר האובייקט user וה־ UID שלו

פעולות המחלקה בעמוד הבא



פעולות המחלקה:

1. `onCreateView`
טעינת קובץ ה XML של הפרגמנט והגדרת רכיבי הממשק הבסיסיים כמו `RecyclerView` וטעינת אובייקט החדר שהועבר דרך `bundle`
2. `onViewCreated`
אתחול מלא של רכיבי הפרגמנט: מציג את שם החדר והיום, מגדיר את `NumberPickers` מציג את לוח השעות, ומטפל בלחיצה על כפתור ההזמנה.
3. `setUpRecyclerView`
יוצר רשימת זמינות לפי שעות עבור היום הנוכחי ומחבר אותה ל `RecyclerView` דרך מתאם `HourScheduleAdapter`
4. `SetUpRoom`
מאפשרת לעדכן את תצוגת לוח השעות עבור חדר שהוזן כפרמטר (למשל אחרי עדכון נתונים).
5. `performBooking`
מבצעת את ההזמנה (מעדכנת את מה שצריך במאגר הנתונים ומודיעה למשתמש)



HomeFragment

שם המחלקה: HomeFragment

תפקיד המחלקה: מסך הבית של האפליקציה, טוען 2 פרגמנטים שונים, הראשון זה להודעות המערכת והשני זה לחדרים פנויים.

תכונות המחלקה:
אין.

פעולות המחלקה:

1. onCreateView

טוענת את 2 הפרגמנטים ומנפחת את ה- layout

2. loadFragment

פעולה זו אחראית על טעינת שני פרגמנטים (תצוגות פנימיות) אל תוך רכיבי התצוגה המתאימים בתוך קובץ ה- XML של HomeFragment



MainRoomsFragment

שם המחלקה: MainRoomsFragment

תפקיד המחלקה: אחראית על הצגת מסך מרכזי שבו ניתן לראות את כל הכיתות בבית הספר. בנוסף המחלקה משמשת כמעבר לחדרים שהמשתמש תפס ולחיפוש מתקדם.

תכונות המחלקה:

1. recyclerView
סוג- RecyclerView
תפקיד: מציג רשימת חדרים בצורה נגללת
שימוש: מעדכן את התצוגה ב- RecyclerView בהתאם לנתונים (כולל חיפוש)
2. roomList
סוג- List<Rooms>
תפקיד: מכיל את כל החדרים שנשלפו מ-Firebase
שימוש: משמש כמאגר נתונים עיקרי של כל הכיתות שנטענו
3. filteredList
סוג- List<Rooms>
תפקיד: מאפשר למשתמש לחפש חדרים לפי מספר כיתה
שימוש: מציג רק את החדרים הרלוונטיים ל- SearchView בתוך ה- RecyclerView
4. searchView
סוג- SearchView
תפקיד: מאפשר למשתמש לחפש חדרים לפי מספר כיתה
שימוש: בעת הזנת טקסט, הרשימה מסוננת באמצעות filterRooms()
5. myRoomsBt
סוג- Button
תפקיד: כפתור המעביר את המשתמש לדף "החדרים שלי"
שימוש: מפעיל Intent ל- MyRoomsActivity כאשר נלחץ
6. AdvSearchBt
סוג- Button
תפקיד: כפתור המעביר את המשתמש לדף "חיפוש מתקדם"
שימוש: מפעיל Intent ל- AdvSearchActivity כאשר נלחץ



7. linearLayoutBt

סוג- LinearLayout

תפקיד: עוטף את כפתורי המעבר למסכים אחרים

פעולות של המחלקה:

1. onCreateView

מופעלת כאשר המסך נטען. מאתחלת את כל הרכיבים הגרפיים כמו רשימת הכיתות (RecyclerView), כפתורים (לכיתות שלי ולחיפוש מתקדם), ומנגנון החיפוש. גם מבצעת בדיקה אם המשתמש הוא מורה – כדי להחליט אם להציג כפתורים מסוימים.

2. populateRoomList

שולפת את כל הכיתות ממסד הנתונים (Firestore), יוצרת עבורן אובייקטים מסוג Rooms ומעדכנת את רשימת התצוגה בהתאם.

3. setupSearchView

מגדירה מה יקרה כאשר המשתמש מקליד בשדה החיפוש – שגורם להפעלת סינון אוטומטי של הרשימה.

4. filterRooms

מסננת את רשימת הכיתות לפי מחרוזת החיפוש, כך שרק כיתות שמתאימות לטקסט יופיעו למשתמש.

5. Show

בודקת האם המשתמש המחובר הוא מורה לפי נתונים ממסד הנתונים. מחזירה ערך בוליאני (true/false) בהתאם, ומציגה הודעות במידת הצורך.

6. OnItemClick

מתבצעת בעת לחיצה על כיתה מתוך הרשימה. אם המשתמש הוא מורה וביום לימודים – ייפתח מסך עם פרטי הכיתה ואפשרות להזמנה.

7. onResume

מופעלת בכל פעם שחוזרים למסך. מנקה את הרשימות הקיימות ומבצעת טעינה מחדש של כל הכיתות מהמסד כדי להציג מידע עדכני.



ManageRoomsFragment

שם המחלקה: ManageRoomsFragment

תפקיד המחלקה: משמשת כממשק לניהול מערכת השעות הקבועה של חדרים בבית הספר (המערכת פתוחה רק לבעלי הגישה המתאימה (מי שחלק מצוות בית הספר).

תכונות המחלקה:

1. `selectedNumber`
סוג: `int`
תפקיד: מייצג את מספר החדר שנבחר כעת, ברירת המחדל היא חדר מספר 1.
שימוש: משמש לשליפת והצגת מערכת השעות של החדר הרלוונטי כאינדקס לאיזה חדר לשלוף.
2. `addRoom`
סוג: `ImageButton`
תפקיד: כפתור שמאפשר למשתמש לפתוח את מסך הוספת חדר חדש.
שימוש: בלחיצה עליו נפתח מסך הוספת חדר (`AddRoomActivity`)
3. `commitBt`
סוג: `MaterialButton`
תפקיד: כפתור לשמירת שינויים שנעשו בלוח השעות של החדר הנבחר.
שימוש: לעדכון מערכת השעות של החדר
4. `scheduleAdapter`
סוג: `ScheduleAdapter`
תפקיד: מתאם בין הנתונים של מערכת השעות לבין תצוגת ה- `RecyclerView`
שימוש: שולט בהצגת נתוני מערכת השעות ובעדכון שלהם מהתצוגה.
5. `schedule`
סוג: `Map<String, List<String>>`
תפקיד: מייצג את מערכת השעות המלאה של חדר מסוים לפי ימות השבוע.
שימוש: משמש להצגה ולעדכון של המידע במאגר הנתונים.
6. `db`
סוג: `FirebaseFirestore`
תפקיד: מאפשר אינטראקציה עם `Firestore`
שימוש: משמש לקריאה ועדכון של מידע הקשור לחדרים ומערכות שעות.



7. roomAutoComplete

סוג: autoCompleteTextView

תפקיד: תיבת טקסט עם השלמה אוטומטית המאפשרת למשתמש לבחור מספר חדר מתוך רשימה של מספרים שימוש: בבחירת החדר לעריכה.

8. numberDisplay

סוג- textView

תפקיד: תווית טקסט שמציגה את מספר החדר שנבחר על ידי המשתמש מתוך autoCompleteTextView שימוש: להציג באופן ברור למשתמש איזה חדר נבחר, לצורך חיווי ויזואלי ומניעת טעויות.

פעולות המחלקה:

1. onCreateView

קובעת את כל הרכיבים הגרפיים של המסך ומבצעת אתחול של המידע והמאזינים היא יוצרת רשימת חדרים מ-1 עד 100 ומגדירה אותה לשימוש בשדה של החיפוש, כברירת מחדל מוצג החדר מספר 1. בנוסף, הפעולה מקשרת את כפתור ההוספה (addRoom) לפעולה שמובילה למסך הוספת חדר חדש, ואת כפתור commitBtn לפעולה ששומרת את השינויים במערכת השעות של החדר הנבחר.

2. getRoomSced

מקבלת מספר חדר ומציגה את מערכת השעות שלו ב- RecyclerView

3. updateRoomSchedule

מעדכנת את מערכת השעות החדשה (שהמשתמש שינה) אל מאגר הנתונים



ManageSchoolNotificationsFragment

שם המחלקה: ManageSchoolNotificationsFragment

תפקיד המחלקה: המחלקה אחראית על ניהול ההודעות הכלליות של בית הספר באפליקציה.

תכונות המחלקה:

1. listView

וג: ListView

תפקיד: רכיב תצוגה המציג את ההודעות הקיימות ברשימה נגללת
שימוש: להציג למשתמש את ההודעות מבית הספר שנשמרו במסד הנתונים.

2. addNotiBt

וג: ImageButton

תפקיד: לפתוח תיבת קלט (AlertDialog) שמאפשרת למשתמש להזין ולשמור הודעה חדשה.
שימוש: לאפשר למשתמש להוסיף הודעה כללית חדשה.

3. adapter

וג: ArrayAdapter<String>

תפקיד: לדאוג שכל הודעה תוצג בצורה תקינה ברשימה הגרפית.
שימוש: משמש לתאם בין הרכיב הגרפי לקוד עצמו

4. messages

וג: List<String>

תפקיד: מאגר המכיל את הודעות המערכת
שימוש: משמש כחלק מתהליך הצגת ההודעות הכלליות למשתמש

5. db

וג- FirebaseFirestore

תפקיד- מאפשר אינטראקציה עם Firestore
שימוש- משמש לקריאה ועדכון של ההודעות הכלליות.

פעולת המחלקה:

onCreateView

פעולה זו אחראית על בניית הממשק הגרפי של המסך ומבצעת את האתחול הראשוני של כל הרכיבים. בנוסף הפעולה מגדירה פקדים לכפתורים וללחיצה על Item ברשימה



MapFragment

שם המחלקה: MapFragment

תפקיד המחלקה: להציג למשתמש את מפת בית הספר. ותאפשר הזזה הקטנה והגדלה של המפה.

תכונות המחלקה:

1. `imageView`

סוג: `ImageView`

תפקיד: רכיב התצוגה המרכזי המוצג למשתמש, בו מוצגת המפה/התמונה המאפשרת זום וגרירה.

שימוש: משמש להצגת המפה ולאפשר למשתמש לבצע פעולות מגע כמו זום וגרירה.

2. `matrix`

סוג: `Matrix`

תפקיד: מטריצה המייצגת את השינויים (הזזות, זום) שמבוצעים על התמונה.

שימוש: משמשת לעדכון מיקום וזום של התמונה בתוך ה- `ImageView`

3. `scale`

סוג: `float`

תפקיד: מייצג את גורם הזום הנוכחי של התמונה.

שימוש: מגביל את טווח הזום בין מינימום ומקסימום כדי למנוע זום יתר.

4. `lastX, lastY`

סוג `float`

תפקיד: שמירת מיקום נקודת המגע האחרונה בעת גרירת התמונה.

שימוש: משמשים לחישוב ההסטה (הפרש המיקום) בתנועת הגרירה.

5. `scaleGestureDetector`

סוג: `ScaleGestureDetector`

תפקיד: מזהה אירועי זום ומאפשר שינוי קנה המידה של התמונה.

שימוש: מפעיל מאזין לאירועי הזום ומבצע התאמת המטריצה בהתאם.

6. `lastScaleTime`

סוג: `long`

תפקיד: שמירת זמן הפעולה האחרונה של הזום, לשם מניעת טיפול מיידי מדי באירועי גרירה.

שימוש: משמש לקביעת "תקופת קירור" בין פעולות זום לגרירה, כדי למנוע התנגשות בין

הפעולות (כך לא יהיו "קפיצות" במפה).



SCALE_COOLDOWN_MS .7

static final long

תפקיד: משך זמן הקירור (במילישניות) בין פעולת זום לגרירה.

שימוש: משמש להגבלת המהירות שבה ניתן לעבור מפעולת זום לגרירה.

פעולות המחלקה:

onCreateView .1

פעולה זו אחראית על יצירת ממשק המשתמש של theFragment, טוענת את תצוגת המפה, מאתחלת את רכיבי הזום והגרירה, ומגדירה מאזין לאירועי מגע שיאפשר למשתמש לבצע זום וגרירה על התמונה.

פירוט על המחלקה הפנימית:

ScaleListener

תפקיד המחלקה:

אחראית על טיפול בזיהוי תנועות "צביטה" על המסך וביצוע פעולת זום על התמונה בהתאם.

תכונות: אין

פעולות המחלקה:

1. פעולה זו מטפלת באירועי הזום, היא מחשבת את גורם הזום החדש, מגבילה אותו לטווח סביר ומעדכנת את המטריצה כדי להגדיל או להקטין את התמונה בהתאם למגע המשתמש.



SchoolUpdatesFragment

שם המחלקה: SchoolUpdatesFragment

תפקיד המחלקה: הצגת הודעות מערכת מתגלגלות באפליקציה

תכונות המחלקה:

1. scrollView
HorizontalScrollView :וג:
תפקיד: מאפשר גלילה אופקית של הודעות המערכת.
שימוש: משמש לתצוגת פס הודעות מתגלגל (Marquee) בצורה ידנית.
2. tvMessages
TextView :וג:
תפקיד: מציג את ההודעות שמובאות ממסד הנתונים בפורמט שורה אחת.
שימוש: מהווה את תוכן ההודעות שאותן המשתמש רואה נעות על המסך.
3. linearLayout
LinearLayout :וג:
תפקיד: מיכל המחזיק את ההודעות ומאפשר שכפול למטרת לולאת גלילה.
שימוש: מאפשר להציב את ההודעות פעמיים אחת אחרי השנייה לצורך יצירת אפקט גלילה אינסופית.
4. handler
Handler :וג:
תפקיד: כלי להפעלת קוד חוזר (loop) עם השהייה.
שימוש: משמש לצורך הרצת פונקציית הגלילה כל כמה מילישניות.
5. scrollRunnable
Runnable :וג:
תפקיד: קוד שרץ שוב ושוב ומבצע את הגלילה האופקית בפועל.
שימוש: מריץ את הגלילה באיטרציות קטנות לאורך זמן.
6. scrollSpeed
int :וג:
תפקיד: מגדיר את מהירות הגלילה (בפיקסלים לאיטרציה אחת).
שימוש: קובע את קצב תנועת ההודעות.



7. isDuplicated

סוג: boolean

תפקיד: מונע שכפול כפול של הודעות בתוך ה- LinearLayout
שימוש: משמש לבדיקת האם ההודעות כבר שוכפלו ליצירת אפקט גלילה חלקה.

8. db

סוג- FirebaseFirestore

תפקיד- מאפשר אינטראקציה עם Firestore
שימוש- משמש לשליפת הודעות מערכת ממסמך בענן.

פעולות המחלקה:

1. onCreateView

פעולה זו אחראית על בניית הממשק הגרפי של המסך ומבצעת אתחול של כל הרכיבים הגרפיים: גלילה, טקסט, שכפול תצוגות ותזמון הגלילה. בנוסף, היא טוענת את ההודעות ממאגר הנתונים מציגה אותן על המסך, ומשכפלת אותן ליצירת אפקט של פס גלילה אינסופי.

2. getOriginalContentWidth

פעולה זו מחשבת את הרוחב הכולל של סט ההודעות המקורי (לפני השכפול), כדי לדעת מתי יש "לדלג" חזרה לתחילת הגלילה.

3. cloneView

פעולה זו משכפלת תצוגת טקסט קיימת לחדשה עם אותן הגדרות עיצוב, טקסט ומידות. השכפול הזה נועד לאפשר גלילה חלקה על ידי יצירת סט זהה נוסף של הודעות לאחר המקור.

4. onPause

מפסיקה את הגלילה שיוצאים מהמסך

5. onResume

מחזירה את הגלילה כשחוזרים למסך



Activities

AddRoomActivity

שם המחלקה: AddRoomActivity

תפקיד המחלקה:

המחלקה האחראית על הוספת חדרים למאגר הנתונים

תכונות:

1. editRoomNumber

וג: EditText

תפקיד: שדה קלט לטקסט עבור הזנת מספר הכיתה החדשה.
שימוש: לאפשר למשתמש להזין את מספר הכיתה שברצונו ליצור במערכת.

2. recyclerViewSchedule

וג: RecyclerView

תפקיד: רכיב תצוגה להצגת מערכת השעות של הכיתה.
שימוש: להציג ולעדכן את מצב השיעורים לאורך היום והשבוע בכיתה הנבחרת.

3. btnCreateRoom

וג: Button

תפקיד: כפתור ביצוע ליצירת כיתה חדשה.
שימוש: כאשר המשתמש לוחץ עליו, מתבצע ניסיון לשמירת הכיתה במאגר הנתונים.

4. scheduleAdapter

וג: ScheduleAdapter

תפקיד: מתאם בין נתוני מערכת השעות לבין תצוגת ה- RecyclerView

5. defaultSchedule

וג: Map<String, List<String>>

תפקיד: מערכת שעות ברירת מחדל.
שימוש: לאתחל כל כיתה חדשה עם מצב "פנוי" לכל שעה ולכל יום בשבוע.

6. db

וג: FirebaseFirestore

תפקיד- מאפשר אינטראקציה עם Firestore
שימוש- לשמירת אובייקט הכיתה החדש במאגר הנתונים



פעולות המחלקה:

1. onCreate

פונקציית אתחול שפועלת בעת טעינת האקטיביטי.
מאתחלת את רכיבי הממשק, יוצרת מערכת שעות ברירת מחדל, מקשרת את ה-
RecyclerView עם מתאם, ומגדירה את פעולת כפתור יצירת הכיתה.

2. createDefaultSchedule

מחזירה מערכת שעות עם שעות מ-1 עד 9, כאשר כל שעה כוללת 5 ערכים – "Available" אחד
לכל יום לימודים. משמשת ליצירת מערכת בסיסית עבור כיתה חדשה.

3. createRoom

פעולה הבונה ושומרת את הכיתה שנוצרה ממאגר הנתונים.



AdvSearchActivity

שם המחלקה: AdvSearchActivity

תפקיד המחלקה: המחלקה האחראית על חיפוש ע"פ יום בשבוע ושעות ביום הזה, בנוסף כאשר המשתמש מצא חדר הוא יכול ללחוץ עליו והמחקה תעבירו לתפיסת חדר ע"פ מערכת יומית. \

תכונות המחלקה:

1. dayPicker
וג: NumberPicker
תפקיד: בורר יום השבוע (ראשון עד חמישי).
שימוש: משמש כקלט לבחירת היום לחיפוש זמינות כיתות.
2. hourFromPicker
וג: NumberPicker
תפקיד: בורר שעת התחלה של טווח החיפוש.
שימוש: בוחר את השעה שממנה המשתמש רוצה לבדוק אם הכיתות פנויות.
3. hourToPicker
וג: NumberPicker
תפקיד: בורר שעת סיום של טווח החיפוש.
שימוש: בוחר את השעה עד אליה רוצים לבדוק את הזמינות.
4. searchBT
וג: Button
תפקיד: רשימת הכיתות הזמינות בהתאם לחיפוש.
שימוש: בלחיצה עליו, נשלפת רשימת כיתות פנויות לפי הטווח שנבחר.
5. roomList
וג: List<Rooms>
תפקיד: רשימת הכיתות הזמינות בהתאם לחיפוש.
שימוש: משמש כמאגר הנתונים המקומי
6. adapter
וג: RoomsAdapter
תפקיד: מחבר בין הנתונים של roomList לRecyclerView
שימוש: מציג את הכיתות הזמינות על המסך בצורה רשימתית.



פעולות המחלקה:

1. onCreate
אתחול ראשוני של האקטיביטי – כולל קישור לרכיבי ממשק, הגדרת אפשרויות ברירת מחדל, ואתחול רשימת הכיתות.
2. populateSearchRoomList
מבצעת שאילתה למאגר הנתונים לקבלת כל הכיתות, מסננת לפי זמינות בטווח שעות מסוים, ומעדכנת את הרשימה המוצגת.
3. OnItemClickListener
מאזין ללחיצה על כיתה מסוימת ברשימה – אם מדובר ביום לימודים, בודק אם המשתמש הוא מורה, ואם כן – מעביר אותו למסך הראשי עם פרטי הכיתה שנבחרה.



InfoAndBookRoomActivity

שם המחלקה: InfoAndBookRoomActivity

תפקיד המחלקה:

מסך שמציג למורה את מערכת הכיתה שהוא בחר. המסך מאפשר לתפוס את הכיתה הזאת בשעות מסוימות ובימים מסוימים בשבוע

תכונות המחלקה:

1. dayPicker

וג: NumberPicker

תפקיד: מאפשר בחירת יום השבוע (ראשון עד חמישי).
שימוש: קלט לבחירת היום שבו רוצים להזמין את הכיתה.

2. hourFromPicker

וג: NumberPicker

תפקיד: מאפשר בחירת שעת התחלה.
שימוש: מציין מאיזו שעה מתחילה ההזמנה.

3. hourToPicker

וג: NumberPicker

תפקיד: מאפשר בחירת שעת סיום.
שימוש: מציין עד איזו שעה הכיתה תוזמן.

4. bookRoomBT

וג: Button

תפקיד: כפתור להזמנת הכיתה.
שימוש: בעת לחיצה, נבדק האם ניתן להזמין את הכיתה ומבוצעת הזמנה במסד הנתונים.

5. firestore

וג: FirebaseFirestore

תפקיד: ממשק למסד הנתונים Firestore
שימוש: משמש לשליפת ועדכון נתונים של כיתות ומשתמשים

6. isUp

וג: boolean

תפקיד: מציין אם העדכון במסד הנתונים בוצע בהצלחה.
שימוש: ניטור הצלחת פעולות כתיבה למאגר הנתונים.



פעולות המחלקה:

1. onCreate

מאתחלת את הרכיבים הגרפיים, מקבלת את אובייקט הכיתה שהועבר מה- Intent מציגה את המידע ומגדירה את התנהגות כפתור ההזמנה.

2. SetUpRoom

מציגה את מערכת השעות של הכיתה שנבחרה ברשימת RecyclerView



LoginActivity

שם המחלקה: LoginActivity

תפקיד המחלקה: המחלקה אחראית על פעולת ההתחברות של המשתמש לאפליקציה.

תכונות המחלקה:

1. emailTextView

וג: EditText

תפקיד: שדה קלט לאימייל של המשתמש.

שימוש: משמש לקריאת כתובת האימייל בזמן התחברות.

2. PasswordTextView

וג: EditText

תפקיד: שדה קלט לסיסמה של המשתמש.

שימוש: משמש לקריאת הסיסמה מהמשתמש לצורך התחברות.

3. Btn

וג: Button

תפקיד: כפתור ההתחברות.

שימוש: מפעיל את הפעולה loginUserAccount כאשר נלחץ.

4. progressBar

וג: ProgressBar

תפקיד: פס טוען (loading)

שימוש: מוצג במהלך התחברות, מוסתר בסיום או בכישלון.

5. mAuth

וג: FirebaseAuth

תפקיד- מנהלת את תהליך האימות עם Firebase, ומאפשרת גישה למידע על המשתמש המחובר.

שימוש- באמצעותו נרשם המשתמש במאגר הנתונים.

6. ToRegi

וג: Button

תפקיד: כפתור להרשמה.

שימוש: מעביר את המשתמש למסך Register



7. `checkBoxRememberMe`

סוג: `CheckBox`

תפקיד: תיבת בימון עבור המשתמש

שימוש: אם מסומנת, שומרת את האימייל והסיסמה של המשתמש מקומית דרך `PrefManager`

פעולות המחלקה:

1. `onCreate`

אתחול ראשוני של המסך והמשתנים.

מתחלת את כל הרכיבים הגרפיים במסך, בודקת אם המשתמש כבר התחבר בעבר, חוסמת את כפתור החזרה, ומאזינה ללחיצות כפתורי ההתחברות וההרשמה

2. `loginUserAccount`

מבצעת התחברות של המשתמש ושומרת נתונים לפי מצב ה-`CheckBox`

3. `startHomeActivity`

מעביר את המשתמש למסך הראשי (`MainActivity`) וסוגר את מסך ההתחברות.

4. `checkAndUpdateFirestoreEmail`

בודקת אם האימייל של המשתמש המאומת שונה מהאימייל השמור במאגר הנתונים אם יש הבדל, מעדכנת את האימייל במסד הנתונים כדי שיהיה תואם למידע האמיתי.

5. `saveLoginDetails`

שומרת פרטי התחברות ב-`PrefManager` במידה ו-`Remember Me` מסומן.



MainActivity

שם המחלקה: MainActivity

תפקיד: מחלקת MainActivity היא הפעילות הראשית באפליקציה, המשמשת כמרכז הניווט והשליטה על התצוגה. היא מטפלת ב־תפריט צד (Drawer), תפריט תחתון (BottomNavigationView), הטענת פרגמנטים מתאימים לפי הקשר, הצגת פרטי המשתמש, והתאמת חוויית השימוש לפי תפקיד המשתמש (צוות / תלמיד).

תכונות המחלקה:

1. drawerLayout
וג: DrawerLayout
תפקיד: תפריט צד שמכיל קישורים למסכים השונים באפליקציה.
שימוש: משמש לפתיחה וסגירה של תפריט הניווט הצדדי.
2. name
וג: GetName
תפקיד: מביא את שם המשתמש מה-Firebase.
שימוש: מציג את שם המשתמש בסרגל העליון ובתפריט הצד.
3. email
וג: GetEmail
תפקיד: מביא את כתובת המייל של המשתמש.
שימוש: מציג את כתובת הדוא"ל של המשתמש בתפריט הצד.
4. nameT
וג: TextView
תפקיד: מציג את שם המשתמש.
שימוש: מופיע בתפריט הצד.
5. EmailT
וג: TextView
תפקיד: מציג את האימייל של המשתמש.
שימוש: מופיע בתפריט הצד.
6. profilePic
וג: ImageView
תפקיד: תצוגת תמונת פרופיל של המשתמש.
שימוש: משויכת לפרופיל המשתמש ונשלפת ממאגר הנתונים.



CHANNEL_ID .7

String: סוג

תפקיד: מזהה את ערוץ ההתראות.

שימוש: משמש להפעלת התראות למשתמש.

פעולות המחלקה:

1. onCreate

מאתחלת את ממשק המשתמש, toolbar, תפריט הצד ותפריט תחתון. טוענת את המסך הראשי (HomeFragment) או את DailyScheduleFragment אם נשלחה כיתה מסוימת. מתחברת לפעולות חיצוניות על מנת להציג נתונים אישיים של המשתמש. בודקת האם המשתמש הוא צוות ומציגה תפריטים מתאימים. מפעילה התראות לפי שעות התחלה מהמערכת.

2. onNavigationItemSelectedListener

מגיבה לבחירה בתפריט הצד ומנווטת בין פרגמנטים שונים בהתאם לאופציית התפריט שנבחרה. מאפשרת גם התנתקות מהמערכת ומעבירה את המשתמש למסך ההתחברות.

3. onBackPressed

אם תפריט הצד פתוח – סוגרת אותו. אחרת – מבצעת את פעולת החזרה הרגילה.



MyRoomsActivity

שם המחלקה: MyRoomsActivity

תפקיד המחלקה: ניהול החדרים שהמשתמש תפס

תכונות המחלקה:

1. recyclerView

סוג: RecyclerView

תפקיד: תצוגת רשימה להצגת ההזמנות של המשתמש.

שימוש: משמש להצגת כיתות שהוזמנו על ידי המשתמש באמצעות מתאם (adapter)

2. adapter

סוג: UserRoomsAdapter

תפקיד: מתאם שמחבר בין רשימת ההזמנות לתצוגת ה- RecyclerView

שימוש: מתקן את הרשימה בכל פעם שיש שינוי ברשימת ההזמנות ומציג את הרשימה ההתחלתית

3. userRooms

סוג: List<User.Booking>

תפקיד: רשימת ההזמנות של המשתמש המחובר.

שימוש: משמשת כמאגר הנתונים המקומי, נטענת מ-Firebase ומועברת למתאם

פעולות המחלקה:

1. onCreate

אתחול ראשוני של המסך.

מאתחלת את התצוגה, מאזינה לשוליים של המערכת (Edge-to-Edge), יוצרת את רשימת ההזמנות ואת המתאם, ומבצעת קריאה לפונקציה populateUserRoomList כדי לטעון את נתוני ההזמנות מה-Firebase.

2. populateUserRoomList

מטפלת בהוספת החדרים ממאגר הנתונים ל-RecyclerView

3. onResume

מנקה את הרשימה הקיימת ומטעינה מחדש את נתוני ההזמנות על ידי קריאה ל-populateUserRoomList.



RegisterActivity

שם המחלקה: RegisterActivity

תפקיד המחלקה: המחלקה האחראית על הרשמת המשתמש לאפליקציה והכנסתו למאגר הנתונים

תכונות המחלקה:

1. emailTextView
סוג: EditText
תפקיד: שדה להזנת כתובת אימייל של המשתמש.
2. passwordTextView
סוג: EditText
תפקיד: שדה להזנת סיסמה של המשתמש.
3. nameEditT
סוג: EditText
תפקיד: שדה להזנת שם פרטי של המשתמש.
4. TeachPassT
סוג: EditText
תפקיד: שדה להזנת סיסמת מורה לצורך אימות תפקיד המורה.
5. Class1
סוג: EditText / AutoCompleteTextView
תפקיד: שדה לבחירת כיתה עבור תלמידים.
שימוש: רק עבור תלמידים – מורים לא אמורים להזין כיתה.
6. Btn
סוג: Button
תפקיד: כפתור להרשמה.
שימוש: בלחיצה עליו מתבצעת בדיקת סיסמת מורה, ולאחר מכן ניסיון הרשמה.
7. progressbar
סוג: ProgressBar
תפקיד: מציג חיווי ויזואלי בעת תהליך ההרשמה.
8. mAuth
סוג: FirebaseAuth
תפקיד: מנהל התחברות והרשמה עם Firebase Authentication.



9. database

סוג: FirebaseDatabase

תפקיד: אחראי על גישה למסמכים ב-Firestore.

10. classarr

סוג: String[]

תפקיד: מערך כל הכיתות בבית הספר לבחירה.

שימוש: משמש לאימות תקינות שם הכיתה שהוזנה.

11. IsT

סוג: boolean

תפקיד: האם המשתמש מזוהה כמורה.

שימוש: קובע אם לאפשר הזנת כיתה, ומהו תפקיד המשתמש בשמירה.

פעולות המחלקה:

1. onCreate

מתחלת את כל הרכיבים הגרפיים במסך, יוצרת מתאם להצעות כיתה לשדה ה-AutoCompleteTextView, מאזינה ללחיצה על כפתור ההרשמה, ומבצעת בדיקת סיסמת מורה.

2. validateFields

מאמתת את תקינות הנתונים שהוזנו.

3. IsClassValid

בודקת אם כיתה מסוימת קיימת במערך הכיתות.

4. checkTeachPass

בודקת אם סיסמת המורה שהוזנה תואמת לסיסמה הקיימת במאגר הנתונים אם כן, מסמנת את IsT כ-true.

לאחר הבדיקה קוראת לפונקציה validateFields ואם כל הנתונים תקינים – מתחילה רישום.

5. registerNewUser

מבצעת רישום משתמש חדש עם FirebaseAuth ושולחת את המשתמש למסך ההתחברות במידה והוא נרשם.



Notification Management

NotificationRecivier

שם המחלקה: NotificationRecivier

תפקיד המחלקה: אחראית לקבל Broadcast מהמערכת, וליצור ממנו התראה (Notification) שמוצגת למשתמש. היא מופעלת אוטומטית כאשר מתקבל Intent עם פרטי ההתראה (כותרת והודעה), ובונה באמצעותם התראה חזותית דרך מערכת ההתראות של Android.

למחלקה זו אין תכונות.

פעולות המחלקה:

1. onReceive

פעולה שמופעלת אוטומטית כאשר מתקבלת התראה (broadcast). מאזינה להתראות שנשלחות דרך AlarmManager, בודקת אם קיבלה כותרת והודעה, בונה התראה (Notification) ומציגה אותה למשתמש באמצעות NotificationManager.



SetNoti

שם המחלקה: SetNoti

תפקיד: אחראית לקביעת תזכורות (Notifications) לשיעורים, כך שהמשתמש יקבל התראה כ-10 דקות לפני תחילת השיעור. היא ממירה מספר שיעור (1-9) לשעת ההתחלה המתאימה לפי מערכת השעות, מחשבת את הזמן של 10 דקות קודם לכן, ואז מתזמנת את ההתראה באמצעות AlarmManager. ההתראה נשלחת דרך BroadcastReceiver שמופעל בזמן המתאים, גם אם האפליקציה סגורה.

תכונות המחלקה: אין

פעולות המחלקה:

1. convertAndSet

פעולה סטטית שמקבלת הקשר (Context) ומספר שיעור ראשון (int firsthour). התפקיד שלה הוא להמיר את מספר השיעור לשעת התחלה מדויקת (שעה ודקות) לפי טבלת זמנים מוגדרת, להתאים את השעה ל-10 דקות מוקדמות יותר, ואז לקרוא לפעולה שמגדירה התראה (Notification) בשעה הזו.

2. scheduleNotification

פעולה פרטית סטטית המקבלת הקשר, כותרת ההודעה, תוכן ההודעה, שעה ודקות. תפקידה ליצור ולהגדיר ערוץ התראות (Notification Channel) במכשירים עם אנדרואיד O ומעלה, לבנות את ההודעה לפי הפרטים, לתזמן אותה עם AlarmManager שיפעיל את ההתראה בזמן המדויק. אם הזמן שנבחר כבר עבר באותו יום, הפעולה מתזמנת את ההתראה ליום הבא.



Utilities

GetDay

מחלקה סטטית שמספקת מידע על היום הנוכחי בשבוע, בהתחשבות באזור הזמן של ישראל (GMT+2).

תכונות:

אין תכונות מחלקה (כל השיטות סטטיות).

פעולות:

getCurrentDay – מחזירה את היום הנוכחי כשם מספרי (int) לפי מערכת ה-Calendar של ג'אווה (לדוגמה, יום ראשון = 1, יום שני = 2 וכו').

getCurrentDayName – מחזירה את שם היום הנוכחי כמחרוזת (String), באנגלית (למשל: "Monday", "Tuesday" וכו').

GetEmail

מחלקה שמטרתה לאחזר את כתובת האימייל של המשתמש הנוכחי מחיבור Firebase Authentication וממסד הנתונים Firestore.

תכונות:

email – מחרוזת לשמירת האימייל שהתקבל (פרטית למחלקה).

פעולות:

ToolText

פונקציה שמבצעת בדיקה האם יש משתמש מחובר דרך FirebaseAuth, ואז מנסה לאחזר את כתובת האימייל שלו ממאגר הנתונים.

מקבלת callback (ממשק ToolTextCallback) דרכו מוחזרת התוצאה – מחרוזת האימייל או הודעת ברירת מחדל ("Not logged in") במקרה שאין משתמש מחובר, המסמך לא קיים, או שקרתה שגיאה.

ממשק:

ToolTextCallback – ממשק callback עם פונקציה אחת:

void onToolTextFetched(String email)

דרכו מוחזרת כתובת האימייל או הודעת השגיאה למבקש המידע.



GetFirstHours

מחלקה שמטרתה לאחזר את השעות הראשונות של הזמנות כיתות עבור המשתמש הנוכחי לפי היום הנוכחי.

תכונות:

אין שדות מאוחסנים במחלקה (הפונקציה סטטית ומטפלת בשימוש בלבד).

פעולות:

GetFirstHours

פונקציה סטטית שמבצעת קריאה למסד הנתונים Firestore כדי לקבל את מסמך המשתמש הנוכחי (בהתבסס על מזהה המשתמש ב-FirebaseAuth). הפונקציה בודקת את רשימת ההזמנות של המשתמש (bookedRooms) לפי היום הנוכחי (באמצעות המחלקה GetDay), ואוספת את השעה הראשונה מכל הזמנה ביום זה. מחזירה את רשימת השעות הראשונות דרך callback של HoursCallback. במידה ויש תקלה בקריאת הנתונים, מחזירה את השגיאה דרך ה-callback.

ממשק callback:

HoursCallback – ממשק עם שתי פונקציות:

void onSuccess(List<Integer> firstHours) – קוראים לה עם רשימת השעות הראשונה שהתקבלה.

void onFailure(Exception e) – קוראים לה במקרה של כשלון בקריאת הנתונים.

GetHour

מחלקה המספקת מידע על השעה הנוכחית לפי אזור הזמן של ירושלים, בהתייחס לשעות הלימודים המוגדרות.

תכונות:

אין שדות מאוחסנים במחלקה (כל הפונקציות סטטיות).

פעולות:

getCurrentHour

מחזירה את השעה הנוכחית במערכת (24 שעות) לפי אזור הזמן Asia/Jerusalem.

getFormattedHour

מחזירה מחרוזת שמתארת את השעה הנוכחית במסגרת מערכת השעות ("Hour X"). אם השעה לא במסגרת שעות הלימודים, מחזירה את הטקסט "Not within school hours".



getScheduleHour

מחשבת ומחזירה את השעה בלוח הזמנים של בית הספר לפי הזמן הנוכחי, או 1-אם הזמן לא בשעות הלימודים.

convertToScheduleHour (פרטי)

פונקציה פנימית שמקבלת שעה ודקה ומחזירה את מספר השעה (1-9) בהתאם לטווחי זמן מוגדרים מראש. מחזירה 1 אם הזמן לא נמצא בתוך אחד מטווחי השעות.

GetIsStaff

מחלקה זו אחראית לבדוק האם המשתמש המחובר כעת הוא איש צוות (staff), ע"י שליפת שדה מתאים ממסמך המשתמש במסד הנתונים של Firestore.

תכונות- אין

פעולות המחלקה:

checkIfStaff

פעולה אסינכרונית שבודקת האם המשתמש המחובר מסומן כאיש צוות במסד הנתונים. אם כן, מחזירה true; אחרת (כולל מקרים של שגיאה או משתמש לא מחובר) מחזירה false. התוצאה נמסרת דרך ממשק callback בשם ToolTextCallback.

toString

מחזירה מחרוזת תיאורית פשוטה של המחלקה (משמשת בעיקר לצורכי ניפוי שגיאות או בדיקות).

ממשק פנימי:

ToolTextCallback

ממשק המאפשר קבלת תוצאה של בדיקת המשתמש. מכיל מתודה אחת: onToolTextFetched(boolean isStaff).



GetIsT

מחלקה זו אחראית לבדוק האם המשתמש המחובר הוא מורה, באמצעות שליפת ערך isT ממסמך המשתמש במסד הנתונים של Firestore.

תכונות- אין

פעולות המחלקה:

-checkIfTeacher

פעולה אסינכרונית שבודקת האם המשתמש המחובר מסומן כמורה מחזירה את התוצאה (true/false) דרך ממשק callback בשם FirestoreCallback. במקרה של שגיאה או אם המשתמש לא מחובר, מוחזרת התשובה false.

ממשק פנימי:

FirestoreCallback

ממשק להחזרת תוצאה בוליאנית על פי בדיקת סטטוס המורה.
כולל את הפונקציה .onResult(boolean isTeacher)

GetName

מחלקה זו אחראית להביא את שם המשתמש המחובר לאפליקציה.

תכונות:

Name

String :וג

תפקיד: לאחסן את שם המשתמש שנשלף מה-Firestore
שימוש: מחזיק את ערך השם שהתקבל מהמסמך (אם קיים)

פעולות:

ToolText

פעולה אסינכרונית שמביאה את שם המשתמש המחובר מהמסמך שלו באוסף users במסד הנתונים. אם המשתמש אינו מחובר או מתרחשת שגיאה, היא מחזירה את ברירת המחדל "Hello". התוצאה מוחזרת באמצעות ממשק ToolTextCallback.

toString

פעולה סטנדרטית שמחזירה מחרוזת תיאור של האובייקט

ממשק פנימי:

ToolTextCallback

ממשק להחזרת שם המשתמש שנשלף.

כולל את הפעולה .onToolTextFetched(String name)



GetProfilePic

מחלקה זו אחראית להבאת תמונת הפרופיל של המשתמש המחובר מתוך מסמך המשתמש במסד הנתונים Firestore. התמונה מאוחסנת כמחרוזת Base64 ומומרת ל-`Bitmap`.

תכונות:

`profilePic`

סוג: `Bitmap`

תפקיד: לאחסן את תמונת הפרופיל המומרת מה-`Base64`

שימוש: נשלפת מתוך ה-`Firestore` ומומרת לפורמט תמונה להצגה באפליקציה

פעולות:

`getProfilePic`

פעולה אסינכרונית אשר מביאה את תמונת הפרופיל של המשתמש מתוך מאגר הנתונים

`decodeBase64ToImage`

פעולה פרטית שממירה מחרוזת `Base64` לאובייקט מסוג `Bitmap`.

משמשת בתוך `getProfilePic` כדי לפענח את התמונה.

ממשק פנימי:

`ToolTextCallback`

ממשק `callback` לקבלת התמונה המתקבלת.

כולל את הפעולה `onProfilePicFetched(Bitmap profilePic)` אשר מופעלת עם התמונה או `null`.



More

PrefManager

מחלקה זו אחראית על ניהול נתוני ההתחברות המקומיים (Email ו-Password) של המשתמש באמצעות SharedPreferences.

תכונות המחלקה:

context

סוג: Context

תפקיד: הקשר של האפליקציה הנדרש לגישה ל-SharedPreferences שימוש: מועבר מה-Activity או Fragment כדי לבצע פעולות שמירה וטעינה של נתונים

פעולות:

PrefManager

אתחול של המחלקה עם הקשר (context) שממנו יתבצעו פעולות SharedPreferences.

saveLoginDetails

שומר את פרטי ההתחברות (email ו- password) בתוך SharedPreferences בשם "LoginDetails".

Forgetlog

מוחק את כל הנתונים שנשמרו תחת "LoginDetails", כלומר מוחק את פרטי ההתחברות – שקול כיציאה מהחשבון.

getEmail

מחזיר את כתובת האימייל שנשמרה ב-SharedPreferences. אם לא נשמר אימייל, מחזיר מחרוזת ריקה.

isUserLoggedOut

בודק אם המשתמש מחובר או מנותק על פי השדות ב-SharedPreferences. מחזיר true אם או כתובת האימייל או הסיסמה ריקים – כלומר המשתמש מנותק.

RoomAdapterInterface

ממשק המשמש לאינטראקציה עם פריטים ב- RecyclerView של חדרים. הממשק מגדיר פעולה אחת שמופעלת כאשר לוחצים על פריט מסוים (כיתה/חדר) ברשימה.

פעולות:

OnItemClickListener

מקבלת את מיקום הפריט שנלחץ (position) בתוך הרשימה.

מיושמת על ידי Activity או Fragment כדי להגיב ללחיצה, למשל ניווט למסך פרטים של החדר או ביצוע הזמנה.



מדריך למשתמש

גרסאות בדיקה:

Pixel 3 Api 34

כאשר מפעילים את האפליקציה בזמן שהיא נטענת מוצגת תמונת הלוגו של האפליקציה.

לאחר טעינת האפליקציה, המשתמש מועבר למסך ההתחברות. במסך הזה, צריך למלא שם משתמש וסיסמה על מנת להיכנס לאפליקציה. בנוסף, במסך ההתחברות קיים כפתור הרשמה עבור מי שעוד לא נרשם לאפליקציה ובלחיצה עליו אתם מועברים למסך הרשמה.

במסך ההרשמה קיימים 5 שדות במידה והמשתמש מורה המשתמש יצטרך למלא שם משתמש, אימייל, סיסמא ואת הסיסמה שחולקה מטעם בית הספר. במידה והמשתמש תלמיד הוא יצטרך למלא שם, סיסמה, מייל וכיתה. במידה והמשתמש צוות, הוא יצטרך למלא שם סיסמה ומייל. בלחיצה על כפתור ההרשמה המשתמש יירשם במאגר הנתונים ויוכל להיכנס לאפליקציה.

לאחר הכניסה לאפליקציה המשתמש מובל למסך הבית. במסך הבית יש רשימה של כל החדרים הפנויים בזה הרגע, במידה והמשתמש לוחץ על חדר החדרים הפנויים, הוא יובל למסך של תפיסת חדר לפי שעות ביום הנוכחי. בנוסף במסך הבית המשתמש רואה את הודעות ביה"ס. בנוסף, במסך הבית יש כמה תפריטים שבלחיצה עליהם המשתמש יכול לגשת לפיצ'רים שונים של האפליקציה.

בלחיצה על פיצ'ר המפה יועבר המשתמש למסך עם מפת ביה"ס שהמשתמש יכול להגדיל, להזיז ולהקטין. למעלה בצד שמאל של המפה קיים מקרא של המפה. חץ למעלה זה אומר קומה למעלה חץ למטה זה אומר קומה למטה.

בלחיצה על פיצ'ר החדרים המשתמש יועבר למסך שבו מוצגים כל חדרי בית הספר בסדר מהמספר הקטן למספר הגדול עם שורת חיפוש למעלה שמתירה חדר לחפש לפי מספרו. בנוסף בתחתית המסך יש שני כפתורים. כפתור של החדרים שאתה תפסת. וכפתור של חיפוש מתקדם. בלחיצה על החדר עצמו המשתמש יועבר למסך שבו מוצגת המערכת השבועית של החדר ואופציה לתפוס את החדר בשעות מסוימות ביום מסוים. כמובן שזה קורה רק אם המשתמש מורה.

בלחיצה על כפתור "החדרים שלי", המשתמש יועבר למסך חדש שבו הוא יראה את כל החדרים שהוא תפס עם כל פרטי התפיסה. בנוסף, ניתן למחוק את החדרים בלחיצה על כפתור הפח.

בלחיצה על כפתור חיפוש מתקדם תוכל לחפש חדר לפי יום בשבוע ובין שעות מסוימות ביום הזה, במסך החיפוש יופיעו חדרים לפי הסינון, במידה והמשתמש לוחץ על חדר מסוים, תופיע המערכת היומית של החדר ואופציה לתפוס ע"פ שעה.



בתפריט הצידי של מסך הבית קיימות כמה אפשרויות, וביניהם לראות את פרטי החשבון ובמידה והמשתמש הוא חלק מצוות ביה"ס לנהל את עדכוני המערכת, לנהל את המערכת הקבועה של החדרים בבית הספר והתנתקות מהאפליקציה.

במסך החשבון שלי, ניתן לראות את פרטי המשתמש, ניתן למחוק תמונת פרופיל, להוסיף תמונת פרופיל, לשנות תמונת פרופיל, לשנות מייל, לאפס סיסמא וכל הפעולות של החשבון במקום אחד.

במסך ניהול הודעות המערכת תופיע רשימה של הודעות המערכת וכפתור בצד ימין למעלה בלחיצה על הכפתור יפתח לך מסך של הוספת הודעת מערכת ובלחיצה על אייטם מהרשימה ההודעת מערכת תימחק לאחר אישור.

במסך ניהול חדרי בית הספר המשתמש יוכל לחפש חדר על פי מספרו וברגע שהוא יכתוב את מספר החדר, מערכת החדר תופיע ותהיה ניתנת לשינוי. נא לשים לב שזו המערכת הקבועה של החדר ולא המערכת השבועית שמתאפסת כל שבוע. בנוסף, בצד ימין למעלה יש כפתור של הוספת חדר שדרכו המשתמש יוכל להוסיף חדר ולהרכיב לו מערכת משל עצמו.

בלחיצה על כפתור ההתנתקות המשתמש יישלח למסך הכניסה ופרטי הכניסה שלו יימחקו מהזיכרון המקומי.

הודעות למשתמש-

10 דקות לפני כל שיעור תישלח הודעה שעומד להתחיל שיעור

מגבלות

בעריכת מערכת החדרים קיימת אופציה מקסימלית לחדר 100

חייב להיות מייל חוקי

סיסמא- מעל 4 תווים

במידה והמשתמש מורה אסור לו לכתוב כיתה.

כיתות- מז עד יב, מ7-1



רפלקציה

העבודה על הפרויקט הייתה ממש נחמדה עבורי, מאוד נהנתי ליצור, לכתוב ללמוד דברים חדשים ולהבין לעומק את מה שאני עושה. היו כישלונות, מקרים שבהם כתבתי קוד במשך זמן ולבסוף הוא לא עבד, אך השתמשתי ב-LogCat, באינטרנט וב-Ai והצלחתי לפתור את הבעיות בקוד, וכאשר הקוד רץ ורואים את הפרויקט עובד, מורגשת תחושת סיפוק אדירה. אך לא כל הנוצץ זהב, החלק השני של הפרויקט, הספר הזה, היה ארוך ומייגע, כתיבת ההסברים היו ארוכים ומתישים, הפירוט המוגזם לקח לי זמן רב ובתחושה שאני מבצע משהו כמעט חסר תועלת, אפשר להגיד שממש סבלתי לכתוב את הספר. בנוסף, לוח הזמנים וההבנה השגויה שלי שספר הפרויקט הינו דבר שלא לוקח יותר מידי זמן לא תרמו בכלל. אם אני יכול להוציא משהו אחד טוב מכתובת ספר הפרויקט הוא שלמדתי את הקוד שכתבתי אפילו יותר לעומק ממה שידעתי כבר. בקיצור, החלק הראשון היה חוויתי והרגשתי שאני ממש יוצר ונהנה, והחלק השני, הבריקטי, פשוט היה סבל. למרות זאת, היתרונות של החלק הראשון גוברים על חסרונות החלק השני, ולכן חווית הפרויקט סך הכל הייתה נחמדה.

תהליך הלמידה היה נחמד ביותר, הלמידה הייתה בסביבת אנדרואיד, למדתי פעולות של אנדרואיד סטודיו, מבני נתונים ומחלקות שלא היו ידועות לי לפני, הגעתי למסקנה שסביבת אנדרואיד היא סביבה די נוחה לעבודה, כאשר יש המון השלמות של הקוד ולא צריך לרשום יותר מידי. בנוסף לכך, למדתי איך להתנהל עם פיירבייס (לא רק באנדרואיד). בשביל ללמוד נעזרתי במורה, באינטרנט ב-AI ובניסוי וטעיה.

יש כמה כלים שהייתי רוצה לקחת להמשך הדרך, כמו למידה לבד, הסתמכות על כמה מקורות מידע, תמיד לשאוף ליותר טוב, מובן, נוח. ולהיות פתוח לרעיונות של אחרים.

בראייה לאחור, הייתי מיישם כמה חלקים אחרת, הייתי משתמש ביותר Utilities, הייתי מוסיף רשימת המתנה לחדרים התפוסים הייתי משפר את עיצוב המערכות (הופך את הממשק ליותר נוח לצפייה), הייתי נותן אופציה לעריכת הודעות המערכת, ולעריכת שעות התפיסה במסך "החדרים שלי". הייתי מוסיף קבוצות למידה של תלמידים(חלק ממש גדול ששקלתי אם להוסיף אך בגלל אילוצי זמן לא הספקתי). יש עוד דברים לשפר אבל בגלל אילוצים הם לא שופרו/נכנסו לפרויקט.

במידה והיו לי משאבים נוספים, הייתי שומר את תמונת הפרופיל ב-Firebase Storage ולא כ-base64 בפירסטור.

הייתי שואל את עצמי
איך ניתן לשפר את יעילות הפרויקט?
כמה דעתי על הפרויקט שונה ממה שהיא עכשיו?

ניתן לשפר את יעילות הפרויקט בהוספת יוטיליטיז ובביקורת עמיתים
דעתי על הפרויקט שונה במעט, למדתי שסביבת אנדרואיד היא די נוחה ושהעבודה על הפרויקט עצמו הייתה יותר מהנה משחשבתי.



בבליוגרפיה

[/https://chatgpt.com](https://chatgpt.com)

[/https://www.geeksforgeeks.org](https://www.geeksforgeeks.org)

<https://firebase.google.com/docs>

<https://stackoverflow.com/questions>

[/https://www.youtube.com](https://www.youtube.com)

<https://claude.ai/login?returnTo=%2F%3F>



נספחים

Adapters

HourScheduleAdapter

```
package com.example.projectarieleyalyotamyona.Adapters;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.R;

import java.util.ArrayList;
import java.util.List;

public class HourScheduleAdapter extends
RecyclerView.Adapter<HourScheduleAdapter.ScheduleViewHolder> {

    private Context context;
    private List<String> scheduleList;

    // 1. Constructor: Initializes adapter with context and daily schedule
    list.
    public HourScheduleAdapter(Context context, List<String> dayschedule)
    {
        this.context = context;
        // Avoid null pointer exceptions by ensuring the list is never
        null
        this.scheduleList = dayschedule != null ? dayschedule : new
        ArrayList<>();
    }

    // 2. ViewHolder: Holds references to views for two time slots in one
    row
    public static class ScheduleViewHolder extends RecyclerView.ViewHolder
    {
        TextView hourTextView1, availabilityTextView1;
        TextView hourTextView2, availabilityTextView2;

        public ScheduleViewHolder(@NonNull View itemView) {
            super(itemView);
            // Initialize TextViews from the layout
            hourTextView1 = itemView.findViewById(R.id.hour1);
            availabilityTextView1 =
```



```
itemView.findViewById(R.id.tv_availability1);
        hourTextView2 = itemView.findViewById(R.id.hour2);
        availabilityTextView2 =
itemView.findViewById(R.id.tv_availability2);
    }
}

@NonNull
@Override
public ScheduleViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
    // 3. Inflate item layout for each row (which contains two time
slots)
    View view =
LayoutInflater.from(context).inflate(R.layout.item_layout_hour_scedual,
parent, false);
    return new ScheduleViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull ScheduleViewHolder holder, int
position) {
    int index = position * 2; // Each row displays 2 time slots, so
calculate actual index

    // 4. Bind first time slot (always exists if index is within
range)
    if (index < scheduleList.size()) {
        String place = scheduleList.get(index);
        String hour1 = "Hour " + (index + 1); // You can replace with
real-time strings if available
        holder.hourTextView1.setText(hour1);
        holder.availabilityTextView1.setText(place);
    }

    // 5. Bind second time slot (only if there is a next item in the
list)
    if (index + 1 < scheduleList.size()) {
        String place2 = scheduleList.get(index + 1);
        String hour2 = "Hour " + (index + 2);
        holder.hourTextView2.setText(hour2);
        holder.availabilityTextView2.setText(place2);
        // Make sure the views are visible
        holder.hourTextView2.setVisibility(View.VISIBLE);
        holder.availabilityTextView2.setVisibility(View.VISIBLE);
    } else {
        // 6. Hide second slot views if no second item for this row
        holder.hourTextView2.setVisibility(View.INVISIBLE);
        holder.availabilityTextView2.setVisibility(View.INVISIBLE);
    }
}

@Override
public int getItemCount() {
```



```
// 7. Total rows = ceil(size / 2) since each row shows two items  
return (scheduleList.size() + 1) / 2;  
}
```



RoomsAdapter

```
package com.example.projectarieleyalyotamyona.Adapters;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.Utilities.GetDay;
import com.example.projectarieleyalyotamyona.Utilities.GetHour;
import com.example.projectarieleyalyotamyona.R;
import com.example.projectarieleyalyotamyona.Utilities.RoomAdapterInterface;
import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;

import java.util.List;

public class RoomsAdapter extends
RecyclerView.Adapter<RoomsAdapter.RoomViewHolder> {

    private final RoomAdapterInterface roomAdapterInterface;
    private Context context;
    private List<Rooms> roomList;

    // Constructor: Initialize context, data list, and click interface
    public RoomsAdapter(Context context, List<Rooms> roomList,
RoomAdapterInterface roomAdapterInterface) {
        this.context = context;
        this.roomList = roomList;
        this.roomAdapterInterface = roomAdapterInterface;
    }

    // ViewHolder class: Holds references to each view in a room item
    layout
    public static class RoomViewHolder extends RecyclerView.ViewHolder {
        TextView roomNum, scheduleStatus, temp, isOccupied;

        public RoomViewHolder(@NonNull View itemView, RoomAdapterInterface
roomAdapterInterface) {
            super(itemView);
            roomNum = itemView.findViewById(R.id.roomNum);
            scheduleStatus = itemView.findViewById(R.id.scheduleStatus);
            temp = itemView.findViewById(R.id.Temp);
            isOccupied = itemView.findViewById(R.id.isOccupied);

            // Set click listener to trigger the adapter interface method
            itemView.setOnClickListener(new View.OnClickListener() {
                @Override
                public void onClick(View v) {
```



```
        if (roomAdapterInterface != null) {
            int pos = getAdapterPosition();
            if (pos != RecyclerView.NO_POSITION) {
                roomAdapterInterface.OnItemClick(pos);
            }
        }
    }
});
}

@NonNull
@Override
public RoomViewHolder onCreateViewHolder(@NonNull ViewGroup parent,
int viewType) {
    // Inflate a layout for each room item
    View itemView =
LayoutInflater.from(context).inflate(R.layout.room_item, parent, false);
    return new RoomViewHolder(itemView, roomAdapterInterface);
}

@Override
public void onBindViewHolder(@NonNull RoomViewHolder holder, int
position) {
    // Get the current Room object from the list
    Rooms room = roomList.get(position);
    boolean IsNOSced = false;

    // Ensure schedule exists and is not empty
    if (room.getSchedule() != null && !room.getSchedule().isEmpty()) {
        GetDay getDay = new GetDay();
        int day = getDay.getCurrentDay() - 1; // Convert 1-based day
to 0-based index
        String scheduleValue = "";
        Integer hourLabel = GetHour.getScheduleHour(); // Logical hour
(e.g., 1-9 for school periods)
        Integer hour = GetHour.getCurrentHour(); // Actual
current hour (e.g., 13:00 → 13)

        if (day >= 0 && day < 5) {
            // Get list of statuses for a specific hour
            List<String> dayStatus =
room.getSchedule().get(hourLabel.toString());
            if (dayStatus != null && dayStatus.size() > day) {
                scheduleValue = dayStatus.get(day); // e.g.,
"Occupied" or "Free"
            }
        }

        // Set schedule based on time conditions
        if (!IsNOSced) {
            holder.scheduleStatus.setText(scheduleValue);
        }
    }
}
```



```
if (hourLabel == -1) {
    // Case: current time is during break
    holder.scheduleStatus.setText("Brake");
}

if (hour < 8 || hour > 16) {
    // Case: outside of school hours
    holder.scheduleStatus.setText("The end of the day");
    IsNOSced = true;
}

if (day > 4) {
    // Case: weekend (Friday-Saturday)
    holder.scheduleStatus.setText("Its The Weekend");
    IsNOSced = true;
}

} else {
    // If the schedule map is null or empty
    holder.scheduleStatus.setText("No schedule available");
}

// Set other room data
holder.roomNum.setText("Room number: " + room.getRoomNum());
holder.temp.setText(room.getTemp() + "°");
holder.isOccupied.setText(room.getIsOccupied() ? "Occupied" :
"Available");
}

@Override
public int getItemCount() {
    // Return the total number of rooms
    return roomList.size();
}
}
```



104



```
mondaySpinner = itemView.findViewById(R.id.spinner_monday);
tuesdaySpinner = itemView.findViewById(R.id.spinner_tuesday);
wednesdaySpinner =
itemView.findViewById(R.id.spinner_wednesday);
thursdaySpinner =
itemView.findViewById(R.id.spinner_thursday);
    }
}

// ViewHolder for the header row (static labels, no logic required)
public static class HeaderViewHolder extends RecyclerView.ViewHolder {
    public HeaderViewHolder(View itemView) {
        super(itemView);
    }
}

@NonNull
@Override
public RecyclerView.ViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
    // Inflate the appropriate layout depending on whether this is a
    header or item
    if (viewType == VIEW_TYPE_HEADER) {
        View headerView =
LayoutInflater.from(context).inflate(R.layout.item_schedule_header,
parent, false);
        return new HeaderViewHolder(headerView);
    } else {
        View view =
LayoutInflater.from(context).inflate(R.layout.item_schedule_rv, parent,
false);
        return new ScheduleViewHolder(view);
    }
}

@Override
public void onBindViewHolder(@NonNull RecyclerView.ViewHolder holder,
int position) {
    if (getItemViewType(position) == VIEW_TYPE_HEADER) {
        // Header is static; no dynamic data binding needed
    } else {
        // Get actual data entry by subtracting 1 due to header
        Map.Entry<String, List<String>> entry =
scheduleList.get(position - 1);
        String hour = entry.getKey(); // e.g., "Hour
1"
        List<String> statuses = entry.getValue(); // List of 5
statuses (Sunday-Thursday)

        ScheduleViewHolder scheduleHolder = (ScheduleViewHolder)
holder;
        scheduleHolder.hourTextView.setText(String.valueOf(hour));

        // Initialize each spinner with status options and select the
```



```
current one
        setupSpinner(scheduleHolder.sundaySpinner, statuses.get(0));
        setupSpinner(scheduleHolder.mondaySpinner, statuses.get(1));
        setupSpinner(scheduleHolder.tuesdaySpinner, statuses.get(2));
        setupSpinner(scheduleHolder.wednesdaySpinner,
statuses.get(3));
        setupSpinner(scheduleHolder.thursdaySpinner, statuses.get(4));
    }
}

// Helper function to set up a spinner and select the default value
private void setupSpinner(Spinner spinner, String defaultValue) {
    ArrayAdapter<CharSequence> adapter =
ArrayAdapter.createFromResource(
        context,
        R.array.room_statuses, // Defined in
res/values/strings.xml
        android.R.layout.simple_spinner_item
    );

    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_i
tem);

    spinner.setAdapter(adapter);

    int position = adapter.getPosition(defaultValue);
    spinner.setSelection(position);
}

@Override
public int getItemCount() {
    // +1 for the header row
    return scheduleList.size() + 1;
}

@Override
public int getItemViewType(int position) {
    // Position 0 is the header
    return (position == 0) ? VIEW_TYPE_HEADER : VIEW_TYPE_ITEM;
}

/**
 * Extracts the current state of the schedule from the RecyclerView
spinners.
 * If a ViewHolder is currently bound and visible, read the updated
values.
 * Otherwise, fall back to the original map values.
 */
public Map<String, List<String>> getScheduleData() {
    Map<String, List<String>> updatedSchedule = new HashMap<>();

    for (int i = 0; i < scheduleList.size(); i++) {
        Map.Entry<String, List<String>> entry = scheduleList.get(i);
        ScheduleViewHolder holder = (ScheduleViewHolder)
recyclerView.findViewHolderForAdapterPosition(i + 1); // +1 for header
```



```
offset

    if (holder != null) {
        // Read updated selections from the spinners
        updatedSchedule.put(entry.getKey(), new
ArrayList<String>() {{
add(holder.sundaySpinner.getSelectedItem().toString());
add(holder.mondaySpinner.getSelectedItem().toString());
add(holder.tuesdaySpinner.getSelectedItem().toString());
add(holder.wednesdaySpinner.getSelectedItem().toString());
add(holder.thursdaySpinner.getSelectedItem().toString());
        }});
    } else {
        // If view is not bound (e.g., offscreen), use existing
values
        updatedSchedule.put(entry.getKey(), entry.getValue());
    }

    return updatedSchedule;
}
```



ScheduleTeachAdapter

```
package com.example.projectarieleyalyotamyona.Adapters;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.R;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

public class ScheduleTeachAdapter extends
RecyclerView.Adapter<RecyclerView.ViewHolder> {

    /**
     * Context from the calling activity or fragment.
     */
    private final Context context;

    /**
     * List of schedule entries, each mapping an hour string to a list of
     status strings per weekday.
     */
    private final List<Map.Entry<String, List<String>>> scheduleList;

    /**
     * View type identifier for header row.
     */
    private static final int VIEW_TYPE_HEADER = 0;

    /**
     * View type identifier for normal schedule item row.
     */
    private static final int VIEW_TYPE_ITEM = 1;

    /**
     * Constructor that initializes the adapter with context and schedule
     map.
     * Converts the map into a list for easier access by position.
     *
     * @param context Context from the activity or fragment using this
     adapter.
     * @param schedule Map where key is hour (String) and value is a List
     of status strings per day.
     */
}
```



```
public ScheduleTeachAdapter(Context context, Map<String, List<String>>
schedule) {
    this.context = context;
    this.scheduleList = new ArrayList<>(schedule.entrySet());
}

/**
 * ViewHolder class representing a single schedule row displaying an
 * hour and statuses for weekdays.
 */
public static class ScheduleViewHolder extends RecyclerView.ViewHolder
{
    TextView hourTextView;
    TextView sundayTextView, mondayTextView, tuesdayTextView,
    wednesdayTextView, thursdayTextView;

    /**
     * Constructor that finds and holds references to the relevant
     * TextViews in the item layout.
     */
    * @param itemView The inflated item view layout.
    */
    public ScheduleViewHolder(@NonNull View itemView) {
        super(itemView);
        hourTextView = itemView.findViewById(R.id.hour);
        sundayTextView = itemView.findViewById(R.id.tv_sunday);
        mondayTextView = itemView.findViewById(R.id.tv_monday);
        tuesdayTextView = itemView.findViewById(R.id.tv_tuesday);
        wednesdayTextView = itemView.findViewById(R.id.tv_wednesday);
        thursdayTextView = itemView.findViewById(R.id.tv_thursday);
    }
}

/**
 * ViewHolder class representing the static header row (with weekday
 * labels).
 */
public static class HeaderViewHolder extends RecyclerView.ViewHolder {
    /**
     * Constructor for header view holder.
     */
    * @param itemView The inflated header layout.
    */
    public HeaderViewHolder(View itemView) {
        super(itemView);
    }
}

/**
 * Creates new ViewHolder objects as needed.
 * Inflates header or item layout based on the view type.
 */
* @param parent The parent ViewGroup that the created view will be
```



```
attached to.
    * @param viewType The view type to create (header or item).
    * @return A new ViewHolder instance corresponding to the given view
type.
    */
    @NonNull
    @Override
    public RecyclerView.ViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
        if (viewType == VIEW_TYPE_HEADER) {
            View headerView =
LayoutInflater.from(context).inflate(R.layout.item_schedule_header,
parent, false);
            return new HeaderViewHolder(headerView);
        } else {
            View view =
LayoutInflater.from(context).inflate(R.layout.item_schedule_teach_rv,
parent, false);
            return new ScheduleViewHolder(view);
        }
    }

    /**
     * Binds schedule data to the ViewHolder at the given position.
     * Skips binding for the static header.
     *
     * @param holder The ViewHolder instance to bind data to.
     * @param position The position in the adapter (0 is header).
     */
    @Override
    public void onBindViewHolder(@NonNull RecyclerView.ViewHolder holder,
int position) {
        if (getItemViewType(position) == VIEW_TYPE_HEADER) {
            // Static header: no binding needed
        } else {
            Map.Entry<String, List<String>> entry =
scheduleList.get(position - 1); // offset by 1 because header occupies
position 0

            String hour = entry.getKey();
            List<String> statuses = entry.getValue();

            ScheduleViewHolder scheduleHolder = (ScheduleViewHolder)
holder;

            scheduleHolder.hourTextView.setText(hour);
            scheduleHolder.sundayTextView.setText(statuses.get(0));
            scheduleHolder.mondayTextView.setText(statuses.get(1));
            scheduleHolder.tuesdayTextView.setText(statuses.get(2));
            scheduleHolder.wednesdayTextView.setText(statuses.get(3));
            scheduleHolder.thursdayTextView.setText(statuses.get(4));
        }
    }

    /**
     * Returns the total number of items in the adapter, including the
```



```
header.  
    *  
    * @return Number of schedule rows plus one header row.  
    */  
    @Override  
    public int getItemCount() {  
        return scheduleList.size() + 1;  
    }  
  
    /**  
    * Determines the type of view for a given position (header or item).  
    *  
    * @param position The adapter position.  
    * @return VIEW_TYPE_HEADER if position is 0; otherwise  
    VIEW_TYPE_ITEM.  
    */  
    @Override  
    public int getItemViewType(int position) {  
        return (position == 0) ? VIEW_TYPE_HEADER : VIEW_TYPE_ITEM;  
    }  
}
```



UserRoomsAdapter

```
package com.example.projectarieleyalyotamyona.Adapters;

import android.app.AlertDialog;
import android.content.Context;
import android.content.DialogInterface;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageButton;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.R;
import com.example.projectarieleyalyotamyona.Data_Modules.User;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.List;
import java.util.Map;

public class UserRoomsAdapter extends
RecyclerView.Adapter<UserRoomsAdapter.UserRoomsViewHolder> {

    /**
     * List of bookings to display.
     */
    private final List<User.Booking> bookingList;

    /**
     * Context from the calling activity or fragment.
     */
    private final Context context;

    /**
     * Constructor initializing the adapter with context and booking list.
     *
     * @param context The context of the activity or fragment.
     * @param bookingList List of User.Booking objects representing the
     user's bookings.
     */
    public UserRoomsAdapter(Context context, List<User.Booking>
bookingList) {
        this.context = context;
        this.bookingList = bookingList;
    }

    /**
```



```
* Inflates the item layout and creates the ViewHolder for each
booking item.
*
* @param parent    The parent ViewGroup into which the new view will
be added.
* @param viewType The view type of the new View.
* @return A new UserRoomsViewHolder instance.
*/
@NonNull
@Override
public UserRoomsViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
    View view =
LayoutInflater.from(context).inflate(R.layout.item_user_rooms, parent,
false);
    return new UserRoomsViewHolder(view);
}

/**
 * Binds booking data to the ViewHolder at the given position.
 * Sets texts and click listener for the delete button with
confirmation dialog.
 *
 * @param holder    The ViewHolder to bind data to.
 * @param position  The adapter position.
 */
@Override
public void onBindViewHolder(@NonNull UserRoomsViewHolder holder, int
position) {
    User.Booking booking = bookingList.get(position);

    holder.roomNum.setText("Room: " + booking.getRoomNumber());
    holder.status.setText("Day: " + booking.getDay());
    holder.temp.setText("Hours: " + String.join(", ",
booking.getHourStatuses()));

    holder.trashButton.setOnClickListener(v -> {
        new AlertDialog.Builder(context)
            .setMessage("Are you sure you want to delete this
booking?")
            .setCancelable(false)
            .setPositiveButton("Yes", (dialog, id) -> {
                int positionToDelete =
holder.getAdapterPosition();
                if (positionToDelete != RecyclerView.NO_POSITION)
                {
                    bookingList.remove(positionToDelete);
                    notifyItemRemoved(positionToDelete);
                    UpdateBooking(bookingList);
                    UpdateRoomSched(booking);
                    Toast.makeText(context, "Booking deleted!",
Toast.LENGTH_SHORT).show();
                }
            })
    })
}
```



```
.setNegativeButton("No", null)
.show();

});

}

/**
 * Updates the user's bookedRooms field in Firestore with the current
 * booking list.
 *
 * @param bookingList List of current bookings to update in Firestore.
 */
public static void UpdateBooking(List<User.Booking> bookingList) {
    FirebaseAuth mAuth = FirebaseAuth.getInstance();
    FirebaseUser currentUser = mAuth.getCurrentUser();
    String userId = currentUser.getId();

    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("users").document(userId)
        .update("bookedRooms", bookingList);
}

/**
 * Updates the schedule of the room in Firestore to mark the
 * previously booked hours as available.
 *
 * @param booking The booking object containing room number, day, and
 * hours to free.
 */
public static void UpdateRoomSched(User.Booking booking) {
    FirebaseFirestore db = FirebaseFirestore.getInstance();

    db.collection("Rooms")
        .document(booking.getRoomNumber())
        .get()
        .addOnSuccessListener(document -> {
            if (document.exists()) {
                // Retrieve the schedule map from Firestore
                document
                    Map<String, List<String>> schedule = (Map<String,
                    List<String>>) document.get("weekSched");
                if (schedule == null) {
                    Log.e("Firestore", "Schedule is null for room:
                    " + booking.getRoomNumber());
                    return;
                }

                // Get day index for the booking day
                int dayIndex = getDayNumber(booking.getDay());

                // Iterate through schedule hours and mark the
                booked day as "Available"
                for (int i = 0; i < schedule.size(); i++) {
                    List<String> hourSched =
                    schedule.get(String.valueOf(i));
```



```
        if (hourSched != null && dayIndex <
hourSched.size()) {
            hourSched.set(dayIndex, "Available");
            schedule.put(String.valueOf(i),
hourSched);
        }
    }

    // Update the schedule back in Firestore
    db.collection("Rooms")
        .document(booking.getRoomNumber())
        .update("weekSched", schedule)
        .addOnSuccessListener(aVoid ->
Log.d("Firestore", "Schedule updated for room: " +
booking.getRoomNumber()))
        .addOnFailureListener(e ->
Log.e("Firestore", "Failed to update schedule for room: " +
booking.getRoomNumber(), e));
    } else {
        Log.w("Firestore", "Room not found: " +
booking.getRoomNumber());
    }
}

    .addOnFailureListener(e -> Log.e("Firestore", "Error
fetching room: " + booking.getRoomNumber(), e));
}

/**
 * Returns the total number of bookings to display in the adapter.
 *
 * @return Number of bookings.
 */
@Override
public int getItemCount() {
    return bookingList.size();
}

/**
 * ViewHolder class representing each booking item row.
 */
public static class UserRoomsViewHolder extends
RecyclerView.ViewHolder {
    TextView roomNum, status, temp;
    ImageButton trashButton;

    /**
     * Constructor finds and holds references to the views for a
     booking item.
     *
     * @param itemView The inflated item view.
     */
    public UserRoomsViewHolder(@NonNull View itemView) {
        super(itemView);
        roomNum = itemView.findViewById(R.id.roomNum);
    }
}
```



```
        status = itemView.findViewById(R.id.status);
        temp = itemView.findViewById(R.id.Temp);
        trashButton = itemView.findViewById(R.id.trashButton);
    }
}

/**
 * Converts a day name string to its corresponding numerical index (0-
 * 6).
 *
 * @param day Name of the day (e.g. "Monday").
 * @return The index of the day (Sunday = 0, Monday = 1, ..., Saturday
 * = 6).
 * @throws IllegalArgumentException if the day string is invalid.
 */
public static int getDayNumber(String day) {
    switch(day.toLowerCase()) {
        case "sunday": return 0;
        case "monday": return 1;
        case "tuesday": return 2;
        case "wednesday": return 3;
        case "thursday": return 4;
        case "friday": return 5;
        case "saturday": return 6;
        default: throw new IllegalArgumentException("Invalid day: " +
day);
    }
}
}
```



Rooms

```
package com.example.projectarieleyalyotamyona.Data_Modules;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Represents a room with its schedule, weekly schedule, temperature, and
 * occupancy status.
 * Implements Serializable for object serialization.
 */
public class Rooms implements Serializable {
    private String roomNum;
    private Map<String, List<String>> schedule;
    private Map<String, List<String>> weekSched;
    private String temp;
    private boolean isOccupied;

    /**
     * Constructor that initializes the room with a given schedule.
     * Copies the given schedule to the weekly schedule.
     *
     * @param roomNum The room number.
     * @param schedule The schedule map with hour labels and list of
     * statuses per day.
     */
    public Rooms(String roomNum, Map<String, List<String>> schedule) {
        this.roomNum = roomNum;
        this.schedule = schedule;
        this.weekSched = deepCopy(schedule);
        this.temp = "25";
        this.isOccupied = false;
    }

    /**
     * Constructor that initializes the room with a default schedule (5
     * weekdays, 9 hours).
     * All hours are marked as "Available" by default.
     *
     * @param roomNum The room number.
     */
    public Rooms(String roomNum) {
        this.roomNum = roomNum;
        this.schedule = new LinkedHashMap<>();

        for (int hour = 1; hour <= 9; hour++) {
            String hourLabel = "Hour " + hour;
            List<String> dayStatus = new ArrayList<>();
            for (int i = 0; i < 5; i++) {
                dayStatus.add("Available");
            }
        }
    }
}
```



```
        }
        this.schedule.put(hourLabel, dayStatus);
    }

    this.weekSched = deepCopy(this.schedule);
    this.temp = "25";
    this.isOccupied = false;
}

/**
 * Creates a deep copy of a schedule map.
 *
 * @param original The original map to copy.
 * @return A new map with copied keys and copied lists of statuses.
 */
private Map<String, List<String>> deepCopy(Map<String, List<String>>
original) {
    Map<String, List<String>> copy = new LinkedHashMap<>();
    for (Map.Entry<String, List<String>> entry : original.entrySet())
    {
        copy.put(entry.getKey(), new ArrayList<>(entry.getValue()));
    }
    return copy;
}

// Setters

/**
 * Sets the room number.
 *
 * @param roomNum The new room number.
 */
public void setRoomNum(String roomNum) {
    this.roomNum = roomNum;
}

/**
 * Sets the room temperature.
 *
 * @param temp The new temperature.
 */
public void setTemp(String temp) {
    this.temp = temp;
}

/**
 * Sets the occupancy status.
 *
 * @param occupied True if occupied, false otherwise.
 */
public void setOccupied(boolean occupied) {
    isOccupied = occupied;
}
```



```
/**
 * Sets the schedule map.
 *
 * @param schedule The new schedule map.
 */
public void setSchedule(Map<String, List<String>> schedule) {
    this.schedule = schedule;
}

/**
 * Sets the weekly schedule map.
 *
 * @param weekSched The new weekly schedule map.
 */
public void setWeekSched(Map<String, List<String>> weekSched) {
    this.weekSched = weekSched;
}

// Getters

/**
 * Returns the room number.
 *
 * @return The room number string.
 */
public String getRoomNum() {
    return roomNum;
}

/**
 * Returns the schedule map.
 *
 * @return The schedule map.
 */
public Map<String, List<String>> getSchedule() {
    return schedule;
}

/**
 * Returns the weekly schedule map.
 *
 * @return The weekly schedule map.
 */
public Map<String, List<String>> getWeekSched() {
    return weekSched;
}

/**
 * Returns the temperature as a string.
 *
 * @return The temperature.
 */
public String getTemp() {
    return temp;
}
```



```
}

/**
 * Returns the occupancy status.
 *
 * @return True if the room is occupied, false otherwise.
 */
public boolean getIsOccupied() {
    return isOccupied;
}

/**
 * Returns a string representation of the room, including schedule
 * details.
 *
 * @return A detailed string with room number, schedule, temperature,
 * occupancy, and weekly schedule.
 */
@Override
public String toString() {
    StringBuilder scheduleDetails = new StringBuilder();
    for (Map.Entry<String, List<String>> entry : schedule.entrySet())
    {
        scheduleDetails.append(entry.getKey()).append(": ");
        List<String> statuses = entry.getValue();
        for (int i = 0; i < statuses.size(); i++) {
            scheduleDetails.append(statuses.get(i)).append(" ")
                .append(getDayByIndex(i)).append(" ");
        }
        scheduleDetails.append("\n");
    }

    return "Rooms{" +
        "roomNum=" + roomNum + '\'' +
        ", schedule=\n" + scheduleDetails +
        ", temp=" + temp + '\'' +
        ", isOccupied=" + isOccupied +
        ", weekSched=" + weekSched +
        '\'' +
        '}';
}

/**
 * Helper method to convert day index to day name (Sunday to
 * Thursday).
 *
 * @param index Index of the day (0-4).
 * @return The day name string, or "Invalid Day" if out of range.
 */
private String getDayByIndex(int index) {
    String[] days = {"Sunday", "Monday", "Tuesday", "Wednesday",
"Thursday"};
    return (index >= 0 && index < days.length) ? days[index] :
"Invalid Day";
}
```



5 יחידות לימוד מדעי המחשב - (luz) - אריאל חלף

```
}  
}
```



User&Booking

```
package com.example.projectarieleyalyotamyona.Data_Modules;

import java.util.ArrayList;
import java.util.List;

/**
 * Represents a user with email, name, role flags, class affiliation, and
 * booked rooms.
 */
public class User {
    private String email;
    private String name;
    private Boolean isT; // Indicates if the user is a teacher
    private Boolean isStaff; // Indicates if the user is staff
    private String class1; // Class the user belongs to
    private List<Booking> bookedRooms; // List of rooms booked by the user

    /**
     * Constructor initializing a User with given details.
     *
     * @param email    User's email.
     * @param name     User's full name.
     * @param class1   Class affiliation.
     * @param isT      True if user is a teacher.
     * @param isStaff  True if user is staff.
     */
    public User(String email, String name, String class1, Boolean isT,
        Boolean isStaff) {
        this.email = email;
        this.name = name;
        this.class1 = class1;
        this.isT = isT;
        this.isStaff = isStaff;
        this.bookedRooms = new ArrayList<>();
    }

    /**
     * Default constructor initializing empty bookedRooms list.
     */
    public User() {
        this.bookedRooms = new ArrayList<>();
    }

    // --- Getters and Setters ---

    /**
     * Returns the class affiliation of the user.
     *
     * @return Class name.
     */
    public String getClass1() {
        return class1;
    }
}
```



```
}

/**
 * Sets the class affiliation of the user.
 *
 * @param class1 Class name.
 */
public void setClass1(String class1) {
    this.class1 = class1;
}

/**
 * Returns true if the user is a teacher.
 *
 * @return True if teacher.
 */
public Boolean getIsT() {
    return isT;
}

/**
 * Sets whether the user is a teacher.
 *
 * @param isT True if teacher.
 */
public void setT(Boolean isT) {
    this.isT = isT;
}

/**
 * Alternate setter for teacher flag.
 *
 * @param IsT True if teacher.
 */
public void setTeachPass(boolean IsT) {
    this.isT = IsT;
}

/**
 * Returns the user's email.
 *
 * @return Email string.
 */
public String getEmail() {
    return email;
}

/**
 * Sets the user's email.
 *
 * @param email Email string.
 */
public void setEmail(String email) {
    this.email = email;
}
```



```
}

/**
 * Returns true if the user is staff.
 *
 * @return True if staff.
 */
public Boolean getStaff() {
    return isStaff;
}

/**
 * Sets whether the user is staff.
 *
 * @param staff True if staff.
 */
public void setStaff(Boolean staff) {
    isStaff = staff;
}

/**
 * Returns the user's full name.
 *
 * @return Name string.
 */
public String getName() {
    return name;
}

/**
 * Sets the user's full name.
 *
 * @param name Name string.
 */
public void setName(String name) {
    this.name = name;
}

/**
 * Returns the list of booked rooms.
 *
 * @return List of Booking objects.
 */
public List<Booking> getBookedRooms() {
    return bookedRooms;
}

/**
 * Sets the list of booked rooms.
 *
 * @param bookedRooms List of Booking objects.
 */
public void setBookedRooms(List<Booking> bookedRooms) {
    this.bookedRooms = bookedRooms;
}
```



```
}

/**
 * Returns a string representation of the User.
 *
 * @return User details string.
 */
@Override
public String toString() {
    return "User{" +
        "email='" + email + '\'' +
        ", name='" + name + '\'' +
        ", isT=" + isT +
        ", isStaff=" + isStaff +
        ", class1='" + class1 + '\'' +
        ", bookedRooms=" + bookedRooms +
        '}';
}

/**
 * Nested static class representing a booking made by the user.
 */
public static class Booking {
    private String roomNumber; // Room number booked
    private String day; // Day of the booking
    private List<String> hourStatuses; // List of hour status strings
    for the booking day

    /**
     * Constructor initializing booking with details.
     *
     * @param roomNumber Room number string.
     * @param day Day of the booking.
     * @param hourStatuses List of hour statuses (e.g., "Busy",
     "Available").
     */
    public Booking(String roomNumber, String day, List<String>
hourStatuses) {
        this.roomNumber = roomNumber;
        this.day = day;
        this.hourStatuses = hourStatuses;
    }

    /**
     * Default constructor needed for Firebase deserialization.
     */
    public Booking() {
    }

    /**
     * Returns the room number of the booking.
     *
     * @return Room number string.
     */
}
```



```
public String getRoomNumber() {
    return roomNumber;
}

/**
 * Sets the room number of the booking.
 *
 * @param roomNumber Room number string.
 */
public void setRoomNumber(String roomNumber) {
    this.roomNumber = roomNumber;
}

/**
 * Returns the day of the booking.
 *
 * @return Day string.
 */
public String getDay() {
    return day;
}

/**
 * Sets the day of the booking.
 *
 * @param day Day string.
 */
public void setDay(String day) {
    this.day = day;
}

/**
 * Returns the list of hour statuses.
 *
 * @return List of hour status strings.
 */
public List<String> getHourStatuses() {
    return hourStatuses;
}

/**
 * Sets the list of hour statuses.
 *
 * @param hourStatuses List of hour status strings.
 */
public void setHourStatuses(List<String> hourStatuses) {
    this.hourStatuses = hourStatuses;
}

/**
 * Returns a string representation of the booking.
 *
 * @return Booking details string.
 */
```



```
@Override
public String toString() {
    return "Booking{" +
        "roomNumber='" + roomNumber + '\'' +
        ", day='" + day + '\'' +
        ", hourStatuses=" + hourStatuses +
        '}';
}
}
```



Fragments

AccountFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.app.Activity;
import android.app.AlertDialog;
import android.content.Context;
import android.content.DialogInterface;
import android.content.Intent;
import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.net.Uri;
import android.os.Bundle;
import android.provider.MediaStore;
import android.util.Base64;
import android.util.Patterns;
import android.view.Gravity;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.view.animation.Animation;
import android.view.animation.AnimationUtils;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.PopupWindow;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import com.example.projectarieleyalyotamyona.LoginActivity;
import com.example.projectarieleyalyotamyona.MainActivity;
import com.example.projectarieleyalyotamyona.Utilities.PrefManager;
import com.example.projectarieleyalyotamyona.R;
import com.google.firebase.auth.AuthCredential;
import com.google.firebase.auth.EmailAuthProvider;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.crashlytics.buildtools.reloc.org.apache.commons.io.output.ByteArrayOutputStream;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.FirebaseFirestore;
```



```
import java.io.IOException;

/**
 * A Fragment that displays and manages the user's account information.
 * This Fragment allows users to view and update their profile
information,
 * including name, email, class, password, and profile picture.
 * <p>
 * The Fragment interacts with Firebase Authentication and Firestore to
 * manage user data persistence and authentication operations.
 */
public class AccountFragment extends Fragment {
    private static final int PICK_IMAGE_REQUEST = 1;

    // UI elements
    private ImageView profilePicture;
    private TextView textName, textEmail, textClass;
    private ImageButton delProfilePicture;

    // Firebase instances
    private FirebaseAuth mAuth;
    private FirebaseFirestore db;
    private FirebaseUser currentUser;

    /**
     * Called to have the fragment instantiate its user interface view.
     * This is optional, and non-graphical fragments can return null.
     *
     * @param inflater The LayoutInflater object that can be used to
inflate
     * any views in the fragment
     * @param container If non-null, this is the parent view that the
fragment's
     * UI should be attached to
     * @param savedInstanceState If non-null, this fragment is being re-
constructed
     * from a previous saved state as given here
     * @return Return the View for the fragment's UI, or null
     */
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable
ViewGroup container, @Nullable Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        View view = inflater.inflate(R.layout.fragment_account, container,
false);

        // Initialize UI elements
        profilePicture = view.findViewById(R.id.profile_picture);
        textName = view.findViewById(R.id.text_name);
        textEmail = view.findViewById(R.id.text_email);
        textClass = view.findViewById(R.id.text_class);
        delProfilePicture = view.findViewById(R.id.delProfilePicture);
    }
}
```



```
ImageButton btnEditName = view.findViewById(R.id.btn_edit_name);
ImageButton btnEditEmail = view.findViewById(R.id.btn_edit_email);
ImageButton btnEditClass = view.findViewById(R.id.btn_edit_class);
ImageButton btnEditPassword =
view.findViewById(R.id.btn_edit_password);
TextView changePicture = view.findViewById(R.id.change_picture);

// Initialize Firebase authentication and database instances
mAuth = FirebaseAuth.getInstance();
db = FirebaseFirestore.getInstance();
currentUser = mAuth.getCurrentUser();

// Load user data from Firestore
loadUserData();

if (profilePicture!=null)
{
    delProfilePicture.setOnClickListener(new
View.OnClickListener() {
        @Override
        public void onClick(View view) {
            new AlertDialog.Builder(view.getContext())
                .setTitle("Delete Profile Picture")
                .setMessage("Are you sure you want to delete
your profile picture?")
                .setPositiveButton("Yes", new
DialogInterface.OnClickListener() {
                    @Override
                    public void onClick(DialogInterface
dialog, int which) {
                        db.collection("users").document(currentUser.getId())
                            .update("profilePicture", "")
                            .addOnSuccessListener(aVoid -> {
                                profilePicture.setImageResource(R.drawable.ic_account);
                                MainActivity.profilePic.setImageResource(R.drawable.ic_account);
                                Toast.makeText(view.getContext(), "Profile picture deleted",
                                Toast.LENGTH_SHORT).show();
                            })
                            .addOnFailureListener(e -> {
                                Toast.makeText(view.getContext(), "Failed to delete picture",
                                Toast.LENGTH_SHORT).show();
                            });
                        }
                    })
                .setNegativeButton("Cancel", null)
                .show();
        }
    })
}
```



```
});  
}  
  
// Set up click listeners for editing profile fields  
changePicture.setOnClickListener(v -> openImageChooser());  
btnEditName.setOnClickListener(v -> editField("name"));  
btnEditEmail.setOnClickListener(v -> showPopup(getView()));  
btnEditClass.setOnClickListener(v -> editField("class"));  
btnEditPassword.setOnClickListener(v -> resetPassword());  
  
return view;  
}  
  
/**  
 * Loads the user data from Firestore and updates the UI elements with  
the retrieved information.  
 * This includes the user's name, email, class, and profile picture.  
 */  
private void loadUserData() {  
    if (currentUser != null) {  
        DocumentReference docRef =  
db.collection("users").document(currentUser.getId());  
        docRef.get().addOnSuccessListener(documentSnapshot -> {  
            if (documentSnapshot.exists()) {  
                textName.setText(documentSnapshot.getString("name"));  
  
textEmail.setText(documentSnapshot.getString("email"));  
                if (documentSnapshot.getString("class") != null) {  
textClass.setText(documentSnapshot.getString("class"));  
                } else {  
                    textClass.setText("You are not a student");  
                }  
  
                // Load profile picture  
                String base64Image =  
documentSnapshot.getString("profilePicture");  
                if (base64Image != null && base64Image.length() > 1) {  
                    Bitmap bitmap = decodeBase64ToImage(base64Image);  
                    profilePicture.setImageBitmap(bitmap);  
                }  
                profilePicture.setVisibility(View.VISIBLE);  
            }  
        }).addOnFailureListener(e -> {  
            Toast.makeText(getActivity(), "Failed to load user data",  
Toast.LENGTH_SHORT).show();  
        });  
    }  
}  
  
/**  
 * Displays a Toast message informing the user that certain fields can  
only be edited
```



```
* by contacting school management.
*
* @param field The name of the field that the user is attempting to
edit
*/
private void editField(String field) {
    Toast.makeText(getActivity(), "To Edit your " + field+" please
contact the school management", Toast.LENGTH_SHORT).show();
}

/**
 * Displays a popup window that allows the user to update their email
address.
 * The popup contains fields for entering a new email and current
password for verification.
 *
 * @param view The view to which the popup window will be anchored
 */
private void showPopup(View view) {
    LayoutInflater inflater = (LayoutInflater)
getContext().getSystemService(Context.LAYOUT_INFLATER_SERVICE);
    View popupView = inflater.inflate(R.layout.popup_layout, null);

    LinearLayout layout = popupView.findViewById(R.id.mainLayout);
    Animation slideUp = AnimationUtils.loadAnimation(getContext(),
R.anim.slide_up);
    layout.startAnimation(slideUp);

    PopupWindow popupWindow = new PopupWindow(
        popupView,
        1000, // Set a fixed width (adjust as needed)
        970, // Set a fixed height (adjust as needed)
        true
    );

    EditText editEmail = popupView.findViewById(R.id.editEmail);
    EditText currentPass = popupView.findViewById(R.id.reEnterPass);
    Button btnUpdateEmail =
popupView.findViewById(R.id.btnUpdateEmail);
    Button btnCancel = popupView.findViewById(R.id.btnCancel);

    btnCancel.setOnClickListener(view1 -> {
        popupWindow.dismiss();
    });

    btnUpdateEmail.setOnClickListener(v -> {
        String newEmail = editEmail.getText().toString().trim();
        if (!newEmail.isEmpty()) {
            updateEmail(newEmail,
currentPass.getText().toString().trim(), popupWindow);
        } else {
            Toast.makeText(this.getContext(), "Please enter an email",
Toast.LENGTH_SHORT).show();
        }
    });
}
```



```
    });

    popupWindow.showAtLocation(view, Gravity.CENTER, 0, 0);
}

/**
 * Updates the user's email address in Firebase Authentication.
 * This method re-authenticates the user with their current password
 * before updating the email,
 * and sends a verification email to the new address.
 *
 * @param newEmail The new email address to be set for the user
 * @param password The user's current password for re-authentication
 * @param popupWindow The popup window that will be dismissed upon
 * completion
 */
private void updateEmail(String newEmail, String password, PopupWindow
popupWindow) {
    if (!Patterns.EMAIL_ADDRESS.matcher(newEmail).matches()) {
        Toast.makeText(popupWindow.getContentView().getContext(),
            "Please enter a valid email address.",
            Toast.LENGTH_SHORT).show();
        return;
    }

    if (currentUser != null) {
        // Ensure currentUser.getEmail() is not null
        if (currentUser.getEmail() == null) {
            Toast.makeText(popupWindow.getContentView().getContext(),
                "Error: Current user email is null. Try signing in
again.",
                Toast.LENGTH_SHORT).show();
            return;
        }

        AuthCredential credential =
EmailAuthProvider.getCredential(currentUser.getEmail(), password);

currentUser.reauthenticate(credential).addOnCompleteListener(task -> {
    if (task.isSuccessful()) {
        currentUser.verifyBeforeUpdateEmail(newEmail)
            .addOnSuccessListener(aVoid -> {

                Toast.makeText(popupWindow.getContentView().getContext(),
                    "Verification email sent. Please
check your inbox.",
                    Toast.LENGTH_SHORT).show();
                popupWindow.dismiss();
                PrefManager prefManager = new
PrefManager(this.getContext());
                prefManager.Forgetlog();
                Intent intent = new
```



```
Intent(this.getContext(), LoginActivity.class);
        startActivity(intent);
    })
    .addOnFailureListener(e -> {

Toast.makeText(popupWindow.getContentView().getContext(),
                e.getMessage(),
                Toast.LENGTH_SHORT).show();

    });

    } else {
        Exception exception = task.getException();

Toast.makeText(popupWindow.getContentView().getContext(),
                "Re-authentication failed: " + (exception !=
null ? exception.getMessage() : "Unknown error"),
                Toast.LENGTH_SHORT).show();

    }
    });
    } else {
        Toast.makeText(popupWindow.getContentView().getContext(),
            "Error: User not logged in.",
            Toast.LENGTH_SHORT).show();

    }

}

/**
 * Displays a confirmation dialog and initiates the password reset
 * process.
 * This method sends a password reset email to the user's registered
 * email address.
 */
private void resetPassword() {
    if (currentUser != null) {
        AlertDialog dialog = new AlertDialog.Builder(getActivity())
            .setTitle("Confirm Password Reset")
            .setMessage("Are you sure you want to reset your
password? A reset email will be sent to " + currentUser.getEmail())
            .setPositiveButton("Confirm", (d, w) ->
resetPassword())
            .setNegativeButton("Cancel", null)
            .show();

        // Disable automatic uppercase

dialog.getButton(AlertDialog.BUTTON_POSITIVE).setAllCaps(false);

dialog.getButton(AlertDialog.BUTTON_NEGATIVE).setAllCaps(false);

dialog.getButton(AlertDialog.BUTTON_NEGATIVE).setTextColor(getResources().
getColor(android.R.color.holo_red_dark));

    }

}
```



```
/**
 * Opens an image chooser dialog that allows the user to select a new
 * profile picture
 * from the gallery or take a new photo with the camera.
 */
private void openImageChooser() {
    Intent galleryIntent = new Intent(Intent.ACTION_GET_CONTENT);
    galleryIntent.setType("image/*");

    Intent cameraIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);

    Intent chooser = Intent.createChooser(galleryIntent, "Select
Picture");
    chooser.putExtra(Intent.EXTRA_INITIAL_INTENTS, new
Intent[]{cameraIntent});

    startActivityForResult(chooser, PICK_IMAGE_REQUEST);
}

/**
 * Handles the result of the image selection activity and updates the
 * profile picture.
 * This method is called when the user has selected an image from the
 * gallery or taken
 * a photo with the camera.
 *
 * @param requestCode The integer request code originally supplied to
startActivityForResult()
 * @param resultCode The integer result code returned by the child
activity
 * @param data An Intent that carries the result data
 */
@Override
public void onActivityResult(int requestCode, int resultCode,
@Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if (requestCode == PICK_IMAGE_REQUEST && resultCode ==
Activity.RESULT_OK) {
        Bitmap bitmap = null;

        if (data != null) {
            if (data.getData() != null) {
                // Selected from gallery
                Uri uri = data.getData();
                try {
                    bitmap =
MediaStore.Images.Media.getBitmap(getActivity().getContentResolver(),
uri);
                } catch (IOException e) {
                    e.printStackTrace();
                }
            } else if (data.getExtras() != null) {

```



```
// Taken from camera
bitmap = (Bitmap) data.getExtras().get("data");
}

if (bitmap != null) {
    profilePicture.setImageBitmap(bitmap); // Show image in
ImageView
    uploadImageToFirestore(bitmap); // Upload to Firestore
    MainActivity.profilePic.setImageBitmap(bitmap);
}
}

/**
 * Decodes a Base64 string into a Bitmap image.
 *
 * @param base64String The Base64 encoded string representation of the
image
 * @return A Bitmap created from the decoded Base64 string
 */
private Bitmap decodeBase64ToImage(String base64String) {
    byte[] decodedBytes = Base64.decode(base64String, Base64.DEFAULT);
    return BitmapFactory.decodeByteArray(decodedBytes, 0,
decodedBytes.length);
}

/**
 * Encodes a Bitmap image into a Base64 string.
 * The image is first scaled to a reasonable size to ensure efficient
storage.
 *
 * @param bitmap The Bitmap image to be encoded
 * @return A Base64 encoded string representation of the image
 */
private String encodeImageToBase64(Bitmap bitmap) {
    // Scale to a reasonable high-res size (e.g., 600x600 max)
    int maxSize = 600;
    int width = bitmap.getWidth();
    int height = bitmap.getHeight();

    float scale = Math.min((float) maxSize / width, (float) maxSize /
height);
    int scaledWidth = Math.round(scale * width);
    int scaledHeight = Math.round(scale * height);
    Bitmap scaledBitmap = Bitmap.createScaledBitmap(bitmap,
scaledWidth, scaledHeight, true);

    ByteArrayOutputStream byteArrayOutputStream = new
ByteArrayOutputStream();
    scaledBitmap.compress(Bitmap.CompressFormat.PNG, 100,
byteArrayOutputStream); // Use PNG for better quality
    byte[] byteArray = byteArrayOutputStream.toByteArray();
}
```



```
        return Base64.encodeToString(byteArray, Base64.DEFAULT);
    }

    /**
     * Uploads the user's profile picture to Firestore as a Base64 encoded
     * string.
     * This method converts the Bitmap to Base64, then saves it to the
     * user's document
     * in the Firestore database.
     *
     * @param bitmap The Bitmap image to be uploaded as the profile
     * picture
     */
    private void uploadImageToFirestore(Bitmap bitmap) {
        if (currentUser == null) return;

        // Convert the image to Base64
        String base64Image = encodeImageToBase64(bitmap);

        // Save it in Firestore under the user's document
        db.collection("users").document(currentUser.getId())
            .update("profilePicture", base64Image)
            .addOnSuccessListener(aVoid ->
                Toast.makeText(getActivity(), "Profile picture
updated!", Toast.LENGTH_SHORT).show()
            )
            .addOnFailureListener(e ->
                Toast.makeText(getActivity(), "Failed to upload
picture: " + e.getMessage(), Toast.LENGTH_SHORT).show()
            );
    }
}
```



AvailableRoomsFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ProgressBar;
import android.widget.Toast;

import com.example.projectarieleyalyotamyona.Adapters.RoomsAdapter;
import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;
import com.example.projectarieleyalyotamyona.Utilities.GetDay;
import com.example.projectarieleyalyotamyona.Utilities.GetHour;
import com.example.projectarieleyalyotamyona.Utilities.GetIsT;
import com.example.projectarieleyalyotamyona.R;

import com.example.projectarieleyalyotamyona.Utilities.RoomAdapterInterface;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * A Fragment that displays a list of available rooms.
 * <p>
 * This fragment retrieves room data from Firestore and displays it in a
 * RecyclerView.
 * It shows available rooms based on the current day and hour schedule.
 * Teachers can
 * click on a room to view its detailed daily schedule.
 * <p>
 * Implements RoomAdapterInterface to handle click events on room items.
 */
public class AvailableRoomsFragment extends Fragment implements
RoomAdapterInterface {

    private RecyclerView recyclerView;
    private RoomsAdapter adapter;
    private List<Rooms> roomList;
    private ProgressBar progressBar;
```



```
/**
 * Required empty public constructor.
 */
public AvailableRoomsFragment() {
    // Required empty public constructor
}

/**
 * Factory method to create a new instance of AvailableRoomsFragment.
 *
 * @return A new instance of AvailableRoomsFragment
 */
public static AvailableRoomsFragment newInstance() {
    return new AvailableRoomsFragment();
}

/**
 * Called to have the fragment instantiate its user interface view.
 * <p>
 * This method initializes the RecyclerView, sets up the adapter, and
prepares
 * the UI elements for displaying available rooms.
 *
 * @param inflater The LayoutInflater object that can be used to
inflate views
 * @param container If non-null, this is the parent view that the
fragment's UI should be attached to
 * @param savedInstanceState If non-null, this fragment is being re-
constructed from a previous saved state
 * @return The View for the fragment's UI
 */
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
    Bundle savedInstanceState) {

    // Inflate the layout for this fragment
    View view = inflater.inflate(R.layout.fragment_rooms_available,
container, false);

    // Initialize RecyclerView
    recyclerView = view.findViewById(R.id.recyclerViewRooms); // Make
sure this matches the RecyclerView ID in your XML
    recyclerView.setLayoutManager(new
LinearLayoutManager(getContext()));

    // Initialize Data
    roomList = new ArrayList<>();

    progressBar = view.findViewById(R.id.progressBarAR);
    progressBar.setVisibility(View.VISIBLE);

    // Set up Adapter
    adapter = new RoomsAdapter(getContext(), roomList, this);
    recyclerView.setAdapter(adapter);
}
```



```
        adapter.notifyDataSetChanged();  
        return view;  
    }  
  
    /**  
     * Populates the room list with data from Firestore.  
     * <p>  
     * This method queries the Firestore database for rooms and filters  
     them based on  
     * availability according to the current day and hour. It adds  
     available rooms to the  
     * roomList and updates the adapter.  
     * <p>  
     * For after-hours (hour > 9) or weekends (day > 4), all rooms are  
     displayed.  
     */  
    private void populateRoomList() {  
        FirebaseFirestore db = FirebaseFirestore.getInstance();  
        db.collection("Rooms").get().  
            .addOnSuccessListener(querySnapshot -> {  
                if (querySnapshot != null && !querySnapshot.isEmpty())  
{  
                    Log.d("PopulateRoomList", "Query success. Document  
size: " + querySnapshot.size());  
  
                    // Log hour and day values  
                    int hour = GetHour.getScheduleHour();  
                    int day = GetDay.getCurrentDay()-1;  
                    Log.d("PopulateRoomList", "Current Hour: " +  
hour);  
  
                    Log.d("PopulateRoomList", "Current Day: " + day);  
  
                    for (DocumentSnapshot document : querySnapshot) {  
                        String roomNum =  
document.getString("roomNum");  
                        Boolean isOccupied =  
document.getBoolean("isOccupied");  
                        Map<String, List<String>> schedule =  
(Map<String, List<String>>) document.get("weekSched");  
  
                        Log.d("PopulateRoomList", "RoomNum: " +  
roomNum);  
  
                        Log.d("PopulateRoomList", "isOccupied: " +  
isOccupied);  
  
                        Log.d("PopulateRoomList", "Schedule: " +  
schedule);  
  
                        // Ensure schedule and its contents are not  
null  
                        if (hour > 9 || day > 4) {  
                            roomList.add(new Rooms( roomNum,  
schedule));  
                        }  
                        else {
```



```
        if (schedule != null) {
            List<String> hourSchedule =
schedule.get(String.valueOf(hour));

            // Check if the hour schedule exists
            and is valid
            if (hourSchedule != null && day >= 0
&& day < hourSchedule.size()) {
                String status =

                // Log the retrieved status for
                debugging
                Log.d("PopulateRoomList", "Hour
Schedule Status: " + status);

                // Safely compare strings
                if (status != null &&
status.equals("Available") && hour <= 9 && day <= 4) {
                    roomList.add(new Rooms(
roomNum, schedule));
                }
            } else {
                Log.w("PopulateRoomList", "Hour
schedule or day index is invalid for room: " + roomNum);
            }
            if(hour==-1)
                roomList.add(new Rooms( roomNum,
schedule));
        }
    }

    Log.d("PopulateRoomList", "Total rooms fetched: "
+ roomList.size());

    // Refresh filtered list and adapter after
    fetching data
    adapter.notifyDataSetChanged();
} else {
    Log.d("PopulateRoomList", "No rooms found.");
}
progressBar.setVisibility(View.GONE);
})
.addOnFailureListener(e -> {
    Log.w("PopulateRoomList", "Error getting rooms data.",
e);
});
}

/**
 * Handles click events on room items in the RecyclerView.
 * <p>
```



```
* This method is called when a user clicks on a room in the list. It
checks if the user
* is a teacher and if it's not a weekend. If both conditions are met,
it navigates to the
* DailyScheduleFragment to display the daily schedule for the
selected room.
*
* @param position The position of the clicked item in the
RecyclerView
*/
@Override
public void onItemClick(int position) {
    GetIsT isT = new GetIsT();
    isT.checkIfTeacher(isTeacher -> {
        if (isTeacher) {
            if (GetDay.getCurrentDay() < 6) {
                // Get the selected room object from the list using
the clicked position
                Rooms selectedRoom = roomList.get(position);

                // Create an instance of the fragment we want to
navigate to
                DailyScheduleFragment dailyScheduleFragment = new
DailyScheduleFragment();

                // Create a Bundle to pass data to the new fragment
                Bundle bundle = new Bundle();
                bundle.putSerializable("Room", selectedRoom); //
Ensure Rooms class implements Serializable
                dailyScheduleFragment.setArguments(bundle); // Attach
the bundle to the fragment

                // Get the FragmentManager from the current context
(which should be an Activity)
                FragmentManager fragmentManager = ((AppCompatActivity)
getContext()).getSupportFragmentManager();

                // Begin a fragment transaction to replace the current
fragment with the new one
                FragmentTransaction transaction =
fragmentManager.beginTransaction();
                transaction.replace(R.id.fragment_container,
dailyScheduleFragment); // Replace with the target fragment
                transaction.addToBackStack(null); // Allows the user
to navigate back to the previous fragment
                transaction.commit(); // Apply the transaction
            }
            else
            {
                Toast.makeText(getContext(), "Its the weekend || no
schedule", Toast.LENGTH_SHORT).show();
            }
        }
    });
}
```



```
}

/**
 * Called when the fragment is visible to the user and actively
 * running.
 * <p>
 * This method refreshes the room list by clearing the existing data
 * and
 * repopulating it from Firestore each time the fragment resumes,
 * ensuring
 * the displayed information is up-to-date.
 */
@Override
public void onResume() {
    super.onResume();
    roomList.clear();
    populateRoomList();
}

}
```



DailyScheduleFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.os.Bundle;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.NumberPicker;
import android.widget.TextView;
import android.widget.Toast;

import com.example.projectarieleyalyotamyona.Adapters.HourScheduleAdapter;
import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;
import com.example.projectarieleyalyotamyona.Data_Modules.User;
import com.example.projectarieleyalyotamyona.Utilities.GetDay;
import com.example.projectarieleyalyotamyona.R;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * A fragment that displays and manages the daily schedule for a selected
 * room.
 * <p>
 * This fragment allows users to view the hourly availability of a
 * specific room for the current day
 * and book time slots. It displays the room information, current day, and
 * a list of hourly schedules.
 * Users can select a range of hours using number pickers and book the
 * room if those hours are available.
 * </p>
 * <p>
 * The fragment interacts with Firebase Firestore to update room booking
 * status and user booking records.
 * </p>
 */
public class DailyScheduleFragment extends Fragment {

    /** Adapter for displaying hourly schedule data in the RecyclerView */
    private HourScheduleAdapter adapter;

    /** RecyclerView that displays the hourly schedule */
    private RecyclerView recyclerView;
```



```
/** Current day name (e.g., "Monday") */
private String day;

/** Current user's ID from Firebase Authentication */
private String id;

/** Firebase Firestore instance for database operations */
private FirebaseFirestore db;

/** NumberPicker for selecting start hour of booking */
private NumberPicker hourFromPicker;

/** NumberPicker for selecting end hour of booking */
private NumberPicker hourToPicker;

/** The room object selected for viewing/booking */
private Rooms selectedRoom;

/** Button for confirming room booking */
private Button bookRoomBT;

/** Current Firebase user object */
private FirebaseAuth user;

/** Firebase Authentication instance */
private FirebaseAuth auth;

/**
 * Default constructor for the fragment.
 * Required empty public constructor.
 */
public DailyScheduleFragment() {
    // Required empty public constructor
}

/**
 * Inflates the fragment layout and initializes the RecyclerView and
 * retrieves the selected room.
 *
 * @param inflater The LayoutInflater object that can be used to
 * inflate views
 * @param container The parent view that the fragment's UI should be
 * attached to
 * @param savedInstanceState If non-null, this fragment is being re-
 * constructed from a previous saved state
 * @return The View for the fragment's UI
 */
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
    Bundle savedInstanceState) {
    View view = inflater.inflate(R.layout.fragment_daily_scedual,
        container, false);
```



```
recyclerView = view.findViewById(R.id.recycler_view_schedule);
recyclerView.setLayoutManager(new
LinearLayoutManager(this.getContext()));

// Get the selected room passed via fragment arguments
selectedRoom = (Rooms) getArguments().getSerializable("Room");

return view;
}

/**
 * Initializes UI components, sets up hour pickers, and configures the
 * booking button after the view is created.
 * <p>
 * This method handles the setup of text views for room and day
 * information,
 * configures number pickers for hour selection, initializes the
 * recycler view with schedule data,
 * and sets up the booking button with its click listener to handle
 * room booking requests.
 * </p>
 *
 * @param view The View returned by onCreateView
 * @param savedInstanceState If non-null, this fragment is being re-
 * constructed from a previous saved state
 */
@Override
public void onViewCreated(View view, Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);

    TextView roomNameTextView = view.findViewById(R.id.roomNum);
    TextView dayTextView = view.findViewById(R.id.day);

    // Set room and day labels
    if (selectedRoom != null && roomNameTextView != null) {
        roomNameTextView.setText("Room: " +
selectedRoom.getRoomNum());
    }

    if (dayTextView != null) {
        day = GetDay.getCurrentDayName();
        dayTextView.setText("Day: " + day);
    }

    // Initialize hour pickers (1 to 9)
    hourFromPicker = view.findViewById(R.id.hourFromPicker);
    hourToPicker = view.findViewById(R.id.hourToPicker);

    hourFromPicker.setMinValue(1);
    hourFromPicker.setMaxValue(9);
    hourFromPicker.setWrapSelectorWheel(true);

    hourToPicker.setMinValue(1);
    hourToPicker.setMaxValue(9);
}
```



```
hourToPicker.setWrapSelectorWheel(true);

hourFromPicker.setValue(1);
hourToPicker.setValue(2);

// Populate RecyclerView
setUpRecyclerView();

// Setup booking button
bookRoomBT = view.findViewById(R.id.bookRoomBT);
Map<String, List<String>> schedule = selectedRoom.getWeekSched();
int dayIndex = GetDay.getCurrentDay() - 1;

auth = FirebaseAuth.getInstance();
user = auth.getCurrentUser();
id = user.getId();

// Handle booking logic
bookRoomBT.setOnClickListener(view1 -> {
    int selectedFromHour = hourFromPicker.getValue();
    int selectedToHour = hourToPicker.getValue();
    List<String> hoursSelected = new ArrayList<>();
    boolean allAvailable = true;

    if (selectedFromHour > selectedToHour) {
        Toast.makeText(getContext(), "From hour is bigger than to
hour", Toast.LENGTH_SHORT).show();
        return;
    }

    // Check availability of all selected hours
    for (int i = selectedFromHour; i <= selectedToHour; i++) {
        String hourKey = String.valueOf(i);
        if
(!schedule.get(hourKey).get(dayIndex).equals("Available")) {
            allAvailable = false;
            break;
        }
    }

    // If all hours available, book them directly
    if (allAvailable) {
        for (int i = selectedFromHour; i <= selectedToHour; i++) {
            String hourKey = String.valueOf(i);
            schedule.get(hourKey).set(dayIndex, "Occupied");
            hoursSelected.add(hourKey);
        }
        performBooking(hoursSelected, schedule);
    } else {
        Toast.makeText(getContext(), "Some hours are already
occupied. Please choose a different time range.",
Toast.LENGTH_SHORT).show();
    }
});
```



```
// Redundant display of room number
if (selectedRoom != null) {
    String roomNumber = selectedRoom.getRoomNum();
    TextView roomNumberTextView = view.findViewById(R.id.roomNum);
    if (roomNumberTextView != null) {
        roomNumberTextView.setText("Room number: " + roomNumber);
    }
    SetUpRoom(selectedRoom);
}

/**
 * Sets up the RecyclerView with the schedule data for the current
 * day.
 * <p>
 * This method extracts the schedule information for the current day
 * from the selected room
 * and populates the RecyclerView with this data using the
 * HourScheduleAdapter.
 * </p>
 */
private void setUpRecyclerView() {
    List<String> scheduleList = new ArrayList<>();
    int dayIndex = GetDay.getCurrentDay() - 1;

    if (dayIndex < 5) {
        for (int i = 1; i <= selectedRoom.getWeekSched().size(); i++)
        {
            String status =
selectedRoom.getWeekSched().get(String.valueOf(i)).get(dayIndex);
            scheduleList.add(status);
        }

        if (getContext() != null) {
            adapter = new HourScheduleAdapter(getContext(), scheduleList);
            recyclerView.setAdapter(adapter);
        }
    }

    /**
     * Updates the RecyclerView with the schedule data for a specific
     * room.
     * <p>
     * This method is similar to setUpRecyclerView() but takes a room
     * parameter,
     * allowing it to be called with different room objects to update the
     * display.
     * </p>
     *
     * @param room The room object containing the schedule data to display
     */
    public void SetUpRoom(Rooms room) {
```



```
List<String> scheduleList = new ArrayList<>();
int dayIndex = GetDay.getCurrentDay() - 1;

if (dayIndex < 5) {
    for (int i = 1; i <= room.getWeekSched().size(); i++) {
        String status =
room.getWeekSched().get(String.valueOf(i)).get(dayIndex);
        scheduleList.add(status);
    }
}

if (getContext() != null) {
    adapter = new HourScheduleAdapter(getContext(), scheduleList);
    recyclerView.setAdapter(adapter);
}

}

/**
 * Performs the room booking operation and updates both the room and
 * user data in Firestore.
 * <p>
 * This method updates the room's schedule in Firestore to mark the
 * selected hours as occupied,
 * and also adds the booking information to the user's list of booked
 * rooms.
 * </p>
 *
 * @param hours List of hour strings that were booked (e.g., ["1",
"2", "3"])
 * @param updatedSchedule The updated schedule map with the booked
hours marked as occupied
 */
private void performBooking(List<String> hours, Map<String,
List<String>> updatedSchedule) {
    String message = "Day: " + day + " || Hours: " + String.join("-",
hours);

    // Update local object
    selectedRoom.setWeekSched(updatedSchedule);

    // Update Firestore with new room data
    db = FirebaseFirestore.getInstance();
    db.collection("Rooms")
        .document(selectedRoom.getRoomNum())
        .set(selectedRoom)
        .addOnSuccessListener(unused ->
            Toast.makeText(getContext(), message,
Toast.LENGTH_LONG).show()
        )
        .addOnFailureListener(e ->
            Toast.makeText(getContext(), "Failed to update
room: " + e.getMessage(), Toast.LENGTH_SHORT).show()
        );
}
```



```
        SetUpRoom(selectedRoom);

        // Create booking object and update user's bookedRooms
        User.Booking booking = new User.Booking(selectedRoom.getRoomNum(),
day, hours);

        if (user != null) {
            db.collection("users")
                .document(id)
                .update("bookedRooms", FieldValue.arrayUnion(booking))
                .addOnSuccessListener(aVoid ->
                    Toast.makeText(getContext(), "Booking appended
successfully!", Toast.LENGTH_SHORT).show()
                )
                .addOnFailureListener(e ->
                    Toast.makeText(getContext(), "Error appending
booking: " + e.getMessage(), Toast.LENGTH_SHORT).show()
                );
        }
    }
}
```



HomeFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;

import com.example.projectarieleyalyotamyona.R;

/**
 * A container fragment that serves as the main home screen of the
 * application.
 * <p>
 * This fragment acts as a host for other fragments, specifically
 * displaying available rooms
 * and school updates by loading them into separate container views. It
 * serves as the primary
 * navigation hub for the application, showing important information at a
 * glance.
 * </p>
 */
public class HomeFragment extends Fragment {

    /**
     * Inflates the fragment layout and loads child fragments into their
     * respective containers.
     * <p>
     * This method initializes the HomeFragment's view hierarchy and
     * immediately loads
     * the AvailableRoomsFragment and SchoolUpdatesFragment into their
     * designated containers.
     * </p>
     *
     * @param inflater The LayoutInflater object that can be used to
     * inflate views
     * @param container The parent view that the fragment's UI should be
     * attached to
     * @param savedInstanceState If non-null, this fragment is being re-
     * constructed from a previous saved state
     * @return The View for the fragment's UI
     */
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        View rootView = inflater.inflate(R.layout.fragment_home,
            container, false);
        loadFragment();
    }
}
```



```
        return rootView;
    }

    /**
     * Loads child fragments into their respective container views.
     * <p>
     * This method uses the child FragmentManager to create a transaction
     that loads
     * the AvailableRoomsFragment into the rooms container and the
     SchoolUpdatesFragment
     * into the notifications container. These fragments provide the core
     functionality
     * displayed on the home screen.
     * </p>
     */
    private void loadFragment() {
        // Get the child FragmentManager to manage fragments inside
        HomeFragment
        FragmentManager fragmentManager = getChildFragmentManager();

        // Begin a transaction
        FragmentTransaction fragmentTransaction =
        fragmentManager.beginTransaction();

        // Replace the current fragment in the FrameLayout (id:
        fragment_container)
        fragmentTransaction.replace(R.id.fragment_container_Rooms, new
        AvailableRoomsFragment());
        fragmentTransaction.replace(R.id.fragment_container_noti, new
        SchoolUpdatesFragment());

        // Commit the transaction
        fragmentTransaction.commit();
    }
}
```



MainRoomsFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.content.Intent;
import android.os.Bundle;
import androidx.fragment.app.Fragment;

import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.LinearLayout;
import android.widget.SearchView;
import android.widget.Toast;

import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.Adapters.RoomsAdapter;
import com.example.projectarieleyalyotamyona.AdvSearchActivity;
import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;
import com.example.projectarieleyalyotamyona.Utilities.GetDay;
import com.example.projectarieleyalyotamyona.Utilities.GetIsT;
import com.example.projectarieleyalyotamyona.InfoAndBookRoomActivity;
import com.example.projectarieleyalyotamyona.MyRoomsActivity;
import com.example.projectarieleyalyotamyona.R;

import com.example.projectarieleyalyotamyona.Utilities.RoomAdapterInterface;
import com.google.android.gms.tasks.Task;
import com.google.android.gms.tasks.Tasks;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * A fragment that displays a list of available rooms and provides search
 * and navigation functionality.
 * <p>
 * This fragment shows a list of rooms fetched from Firebase Firestore and
 * allows users to:
 * <ul>
 * <li>Search for rooms by name</li>
 * <li>Navigate to My Rooms screen</li>
 * <li>Navigate to Advanced Search screen</li>
 * <li>Select a room to view details and book (for teachers only)</li>
 * </ul>
 * </p>
 */
```



```
* <p>
* It implements RoomAdapterInterface to handle clicks on room items in
the RecyclerView.
* The fragment also implements role-based UI visibility, showing
additional options for teachers.
* </p>
*/
public class MainRoomsFragment extends Fragment implements
RoomAdapterInterface {

    /** RecyclerView for displaying the list of rooms */
    private RecyclerView recyclerView;

    /** Adapter for binding room data to the RecyclerView */
    private RoomsAdapter adapter;

    /** Complete list of rooms fetched from Firestore */
    private List<Rooms> roomList;

    /** Filtered list of rooms based on search query */
    private List<Rooms> filteredList;

    /** SearchView for filtering rooms by name */
    private SearchView searchView;

    /** Button to navigate to the MyRoomsActivity */
    private Button myRoomsBt;

    /** Button to navigate to the AdvSearchActivity */
    private Button AdvSearchBt;

    /** Container for buttons that are only visible to teachers */
    private LinearLayout linearLayoutBt;

    /**
     * Required empty public constructor.
     */
    public MainRoomsFragment() {
        // Required empty public constructor
    }

    /**
     * Factory method to create a new instance of this fragment.
     *
     * @return A new instance of MainRoomsFragment
     */
    public static MainRoomsFragment newInstance() {
        return new MainRoomsFragment();
    }

    /**
     * Inflates the fragment layout and initializes UI components.
     * <p>
     * This method sets up the RecyclerView with its adapter, initializes
```



```
the search functionality,
    * configures navigation buttons, and checks user permissions to
    show/hide teacher-specific options.
    * </p>
    *
    * @param inflater The LayoutInflater object that can be used to
    inflate views
    * @param container The parent view that the fragment's UI should be
    attached to
    * @param savedInstanceState If non-null, this fragment is being re-
    constructed from a previous saved state
    * @return The View for the fragment's UI
    */
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
    Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        View view = inflater.inflate(R.layout.fragment_main_rooms,
    container, false);
        // Get a reference to the ImageButton

        // Initialize RecyclerView
        recyclerView = view.findViewById(R.id.recycler_view);
        recyclerView.setLayoutManager(new
    LinearLayoutManager(getContext()));

        // Initialize and populate the room list
        roomList = new ArrayList<>();

        // Initialize filtered list for search functionality
        filteredList = new ArrayList<>(roomList);
        // Set up the adapter
        adapter = new RoomsAdapter(getContext(), filteredList, this);
        recyclerView.setAdapter(adapter);

        myRoomsBt = view.findViewById(R.id.myRoomsBt);
        myRoomsBt.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(getActivity(),
    MyRoomsActivity.class);
                startActivity(intent);
            }
        });
        AdvSearchBt= view.findViewById(R.id.AdvSearch);

        AdvSearchBt.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(getActivity(),
    AdvSearchActivity.class);
                startActivity(intent);
            }
        });
    }
};
```



```
// Initialize SearchView
searchView = view.findViewById(R.id.search_view);
setupSearchView();

linearLayoutBt = view.findViewById(R.id.linearLayoutBt);

GetIsT getIsT = new GetIsT();
getIsT.checkIfTeacher(new GetIsT.FirestoreCallback() {
    @Override
    public void onResult(boolean isTeacher) {
        if(isTeacher)
        {
            linearLayoutBt.setVisibility(View.VISIBLE);
        }
    }
});

return view;
}

/**
 * Fetches room data from Firestore and populates the room list.
 * <p>
 * This method queries the "Rooms" collection in Firestore, creates
Room objects from
 * the retrieved data, and updates the adapter with the new data.
 * </p>
 */
private void populateRoomList() {
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("Rooms").get()
        .addOnSuccessListener(querySnapshot -> {
            if (querySnapshot != null && !querySnapshot.isEmpty())
            {
                Log.d("PopulateRoomList", "Query success. Document
size: " + querySnapshot.size());
                for (DocumentSnapshot document : querySnapshot) {
                    String roomNum =
document.getString("roomNum");
                    Boolean isOccupied =
document.getBoolean("isOccupied");
                    Map<String, List<String>> schedule =
(Map<String, List<String>>) document.get("weekSched");

                    Log.d("PopulateRoomList", "RoomNum: " +
roomNum);
                    Log.d("PopulateRoomList", "isOccupied: " +
isOccupied);
                    Log.d("PopulateRoomList", "Schedule: " +
schedule);

                    roomList.add(new Rooms(roomNum, schedule));
                }
            }
        });
}
```



```
Log.d("PopulateRoomList", "Total rooms fetched: "
+ roomList.size());

// Refresh filtered list and adapter after
fetching data

filteredList.clear();
filteredList.addAll(roomList);
adapter.notifyDataSetChanged(); // Refresh the
RecyclerView

    } else {
        Log.d("PopulateRoomList", "No rooms found.");
    }
})
.addOnFailureListener(e -> {
    Log.w("PopulateRoomList", "Error getting rooms data.",
e);
});
});

/**
 * Configures the SearchView to filter rooms by name.
 * <p>
 * Sets up listeners for text changes and query submissions to filter
the room list
 * in real-time as the user types a search query.
 * </p>
 */
private void setupSearchView() {
    searchView.setOnQueryTextListener(new
SearchView.OnQueryTextListener() {
        @Override
        public boolean onQueryTextSubmit(String query) {
            filterRooms(query);
            return false;
        }

        @Override
        public boolean onQueryTextChange(String newText) {
            filterRooms(newText);
            return false;
        }
    });
}

/**
 * Determines if the current user is a teacher by checking Firestore.
 * <p>
 * This method retrieves the current user's document from Firestore
and checks
 * the "isT" field to determine if the user has teacher privileges.
 * </p>
 *
 * @return true if the current user is a teacher, false otherwise
 */
```



```
public boolean Show() {
    // Retrieve the current authenticated user from FirebaseAuth
    FirebaseUser currentUser =
    FirebaseAuth.getInstance().getCurrentUser();

    // Check if a user is logged in
    if (currentUser != null) {
        // Get an instance of FirebaseFirestore
        FirebaseFirestore db = FirebaseFirestore.getInstance();

        // Retrieve the user's document from the 'users' collection
        using their UID
        Task<DocumentSnapshot> task =
        db.collection("users").document(currentUser.getId()).get();

        try {
            // Wait for the Firestore task to complete and get the
            document snapshot
            DocumentSnapshot documentSnapshot = Tasks.await(task);

            // Check if the document exists in Firestore
            if (documentSnapshot.exists()) {
                // Get the value of the 'isT' field (is user a
                teacher?)
                Boolean isTeacher =
                documentSnapshot.getBoolean("isT");

                // If 'isT' exists and is true (the user is a
                teacher), show the button
                if (isTeacher != null && isTeacher) {
                    return true; // User is a teacher
                }
            } else {
                // If the document doesn't exist (no data found for
                the user), show an error message
                Toast.makeText(getApplicationContext(), "User data not found.",
                Toast.LENGTH_SHORT).show();
            }
        } catch (Exception e) {
            // If there was an error while fetching user data, show
            the error message
            Toast.makeText(getApplicationContext(), "Error fetching user data: "
            + e.getMessage(), Toast.LENGTH_SHORT).show();
        }
    } else {
        // If no user is logged in, show a message indicating that no
        user is logged in
        Toast.makeText(getApplicationContext(), "No user logged in.",
        Toast.LENGTH_SHORT).show();
    }

    // Return false if the user is not a teacher or if there was an
    issue
    return false;
}
```



```
/**
 * Filters the room list based on a search query.
 * <p>
 * This method updates the filteredList with rooms whose names contain
the search query
 * (case-insensitive) and notifies the adapter to refresh the UI.
 * </p>
 *
 * @param query The search text to filter rooms by
 */
private void filterRooms(String query) {
    filteredList.clear();
    for (Rooms room : roomList) {
        if
(room.getRoomNum().toLowerCase().contains(query.toLowerCase())) {
            filteredList.add(room);
        }
    }

    adapter.notifyDataSetChanged(); // Notify the adapter of data
changes
}

/**
 * Handles clicks on room items in the RecyclerView.
 * <p>
 * This method is called when a user clicks on a room in the list. If
the user is a teacher
 * and it's not a weekend, it opens the InfoAndBookRoomActivity for
the selected room.
 * </p>
 *
 * @param position The position of the clicked item in the
RecyclerView
 */
@Override
public void onItemClick(int position) {
    GetIsT isT = new GetIsT();
    isT.checkIfTeacher(isTeacher -> {
        if (isTeacher) {
            if (GetDay.getCurrentDay() < 6) {
                // Get the selected room from the list
                Rooms selectedRoom = roomList.get(position);

                // Create an intent and pass the room data
                Intent intent = new Intent(getContext(),
InfoAndBookRoomActivity.class);

                // Pass room details (you can pass specific fields or
the entire object if it's Parcelable)
                intent.putExtra("Room:", selectedRoom);

                // Start the activity
            }
        }
    });
}
```



```
        getContext().startActivity(intent);
    }
    else
    {
        Toast.makeText(getContext(), "Its the weekend || no
schedule", Toast.LENGTH_SHORT).show();
    }
}
});
}

/**
 * Called when the fragment becomes visible to the user.
 * <p>
 * This method clears and reloads the room list from Firestore to
ensure data is fresh
 * each time the fragment is displayed.
 * </p>
 */
public void onResume() {
    roomList.clear();
    filteredList.clear();
    super.onResume();
    populateRoomList();
}
}
```



ManageRoomsFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.AdapterView;
import android.widget.AutoCompleteTextView;
import android.widget.ImageButton;
import android.widget.TextView;
import android.widget.Toast;

import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.Adapters.ScheduleAdapter;
import com.example.projectarieleyalyotamyona.AddRoomActivity;
import com.example.projectarieleyalyotamyona.R;
import com.google.android.material.button.MaterialButton;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.List;
import java.util.Map;

/**
 * A Fragment that manages room schedules within the application.
 * <p>
 * This fragment provides functionality to:
 * <ul>
 * <li>View room schedules from Firestore database</li>
 * <li>Select rooms from a dropdown menu</li>
 * <li>Navigate to the room addition screen</li>
 * <li>Update room schedules in the database</li>
 * </ul>
 * The fragment uses RecyclerView to display schedule data and integrates
 * with
 * Firebase Firestore for data persistence.
 */
public class ManageRoomsFragment extends Fragment {

    /** Default selected room number */
    private int selectedNumber = 1;

    /** Button to navigate to the add room screen */
    private ImageButton addRoom;

    /** Button to commit schedule changes to Firestore */
    private MaterialButton commitBt;
```



```
/** RecyclerView for displaying room schedule information */
private RecyclerView roomRv;

/** Adapter for binding schedule data to the RecyclerView */
private ScheduleAdapter scheduleAdapter;

/** Data structure holding the schedule information for a selected
room */
private Map<String, List<String>> schedule;

/** Firebase Firestore database instance */
private FirebaseFirestore db;

/**
 * Required empty public constructor for Fragment instantiation.
 */
public ManageRoomsFragment() {
    // Required empty public constructor
}

/**
 * Inflates the fragment layout and initializes UI components.
 * <p>
 * This method sets up:
 * <ul>
 * <li>Room number selection via AutoCompleteTextView</li>
 * <li>Click listeners for buttons</li>
 * <li>Initial room schedule loading</li>
 * </ul>
 *
 * @param inflater The LayoutInflater object to inflate views
 * @param container The parent view that the fragment UI should be
attached to
 * @param savedInstanceState Previous state of the fragment, if
available
 * @return The View for the fragment's UI
 */
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
    Bundle savedInstanceState) {

    // Inflate the layout for this fragment
    View view = inflater.inflate(R.layout.fragment_manage_rooms,
container, false);

    // Find AutoCompleteTextView and TextView
    AutoCompleteTextView roomAutoComplete =
view.findViewById(R.id.roomAutoComplete);
    TextView numberDisplay = view.findViewById(R.id.numberDisplay);
    roomRv = view.findViewById(R.id.roomRV);
    db = FirebaseFirestore.getInstance();
    commitBt = view.findViewById(R.id.commitBt);

    // Create an array of room numbers (can be dynamically generated
```



```
or retrieved from a database)
    String[] roomNumbers = new String[100];
    for (int i = 0; i < 100; i++) {
        roomNumbers[i] = String.valueOf(i + 1); // Room numbers from 1
to 100
    }

    // Create an ArrayAdapter with the room numbers
    ArrayAdapter<String> adapter = new ArrayAdapter<>(getContext(),
android.R.layout.simple_dropdown_item_1line, roomNumbers);

    // Set the adapter to the AutoCompleteTextView
    roomAutoComplete.setAdapter(adapter);

    // Set the threshold to 1 so that suggestions show after typing
the first character
    roomAutoComplete.setThreshold(1);
    getRoomSced(1);

    // Set the default value
    roomAutoComplete.setText(String.valueOf(selectedNumber));

    // Set listener to detect value changes
    roomAutoComplete.setOnItemClickListener((parent, view1, position,
id) -> {
        selectedNumber = Integer.parseInt((String)
parent.getItemAtPosition(position));
        numberDisplay.setText("Room number: " + selectedNumber);
        getRoomSced(selectedNumber);
    });

    addRoom = view.findViewById(R.id.add_room_button);
    addRoom.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            // Use getActivity() to obtain the context if this is a
fragment
            Intent intent = new Intent(getActivity(),
AddRoomActivity.class);
            startActivity(intent);
        }
    });

    commitBt.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            updateRoomSchedule(selectedNumber);
        }
    });

    return view;
}

/**
```



```
* Retrieves the schedule for a specified room from Firestore.
* <p>
* This method:
* <ul>
*     <li>Queries the Firestore database for the specified room
number</li>
*     <li>If the room exists, loads its schedule data into the
RecyclerView</li>
*     <li>If the room doesn't exist, displays a toast message and
hides relevant UI elements</li>
* </ul>
*
* @param roomNum The room number to retrieve the schedule for
*/
public void getRoomSced(int roomNum) {
    db.collection("Rooms")
        .document(String.valueOf(roomNum)).get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                schedule = (Map<String, List<String>>)
documentSnapshot.get("schedule");
                scheduleAdapter = new
ScheduleAdapter(this.getContext(), schedule, roomRv);
                roomRv.setLayoutManager(new
LinearLayoutManager(this.getContext()));
                roomRv.setAdapter(scheduleAdapter);
                roomRv.setVisibility(View.VISIBLE);
                commitBt.setVisibility(View.VISIBLE);
            }
            else {
                Toast.makeText(getContext(), "Room does not
exist", Toast.LENGTH_LONG).show();
                roomRv.setVisibility(View.GONE);
                commitBt.setVisibility(View.GONE);
            }
        });
}

/**
* Updates the schedule for a specified room in Firestore.
* <p>
* This method:
* <ul>
*     <li>Retrieves the current schedule data from the adapter</li>
*     <li>Updates the document in Firestore with the new schedule
data</li>
*     <li>Shows appropriate toast messages for success or
failure</li>
* </ul>
*
* @param roomNum The room number to update the schedule for
*/
public void updateRoomSchedule(int roomNum) {
    Map<String, List<String>> updatedSchedule =
```



```
scheduleAdapter.getScheduleData();

    db.collection("Rooms")
        .document(String.valueOf(roomNum))
        .update("schedule", updatedSchedule)
        .addOnSuccessListener(aVoid -> {
            Toast.makeText(getContext(), "Schedule updated",
Toast.LENGTH_LONG).show();
        })
        .addOnFailureListener(e -> {
            Log.e("Firestore", "Failed to update schedule for room
" + roomNum, e);
            Toast.makeText(getContext(), "Schedule failed to
update", Toast.LENGTH_LONG).show();
        });
    }
}
```



ManageSchoolNotificationsFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.app.AlertDialog;
import android.content.DialogInterface;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;

import com.example.projectarieleyalyotamyona.R;
import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.List;

/**
 * A Fragment that manages school notifications within the application.
 * <p>
 * This fragment provides functionality to:
 * <ul>
 * <li>View existing school notifications retrieved from
 * Firestore</li>
 * <li>Add new notifications to the system</li>
 * <li>Delete notifications after confirmation</li>
 * </ul>
 * The fragment uses a ListView to display notifications and integrates
 * with
 * Firebase Firestore for data persistence.
 */
public class ManageSchoolNotificationsFragment extends Fragment {

    /** ListView for displaying all school notifications */
    private ListView listView;

    /** Button to add new notifications */
    private ImageButton addNotiBt;

    /** Adapter for binding notification messages to the ListView */
```



```
private ArrayAdapter<String> adapter;

/** List containing all notification messages */
private List<String> messages;

/** Firebase Firestore database instance */
private FirebaseFirestore db;

/**
 * Required empty public constructor for Fragment instantiation.
 */
public ManageSchoolNotificationsFragment() {
    // Required empty public constructor
}

/**
 * Inflates the fragment layout and initializes UI components.
 * <p>
 * This method sets up:
 * <ul>
 * <li>Firestore connection to retrieve notification data</li>
 * <li>ListView with adapter for displaying notifications</li>
 * <li>Click listeners for item removal and adding new
notifications</li>
 * </ul>
 *
 * @param inflater The LayoutInflater object to inflate views
 * @param container The parent view that the fragment UI should be
attached to
 * @param savedInstanceState Previous state of the fragment, if
available
 * @return The View for the fragment's UI
 */
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
    Bundle savedInstanceState) {
    View view =
inflater.inflate(R.layout.fragment_manage_school_notifications, container,
false);

    messages = new ArrayList<>();
    db = FirebaseFirestore.getInstance();

    // Find the ListView
    listView = view.findViewById(R.id.notifications_list);
    addNotiBt = view.findViewById(R.id.add_noti_button);

    // Load notification messages from Firestore
    db.collection("SharedData").document("Yoana").get()
        .addOnSuccessListener(new
OnSuccessListener<DocumentSnapshot>() {
        @Override
        public void onSuccess(DocumentSnapshot
documentSnapshot) {
```



```
        if(documentSnapshot.get("NotiList")!=null) {
            messages = (List<String>)
documentSnapshot.get("NotiList");
        }

        // Create and set adapter
        adapter = new ArrayAdapter<>(
            requireContext(),
            android.R.layout.simple_list_item_1,
            messages);
        listView.setAdapter(adapter);
    }
}

.addOnFailureListener(new OnFailureListener() {
    @Override
    public void onFailure(@NonNull Exception e) {
        Toast.makeText(getContext(),"Failed to load
messages",Toast.LENGTH_LONG).show();
    }
});

// Set up click listener for notification items - for deletion
listView.setOnItemClickListener(new
AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View v, int
position, long id) {
        String selectedMessage = messages.get(position);

        // Show the message inside the dialog
        TextView messageTv = new TextView(getContext());
        messageTv.setText(selectedMessage);
        messageTv.setPadding(40, 30, 40, 30);
        messageTv.setTextSize(16);

        new AlertDialog.Builder(getContext())
            .setTitle("Are you sure you want to delete the
notification?")
            .setView(messageTv)
            .setPositiveButton("Yes", new
DialogInterface.OnClickListener() {
                @Override
                public void onClick(DialogInterface
dialogInterface, int i) {
                    messages.remove(position);

                    db.collection("SharedData").document("SchoolNoti")
                        .update("NotiList", messages)
                        .addOnSuccessListener(new
OnSuccessListener<Void>() {
                            @Override
                            public void onSuccess(Void
unused) {
```



```
adapter.notifyDataSetChanged();

Toast.makeText(getContext(), "Notification deleted",
Toast.LENGTH_SHORT).show();

    }

    });

    }

    })
    .setNegativeButton("Cancel", null)
    .show();

}

});

// Set up click listener for adding new notifications
addNotiBt.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        // Create an EditText
        final EditText input = new EditText(getContext());
        input.setHint("Enter notification");

        // Build AlertDialog
        new AlertDialog.Builder(getContext())
            .setTitle("New Notification")
            .setView(input)
            .setPositiveButton("Add", (dialog, which) -> {
                String message =
input.getText().toString().trim();
                if (!message.isEmpty()) {
                    messages.add(message);

db.collection("SharedData").document("Yoana")
                    .update("NotiList", messages)
                    .addOnSuccessListener(new
OnSuccessListener<Void>() {

                        @Override
                        public void onSuccess(Void
unused) {

adapter.notifyDataSetChanged();

Toast.makeText(getContext(), "Notification added: " + message,
Toast.LENGTH_SHORT).show();

                }

            } else {
                Toast.makeText(getContext(), "Message
can't be empty", Toast.LENGTH_SHORT).show();
            }
        })
        .setNegativeButton("Cancel", null)
        .show();

    }

});
```



```
    return view;  
}
```



MapFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.annotation.SuppressLint;
import android.graphics.Matrix;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.MotionEvent;
import android.view.ScaleGestureDetector;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

import com.example.projectarieleyalyotamyona.R;

/**
 * A Fragment that displays an interactive map with pan and zoom
 * functionality.
 * <p>
 * This fragment provides an image-based map with the following features:
 * <ul>
 * <li>Pan navigation by touch gestures</li>
 * <li>Pinch-to-zoom functionality with minimum and maximum scale
 * limits</li>
 * <li>Cooldown period to prevent unintended pan operations
 * immediately after scaling</li>
 * </ul>
 * The map uses a Matrix transformation to handle both translation and
 * scaling operations
 * while maintaining touch gesture performance.
 */
public class MapFragment extends Fragment {

    /** ImageView that displays the map */
    private ImageView imageView;

    /** Matrix used for transformations (pan and zoom) */
    private Matrix matrix = new Matrix();

    /** Current scale factor of the map */
    private float scale = 1f;

    /** Last touch X coordinate for calculating pan deltas */
    private float lastX = 0f;

    /** Last touch Y coordinate for calculating pan deltas */
    private float lastY = 0f;
```



```
/** Detector for pinch-to-zoom gestures */
private ScaleGestureDetector scaleGestureDetector;

/** Timestamp of the last scale operation for implementing cooldown */
private long lastScaleTime = 0;

/** Duration in milliseconds to prevent panning immediately after
scaling */
private static final long SCALE_COOLDOWN_MS = 50;

/**
 * Required empty public constructor for Fragment instantiation.
 */
public MapFragment() {
    // Required empty public constructor
}

/**
 * Inflates the fragment layout and initializes the interactive map.
 * <p>
 * This method sets up:
 * <ul>
 * <li>The ImageView with the map resource</li>
 * <li>Touch event handling for pan operations</li>
 * <li>Scale gesture detector for zoom operations</li>
 * </ul>
 *
 * @param inflater The LayoutInflater object to inflate views
 * @param container The parent view that the fragment UI should be
attached to
 * @param savedInstanceState Previous state of the fragment, if
available
 * @return The View for the fragment's UI
 */
@Override
@SuppressLint("ClickableViewAccessibility")
public View onCreateView(@NonNull LayoutInflater inflater,
    @Nullable ViewGroup container,
    @Nullable Bundle savedInstanceState) {
    View view = inflater.inflate(R.layout.fragment_map, container,
false);

    imageView = view.findViewById(R.id.imageView); // Make sure your
XML has this ID
    imageView.setScaleType(ImageView.ScaleType.MATRIX);
    imageView.setImageResource(R.drawable.map_test_1); // Replace with
your actual map/image

    scaleGestureDetector = new ScaleGestureDetector(getContext(), new
ScaleListener());

    imageView.setOnTouchListener((v, event) -> {
        scaleGestureDetector.onTouchEvent(event);
    });
}
```



```
switch (event.getActionMasked()) {
    case MotionEvent.ACTION_DOWN:
        lastX = event.getX();
        lastY = event.getY();
        break;

    case MotionEvent.ACTION_MOVE:
        // Check if we're in the cooldown period after a scale
operation
        long currentTime = System.currentTimeMillis();
        if (event.getPointerCount() == 1 && currentTime -
lastScaleTime > SCALE_COOLDOWN_MS) {
            float dx = event.getX() - lastX;
            float dy = event.getY() - lastY;
            matrix.postTranslate(dx, dy);
            imageView.setImageMatrix(matrix);
            lastX = event.getX();
            lastY = event.getY();
        }
        break;

    case MotionEvent.ACTION_UP:
        v.performClick(); // 🛠️ Fix: Notify click happened
        break;
}

return true;
});

return view;
}

/**
 * Inner class that handles scale (zoom) gestures for the map.
 * <p>
 * This class:
 * <ul>
 *     <li>Processes pinch gestures to zoom in and out</li>
 *     <li>Enforces minimum (0.5x) and maximum (5.0x) zoom levels</li>
 *     <li>Applies scale transformations to the map's matrix</li>
 *     <li>Updates the cooldown timestamp to coordinate with pan
operations</li>
 * </ul>
 */
private class ScaleListener extends
ScaleGestureDetector.SimpleOnScaleGestureListener {
    /**
     * Called when a scale gesture is being performed.
     *
     * @param detector The detector that reported the event
     * @return True if the event was handled, false otherwise
     */
    @Override
```



```
public boolean onScale(ScaleGestureDetector detector) {
    scale *= detector.getScaleFactor();
    scale = Math.max(0.5f, Math.min(scale, 5.0f));
    matrix.postScale(detector.getScaleFactor(),
detector.getScaleFactor(),
        detector.getFocusX(), detector.getFocusY());
    imageView.setImageMatrix(matrix);

    // Update the last scale time to enforce the cooldown
    lastScaleTime = System.currentTimeMillis();
    return true;
}
}
```



SchoolUpdatesFragment

```
package com.example.projectarieleyalyotamyona.Fragments;

import android.os.Bundle;
import android.os.Handler;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.HorizontalScrollView;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import com.example.projectarieleyalyotamyona.R;
import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;

import java.util.List;

/**
 * This Fragment displays school updates or announcements in a
 * horizontally scrolling marquee.
 * It fetches the messages from a Firestore database, duplicates them for
 * continuous scrolling,
 * and handles the scrolling animation.
 */
public class SchoolUpdatesFragment extends Fragment {

    private HorizontalScrollView scrollView; // The container for the
    scrolling text.
    private TextView tvMessages;           // The TextView that holds
    the messages.
    private LinearLayout linearLayout;      // Holds TextViews, used for
    duplication.
    private Handler handler = new Handler(); // Handles the scrolling
    animation using postDelayed.
    private Runnable scrollRunnable;        // The code that performs
    the scrolling action.
    private int scrollSpeed = 3;            // Speed of the horizontal
    scrolling in pixels.
    private boolean isDuplicated = false;   // Flag to ensure messages
    are duplicated only once.
    private FirebaseFirestore db = FirebaseFirestore.getInstance(); //
    Access Firestore database.

    /**
     * Called when the Fragment is created. Initializes the UI and starts
     fetching data.
     */
}
```



```
*
* @param inflater      LayoutInflater to inflate the layout.
* @param container     ViewGroup that this fragment's UI will
attach to.
* @param savedInstanceState Bundle for any saved state (not used
here).
* @return The View for the fragment's UI.
*/
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
                          Bundle savedInstanceState) {

    // Inflate the layout for this fragment (fragment_system_not.xml).
    View view = inflater.inflate(R.layout.fragment__system_not,
container, false);

    // Get references to UI elements from the layout.
    scrollView = view.findViewById(R.id.horizontalScrollView);
    linearLayout = view.findViewById(R.id.linearLayout);
    tvMessages = view.findViewById(R.id.tvMessage);

    // Fetch the school updates from the Firestore database.
    db.collection("SharedData").document("Yoana").get()
        .addOnSuccessListener(new
OnSuccessListener<DocumentSnapshot>() {
        @Override
        public void onSuccess(DocumentSnapshot
documentSnapshot) {
            // Get the list of notifications from the
document.
            List<String> notiList = (List<String>)
documentSnapshot.get("NotiList");
            if (notiList != null) {
                // Build the message string, concatenating all
notifications.
                String message = "  ||  " + notiList.get(0);
                for (int i = 1; i < notiList.size(); i++) {
                    message = message + "  ||  " +
notiList.get(i);
                }
                tvMessages.setText(message); // Set the text
in the TextView.

                // Post a Runnable to the UI thread to start
the scrolling after the view is laid out.
                scrollView.post(() -> {
                    // Duplicate messages for continuous
scrolling effect.
                    if (!isDuplicated) {
                        int childCount =
linearLayout.getChildCount();
                        for (int i = 0; i < childCount; i++) {
                            View original =
linearLayout.getChildAt(i);
```



```
View copy = cloneView(original);
// Use helper to clone TextView
    linearLayout.addView(copy);
    }
    isDuplicated = true;
}

// Define the scrolling behavior in a
Runnable.
scrollRunnable = new Runnable() {
    @Override
    public void run() {
        int currentScroll =
scrollView.getScrollX();
        int originalContentWidth =
getOriginalContentWidth();

        scrollView.scrollBy(scrollSpeed,
0);

        // Loop back to the beginning when
the end of the original content is reached.
        if (currentScroll >=
originalContentWidth) {
            scrollView.scrollTo(0, 0);
        }
        handler.postDelayed(this, 10); //
Repeat scrolling after 10ms.
    }
};

// Start the scrolling animation with a
delay.
handler.postDelayed(scrollRunnable, 100);
});
}
}
.addOnFailureListener(new OnFailureListener() {
    @Override
    public void onFailure(@NonNull Exception e) {
        Toast.makeText(getContext(), "Failed to load
messages", Toast.LENGTH_LONG).show();
    }
});

return view;
}

/**
 * Calculates the total width of the original set of messages (before
duplication).
 * This is used to determine when to loop the scrolling back to the
beginning.
```



```
*
* @return The total width of the original messages in pixels.
*/
private int getOriginalContentWidth() {
    int half = linearLayout.getChildCount() / 2; // Width of only the
first half (original messages)
    int width = 0;
    for (int i = 0; i < half; i++) {
        width += linearLayout.getChildAt(i).getWidth();
    }
    return width;
}

/**
 * Clones a TextView, copying its properties (text, size, color,
etc.).
 * This is used to create the duplicated set of messages for
continuous scrolling.
 *
 * @param view The View (TextView) to be cloned.
 * @return A new TextView with the same properties as the original, or
null if the view is not a TextView.
 */
private View cloneView(View view) {
    if (view instanceof TextView) {
        TextView original = (TextView) view;
        TextView copy = new TextView(requireContext());
        copy.setLayoutParams(original.getLayoutParams());
        copy.setText(original.getText());
        copy.setTextSize(original.getTextSize() /
getResources().getDisplayMetrics().scaledDensity);
        copy.setTextColor(original.getTextColors());
        copy.setPadding(
            original.getPaddingLeft(),
            original.getPaddingTop(),
            original.getPaddingRight(),
            original.getPaddingBottom()
        );
        return copy;
    }
    return null;
}

/**
 * Called when the Fragment is no longer visible. Stops the scrolling
animation
 * to prevent it from running in the background.
 */
@Override
public void onPause() {
    super.onPause();
    handler.removeCallbacks(scrollRunnable); // Stop the scrolling.
}
```



```
/**
 * Called when the fragment resumes.
 * Restarts the scrolling animation.
 */
@Override
public void onResume() {
    super.onResume();
    if (scrollRunnable != null) {
        handler.postDelayed(scrollRunnable, 100);
    }
}
```



Activities

AddRoomActivity

```
package com.example.projectarieleyalyotamyona;

import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.Adapters.ScheduleAdapter;
import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Activity for adding a new room with a schedule to the database.
 * This activity allows the user to input a room number and define a
 * schedule for the room.
 * The schedule is managed using a RecyclerView and a custom adapter. The
 * room data, including
 * the schedule, is stored in a Firestore database.
 */
public class AddRoomActivity extends AppCompatActivity {

    private EditText editRoomNumber; // EditText for entering the
    room number.
    private RecyclerView recyclerViewSchedule; // RecyclerView to display
    and edit the room schedule.
    private Button btnCreateRoom; // Button to trigger the room
    creation process.
    private ScheduleAdapter scheduleAdapter; // Adapter for the
    RecyclerView to manage schedule data.
    private Map<String, List<String>> defaultSchedule; // Default schedule
    data (hour slots and availability).
    private FirebaseFirestore db; // Instance of Firebase
    Firestore for database interaction.

    /**
     * Called when the activity is created. Initializes the UI,
     * Firestore, and sets up the RecyclerView.
     */
}
```



```
* @param savedInstanceState If the activity is being re-initialized
after
* previously being shut down then this Bundle contains the data it
most
* recently supplied in {@link #onSaveInstanceState}.  Otherwise it is
null.
*/
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_add_room);

    // Initialize Firebase Firestore.
    db = FirebaseFirestore.getInstance();

    // Initialize UI components.
    editRoomNumber = findViewById(R.id.edit_room_number);
    recyclerViewSchedule = findViewById(R.id.recycler_view_schedule);
    btnCreateRoom = findViewById(R.id.btn_create_room);

    // Create the default schedule data.
    defaultSchedule = createDefaultSchedule();

    // Set up the RecyclerView and its adapter.
    scheduleAdapter = new ScheduleAdapter(this, defaultSchedule,
recyclerViewSchedule);
    recyclerViewSchedule.setLayoutManager(new
LinearLayoutManager(this));
    recyclerViewSchedule.setAdapter(scheduleAdapter);

    // Set up the button click listener to handle room creation.
    btnCreateRoom.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            createRoom();
        }
    });
}

/**
 * Creates the default schedule data.  The schedule is a map of hours
to a list of availability slots.
 *
 * @return A map representing the default schedule, where keys are
hours (e.g., "1", "2")
 * and values are lists of strings representing availability (e.g.,
"Available").
 */
private Map<String, List<String>> createDefaultSchedule() {
    Map<String, List<String>> schedule = new HashMap<>();
    List<String> hours = List.of("1", "2", "3", "4", "5", "6", "7",
"8", "9");
    for (String hour : hours) {
        schedule.put(hour, new ArrayList<>(List.of("Available",
```



```
"Available", "Available", "Available", "Available")));
    }
    return schedule;
}

/**
 * Creates a new room with the entered room number and schedule, and
 * uploads it to Firestore.
 * This method retrieves the room number from the EditText, gets the
 * updated schedule from the
 * adapter, creates a `Rooms` object, and then uploads this object to
 * the "Rooms" collection
 * in Firestore. It also handles input validation and displays
 * success/failure messages.
 */
private void createRoom() {
    String roomNumber = editRoomNumber.getText().toString().trim();
    if (roomNumber.isEmpty()) {
        Toast.makeText(this, "Please enter a room number",
            Toast.LENGTH_SHORT).show();
        return;
    }

    // Retrieve the updated schedule data from the adapter.
    Map<String, List<String>> updatedSchedule =
        scheduleAdapter.getScheduleData();

    // Create a Room object.
    Rooms newRoom = new Rooms(roomNumber, updatedSchedule);

    // Upload the new room data to Firestore.
    db.collection("Rooms")
        .document(roomNumber)
        .set(newRoom)
        .addOnSuccessListener(unused -> {
            Toast.makeText(AddRoomActivity.this, "Room created
            successfully!", Toast.LENGTH_SHORT).show();
            finish(); // Close the activity upon success.
        })
        .addOnFailureListener(e -> {
            Toast.makeText(AddRoomActivity.this, "Failed to create
            room: " + e.getMessage(), Toast.LENGTH_SHORT).show();
            Log.e("Firestore", "Error creating room", e); // Log
            the error.
        });
    }
}
```



AdvSearchActivity

```
package com.example.projectarieleyalyotamyona;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.NumberPicker;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.Adapters.RoomsAdapter;
import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;
import com.example.projectarieleyalyotamyona.Utilities.GetDay;
import com.example.projectarieleyalyotamyona.Utilities.GetIsT;
import com.example.projectarieleyalyotamyona.Utilities.RoomAdapterInterface;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * Activity for performing advanced search for available rooms based on
 * day and time.
 * This activity allows users to select a day and a time range (from hour
 * to hour)
 * and displays a list of rooms that are available within that criteria.
 */
public class AdvSearchActivity extends AppCompatActivity implements
RoomAdapterInterface {

    private NumberPicker dayPicker; // NumberPicker for selecting
the day.
    private NumberPicker hourFromPicker; // NumberPicker for selecting
the starting hour.
    private NumberPicker hourToPicker; // NumberPicker for selecting
the ending hour.
    private Button searchBT; // Button to trigger the search for
available rooms.
    private RoomsAdapter adapter; // Adapter for the RecyclerView
to display the list of rooms.
    private List<Rooms> roomList; // List to store the rooms that
match the search criteria.
```



```
/**
 * Called when the activity is created. Initializes the UI components,
 * configures the NumberPickers, and sets up the RecyclerView.
 *
 * @param savedInstanceState If the activity is being re-initialized
after
 * previously being shut down then this Bundle contains the data it
most
 * recently supplied in {@link #onSaveInstanceState}. Otherwise it is
null.
 */
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this); // Enable edge-to-edge display.
    setContentView(R.layout.activity_adv_search); // Set the layout
for this activity.

    // Adjust padding for system bars to avoid overlapping with UI
elements.
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
(v, insets) -> {
        Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom);
        return insets;
    });

    // Initialize view components.
    dayPicker = findViewById(R.id.dayPicker);
    hourFromPicker = findViewById(R.id.hourFromPicker);
    hourToPicker = findViewById(R.id.hourToPicker);

    // Configure the day picker.
    dayPicker.setMinValue(1);
    dayPicker.setMaxValue(5);
    dayPicker.setDisplayedValues(new String[]{"Sunday", "Monday",
"Tuesday", "Wednesday", "Thursday"});
    dayPicker.setWrapSelectorWheel(true); // Make the picker wrap
around.

    // Configure the hour pickers.
    hourFromPicker.setMinValue(1);
    hourFromPicker.setMaxValue(9);
    hourFromPicker.setWrapSelectorWheel(true);
    hourToPicker.setMinValue(1);
    hourToPicker.setMaxValue(9);
    hourToPicker.setDisplayedValues(new String[]{"1", "2", "3", "4",
"5", "6", "7", "8", "9"});
    hourToPicker.setWrapSelectorWheel(true);

    // Set default values for the pickers.
```



```
dayPicker.setValue(1);
hourFromPicker.setValue(1);
hourToPicker.setValue(2);

// Set up the RecyclerView to display the search results.
RecyclerView recyclerView =
findViewById(R.id.recycler_view_schedule);
recyclerView.setLayoutManager(new LinearLayoutManager(this));
roomList = new ArrayList<>(); // Initialize the room list.
adapter = new RoomsAdapter(AdvSearchActivity.this, roomList,
this); // Initialize the adapter.
recyclerView.setAdapter(adapter); // Set the adapter for the
RecyclerView.

// Set up the search button click listener.
searchBT = findViewById(R.id.advSearchBT);
searchBT.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        int selectedDay = dayPicker.getValue();
        int selectedFromHour = hourFromPicker.getValue();
        int selectedToHour = hourToPicker.getValue();
        // Validate the hour range.
        if (selectedFromHour <= selectedToHour) {
            roomList.clear(); // Clear the previous search
results.
            populateSearchRoomList(selectedDay, selectedFromHour,
selectedToHour); // Perform the search.
        } else {
            Toast.makeText(AdvSearchActivity.this, "Pick normal
hours", Toast.LENGTH_SHORT).show();
        }
    }
});

/**
 * Populates the list of rooms based on the selected day and time
range.
 * This method queries the Firestore database for rooms and checks
their availability
 * for the given day and time range.
 *
 * @param day The selected day (1-5).
 * @param hourFrom The starting hour of the time range.
 * @param hourTo The ending hour of the time range.
 */
private void populateSearchRoomList(int day, int hourFrom, int hourTo)
{
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("Rooms").get()
        .addOnSuccessListener(querySnapshot -> {
            if (querySnapshot != null && !querySnapshot.isEmpty())
{
```



```
for (DocumentSnapshot document : querySnapshot) {
    String roomNum =
document.getString("roomNum");
    Map<String, List<String>> schedule =
(Map<String, List<String>>) document.get("weekSched");

    boolean isOccupiedInRange = false;
    // Check if the room is occupied during any of
the selected hours.

    for (int i = hourFrom; i <= hourTo; i++) {
        List<String> daysList =
schedule.get(String.valueOf(i));
        // Check if the schedule for the hour
exists and if the day is within the schedule's range.
        if (daysList != null && day - 1 <
daysList.size()) {
            if ("Occupied".equals(daysList.get(day
- 1))) {
                isOccupiedInRange = true;
                break; // No need to check other
hours if one is occupied.
            }
        }
        // If the room is not occupied for the entire
range, add it to the result list.
        if (!isOccupiedInRange) {
            roomList.add(new Rooms(roomNum,
schedule));
        }
        adapter.notifyDataSetChanged(); // Notify the
adapter that the data has changed.
    }
}

.addOnFailureListener(e -> {
    // Handle the error silently, or add error-handling
logic if needed.
});

}

/**
 * Handles the item click event in the RecyclerView.
 * This method is called when a room item is clicked, and it opens the
MainActivity
 * to display the room's schedule, but only on weekdays.
 *
 * @param position The position of the clicked item in the
RecyclerView.
 */
@Override
public void onItemClick(int position) {
    // Only allow navigation on weekdays (Monday to Friday).
    if (GetDay.getCurrentDay() < 6) {
```



```
GetIsT isT = new GetIsT();
isT.checkIfTeacher(isTeacher -> {
    if (isTeacher) {
        Rooms selectedRoom = roomList.get(position); // Get
the selected room.
        // Create an intent to open the MainActivity.
        Intent intent = new Intent(this, MainActivity.class);
        intent.putExtra("Room", selectedRoom); // Pass the
selected room data.
        intent.putExtra("isP", true); // Indicate that the
room is being passed from this activity.
        startActivity(intent); // Start the MainActivity.
    }
});
} else {
    Toast.makeText(AdvSearchActivity.this, "Its the weekend || no
schedule", Toast.LENGTH_SHORT).show();
}
}
```



InfoAndBookRoomActivity

```
package com.example.projectarieleyalyotamyona;

import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.NumberPicker;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.Adapters.ScheduleTeachAdapter;
import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;
import com.example.projectarieleyalyotamyona.Data_Modules.User;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Activity for displaying room information and allowing users to book a
 * room for a specific time slot.
 * This activity retrieves room details from the intent, displays the
 * room's schedule,
 * and handles the booking process by updating the room's schedule in
 * Firestore
 * and appending the booking to the user's booking history.
 */
public class InfoAndBookRoomActivity extends AppCompatActivity {

    private NumberPicker dayPicker; // NumberPicker for selecting
    the day of the booking.
    private NumberPicker hourFromPicker; // NumberPicker for selecting
    the starting hour of the booking.
    private NumberPicker hourToPicker; // NumberPicker for selecting
    the ending hour of the booking.
    private Button bookRoomBT; // Button to initiate the room
    booking process.
    private FirebaseFirestore db; // Instance of Firebase
    Firestore for database interaction.
    private boolean isUp; // Flag to track update status
    (currently unused).
```



```
/**
 * Called when the activity is created.  Initializes the UI
 components,
 * retrieves room data from the intent, and sets up the RecyclerView
 to display the schedule.
 *
 * @param savedInstanceState If the activity is being re-initialized
 after
 * previously being shut down then this Bundle contains the data it
 most
 * recently supplied in {@link #onSaveInstanceState}.  Otherwise it is
 null.
 */
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_room_info_and_book);

    // Initialize NumberPickers for day and time selection.
    dayPicker = findViewById(R.id.dayPicker);
    hourFromPicker = findViewById(R.id.hourFromPicker);
    hourToPicker = findViewById(R.id.hourToPicker);

    // Configure Day Picker (1 to 5, representing days of the week).
    dayPicker.setMinValue(1);
    dayPicker.setMaxValue(5);
    dayPicker.setDisplayedValues(new String[]{"Sunday", "Monday",
    "Tuesday", "Wednesday", "Thursday"});
    dayPicker.setWrapSelectorWheel(true); // Allows the picker to
scroll in a loop.

    // Configure Hour Pickers (1 to 9, representing hours).
    hourFromPicker.setMinValue(1);
    hourFromPicker.setMaxValue(9);
    hourFromPicker.setWrapSelectorWheel(true);
    hourToPicker.setMinValue(1);
    hourToPicker.setMaxValue(9);
    hourToPicker.setDisplayedValues(new String[]{"1", "2", "3", "4",
    "5", "6", "7", "8", "9"});
    hourToPicker.setWrapSelectorWheel(true);

    // Set default values for the pickers.
    dayPicker.setValue(1);
    hourFromPicker.setValue(1);
    hourToPicker.setValue(2);

    // Retrieve the Rooms object, passed via intent.
    Rooms room = (Rooms) getIntent().getSerializableExtra("Room:");
    Map<String, List<String>> schedule = room.getSchedule(); // Get
the schedule from the Rooms object.

    // Initialize the book room button and set its click listener.
    bookRoomBT = findViewById(R.id.bookRoomBT);
```



```
bookRoomBT.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        int selectedDay = dayPicker.getValue(); // Get
selected day (1-5).
        int selectedFromHour = hourFromPicker.getValue(); // Get
selected start hour (1-9).
        int selectedToHour = hourToPicker.getValue(); // Get
selected end hour (1-9).

        // Get the day name for display in the confirmation
message.
        String[] days = {"Sunday", "Monday", "Tuesday",
"Wednesday", "Thursday"};
        String selectedDayName = days[selectedDay - 1]; // Adjust
for zero-based indexing.

        List<String> daysmap; // Temporary list to hold the
schedule for a specific hour.
        boolean flag = true; // Flag to indicate if the booking
is successful.
        List<String> hours = new ArrayList<>(); // List to store
the booked hours.
        int j = 0; // Counter for the hours list.

        // Validate the hour range.
        if (selectedFromHour <= selectedToHour) {
            // Iterate through the selected hours to check
availability.
            for (int i = selectedFromHour; i <= selectedToHour;
i++) {
                daysmap = schedule.get(Integer.toString(i)); //
Get the schedule for the current hour.
                String status = daysmap.get(selectedDay - 1); //
Get the status for the selected day.

                // If the room is available, mark it as occupied.
                if (status.equals("Available")) {

schedule.get(Integer.toString(i)).set(selectedDay - 1, "Occupied"); //
Update the schedule.
                    hours.add(j, String.valueOf(i)); // Add the
hour to the list of booked hours.
                    j++;
                } else {
                    // If the room is not available, show a
message and set the flag to false.
                    Toast.makeText(InfoAndBookRoomActivity.this,
"Bad schedule", Toast.LENGTH_LONG).show();
                    flag = false;
                    break; // Exit the loop after the first
conflict.
                }
            }
        }
    }
})
```



```
// If the booking was successful (no conflicts).
if (flag) {
    // Construct a confirmation message.
    String message = "Day: " + selectedDayName +
        " || Hours: " + selectedFromHour +
        "-" + selectedToHour;

    db = FirebaseFirestore.getInstance(); // Get
Firestore instance.

    // Update the room's schedule in Firestore.
    db.collection("Rooms")
        .document(room.getRoomNum())
        .update("weekSched", schedule) //
"weekSched" is the field name in Firestore.
        .addOnSuccessListener(unused -> {

Toast.makeText(InfoAndBookRoomActivity.this, message,
Toast.LENGTH_LONG).show();

        })
        .addOnFailureListener(e -> {

Toast.makeText(InfoAndBookRoomActivity.this, "Failed to update room: " +
e.getMessage(), Toast.LENGTH_SHORT).show();
            isUp = false; // Set the flag to
indicate failure (currently unused).
        });

        SetUpRoom(room); // Update the RecyclerView to
show the updated schedule.

        // Create a Booking object to store in the user's
document.
        User.Booking booking = new
User.Booking(room.getRoomNum(), selectedDayName, hours);
        FirebaseAuth auth = FirebaseAuth.getInstance(); //
Get Auth instance.
        FirebaseUser user = auth.getCurrentUser(); //
Get the current user.

        // If a user is logged in, update their booking
history.
        if (user != null) {
            String id = user.getId(); // Get the user's
ID.

            db.collection("users")
                .document(id)
                .update("bookedRooms",
FieldValue.arrayUnion(booking)) // "bookedRooms" is the field name.
                .addOnSuccessListener(aVoid -> {

Toast.makeText(InfoAndBookRoomActivity.this, "Booking appended
```



```
successfully!", Toast.LENGTH_SHORT).show();
    })
    .addOnFailureListener(e -> {

Toast.makeText(InfoAndBookRoomActivity.this, "Error appending booking: " +
e.getMessage(), Toast.LENGTH_SHORT).show();
        isUp = false; // Set flag on
failure (currently unused).
    });
    } else {
        Log.e("InfoAndBookRoomActivity", "No
authenticated user found.");
    }
    } else {
        // Show a message if the hour range is invalid.
        Toast.makeText(InfoAndBookRoomActivity.this, "From is
bigger then to ", Toast.LENGTH_SHORT).show();
    }
}
});

// If a room object was successfully passed to this activity.
if (room != null) {
    String roomNumber = room.getRoomNum(); // Extract the room
number.

    // Display the room number.
    TextView roomNumberTextView =
findViewById(R.id.roomNumberTextView);
    if (roomNumberTextView != null) {
        roomNumberTextView.setText("Room number: " + roomNumber);
    } else {
        Log.e("InfoAndBookRoomActivity", "TextView for room number
not found!");
    }

    SetUpRoom(room); // Set up the RecyclerView to display the
room's schedule.
    } else {
        Log.e("InfoAndBookRoomActivity", "Rooms object is null!");
    }
}

/**
 * Sets up the RecyclerView to display the room's schedule.
 *
 * @param room The Rooms object containing the schedule data.
 */
public void SetUpRoom(Rooms room) {
    Map<String, List<String>> schedule = room.getSchedule(); // Get
the schedule from the Rooms object.
    RecyclerView recyclerView =
```



```
findViewById(R.id.recycler_view_schedule); // Find the RecyclerView.
recyclerView.setLayoutManager(new LinearLayoutManager(this)); //
Set the layout manager.

// Convert the schedule (which might be a HashMap) to a Map that's
easier to use with the adapter.
Map<String, List<String>> scheduleMap = new HashMap<>();
for (Map.Entry<String, List<String>> entry : schedule.entrySet())
{
    scheduleMap.put(entry.getKey(), entry.getValue());
}

// Create and set the adapter for the RecyclerView.
ScheduleTeachAdapter adapter = new ScheduleTeachAdapter(this,
scheduleMap);
recyclerView.setAdapter(adapter);
}
```



LoginActivity

```
package com.example.projectarieleyalyotamyona;

import android.annotation.SuppressLint;
import android.os.Bundle;
import android.content.Intent;
import android.text.TextUtils;
import android.util.Log;
import android.view.View;
import android.widget.CheckBox;
import android.widget.ProgressBar;
import android.widget.Toast;
import android.widget.EditText;
import android.widget.Button;

import com.example.projectarieleyalyotamyona.Data_Modules.User;
import com.example.projectarieleyalyotamyona.Utilities.PrefManager;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.AuthResult;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;
import androidx.activity.OnBackPressedCallback;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

/**
 * Activity for user login functionality.
 * This activity allows users to log in to their accounts using email and
 * password.
 * It also provides options for "Remember Me" functionality using shared
 * preferences.
 */
public class LoginActivity extends AppCompatActivity {

    private EditText emailTextView; // EditText for entering the
    user's email.
    private EditText passwordTextView; // EditText for entering the
    user's password.
    private Button Btn; // Button to initiate the login
    process.
    private ProgressBar progressbar; // ProgressBar to show loading
    during login.
    private FirebaseAuth mAuth; // Firebase Authentication
    instance.
    private Button ToRegi; // Button to navigate to the
    registration activity.
```



```
private CheckBox checkBoxRememberMe; // CheckBox for "Remember Me"
functionality.

/**
 * Called when the activity is created. Initializes the UI components,
 * sets up Firebase Authentication, and handles login logic.
 *
 * @param savedInstanceState If the activity is being re-initialized
after
 * previously being shut down then this Bundle contains the data it
most
 * recently supplied in {@link #onSaveInstanceState}. Otherwise it is
null.
 */
@SuppressLint("MissingInflatedId")
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    // Disable back button functionality
    getOnBackPressedDispatcher().addCallback(this, new
OnBackPressedCallback(true) {
        @Override
        public void handleOnBackPressed() {
            // Do nothing
        }
    });

    setContentView(R.layout.activity_login);

    // Firebase Authentication instance
    mAuth = FirebaseAuth.getInstance();

    // Initializing views
    emailTextView = findViewById(R.id.email);
    passwordTextView = findViewById(R.id.password);
    Btn = findViewById(R.id.login);
    progressBar = findViewById(R.id.progressBar);
    ToRegi = findViewById(R.id.Register);
    checkBoxRememberMe = findViewById(R.id.checkBoxRememberMe);

    // RegisterActivity button click listener to redirect to
RegisterActivity activity
    ToRegi.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent intent = new Intent(LoginActivity.this,
RegisterActivity.class);
            startActivity(intent);
        }
    });

    // Check if user is already logged in
    if (!new PrefManager(this).isUserLoggedIn()) {
```



```
        startHomeActivity();
    }

    // Set onClickListener for login button
    Btn.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            loginUserAccount();
        }
    });
}

/**
 * Starts the main activity after successful login.
 */
private void startHomeActivity() {
    Intent intent = new Intent(LoginActivity.this,
MainActivity.class);
    startActivity(intent);
    finish();
}

/**
 * Handles the user login process, including input validation,
 * Firebase authentication, and redirection to the main activity.
 */
private void loginUserAccount() {
    // Show progress bar
    progressBar.setVisibility(View.VISIBLE);

    // Retrieve email and password inputs
    String email = emailTextView.getText().toString();
    String password = passwordTextView.getText().toString();

    // Validate email and password
    if (TextUtils.isEmpty(email)) {
        Toast.makeText(getApplicationContext(), "Please enter email!",
Toast.LENGTH_LONG).show();
        progressBar.setVisibility(View.GONE);
        return;
    }

    if (TextUtils.isEmpty(password)) {
        Toast.makeText(getApplicationContext(), "Please enter
password!", Toast.LENGTH_LONG).show();
        progressBar.setVisibility(View.GONE);
        return;
    }

    // Attempt sign-in with Firebase Authentication
    mAuth.signInWithEmailAndPassword(email, password)
        .addOnCompleteListener(new
OnCompleteListener<AuthResult>() {
        @Override
```



```
        public void onComplete(@NonNull Task<AuthResult> task)
        {
            checkAndUpdateFirestoreEmail();
            if (task.isSuccessful()) {
                FirebaseFirestore database =
                FirebaseFirestore.getInstance();
                FirebaseUser userId = mAuth.getCurrentUser();

                // Check if "Remember Me" is checked to save
                login details

                if (checkBoxRememberMe.isChecked()) {
                    saveLoginDetails(email, password);
                }

                if (userId != null) {
                    DocumentReference docRef =
                    database.collection("users").document(userId.getId());
                    docRef.get().addOnSuccessListener(new
                    OnSuccessListener<DocumentSnapshot>() {
                        @Override
                        public void onSuccess(DocumentSnapshot
                        documentSnapshot) {
                            User user =
                            documentSnapshot.toObject(User.class);
                            if (user != null) {
                                Toast.makeText(LoginActivity.this, "Welcome! " + user.getName(),
                                Toast.LENGTH_SHORT).show();
                                startHomeActivity();
                            } else {
                                Toast.makeText(LoginActivity.this, "User data not found!",
                                Toast.LENGTH_SHORT).show();
                            }
                        }
                    });

                    progressbar.setVisibility(View.GONE);
                }
            }
            else {
                // Sign-in failed
                Toast.makeText(getApplicationContext(),
                "LoginActivity failed!", Toast.LENGTH_LONG).show();
                progressbar.setVisibility(View.GONE);
            }
        }
    });
}

/**
 * Checks if the email in Firestore is the same as the current user's
 * email and updates it if necessary.
 * This method is used to ensure that the email stored in the
 * Firestore database is up-to-date
 * with any changes the user might have made to their account.
 */
```



```
*/
private void checkAndUpdateFirestoreEmail() {
    FirebaseUser currentUser =
    FirebaseAuth.getInstance().getCurrentUser();

    if (currentUser != null) {
        String newEmail = currentUser.getEmail(); // Get updated email
        String userId = currentUser.getUid(); // Get user ID

        FirebaseFirestore db = FirebaseFirestore.getInstance();
        DocumentReference userRef =
        db.collection("users").document(userId);

        userRef.get().addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                String savedEmail =
                documentSnapshot.getString("email"); // Get stored email in Firestore

                if (!newEmail.equals(savedEmail)) { // If emails do
                not match, update Firestore
                    userRef.update("email", newEmail)
                        .addOnSuccessListener(aVoid ->
                Log.d("DEBUG", "Firestore email updated after verification"))
                        .addOnFailureListener(e -> Log.e("DEBUG",
                "Failed to update email in Firestore: " + e.getMessage()));
                }
            }
        }).addOnFailureListener(e -> Log.e("DEBUG", "Failed to
        retrieve user data: " + e.getMessage()));
    }
}

/**
 * Saves the user's login details (email and password) using
 * PrefManager.
 * This is used for the "Remember Me" functionality.
 *
 * @param email The user's email.
 * @param password The user's password.
 */
private void saveLoginDetails(String email, String password) {
    new PrefManager(this).saveLoginDetails(email, password);
}
}
```



MainActivity

```
package com.example.projectarieleyalyotamyona;

import androidx.annotation.NonNull;
import androidx.appcompat.app.ActionBarDrawerToggle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.app.AppCompatActivityDelegate;
import androidx.appcompat.widget.Toolbar;
import androidx.core.view.GravityCompat;
import androidx.drawerlayout.widget.DrawerLayout;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;

import android.content.Intent;
import android.graphics.Bitmap;
import android.os.Bundle;
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import com.example.projectarieleyalyotamyona.Data_Modules.Rooms;
import com.example.projectarieleyalyotamyona.Fragments.AccountFragment;
import com.example.projectarieleyalyotamyona.Fragments.DailyScheduleFragment;
import com.example.projectarieleyalyotamyona.Fragments.HomeFragment;
import com.example.projectarieleyalyotamyona.Fragments.MainRoomsFragment;
import com.example.projectarieleyalyotamyona.Fragments.ManageRoomsFragment;
import com.example.projectarieleyalyotamyona.Fragments.ManageSchoolNotificationsF
ragment;
import com.example.projectarieleyalyotamyona.Fragments.MapFragment;
import com.example.projectarieleyalyotamyona.Utilities.GetEmail;
import com.example.projectarieleyalyotamyona.Utilities.GetFirstHours;
import com.example.projectarieleyalyotamyona.Utilities.GetIsStaff;
import com.example.projectarieleyalyotamyona.Utilities.GetName;
import com.example.projectarieleyalyotamyona.Utilities.GetProfilePic;
import com.example.projectarieleyalyotamyona.Utilities.PrefManager;
import com.google.android.material.bottomnavigation.BottomNavigationView;
import com.google.android.material.navigation.NavigationView;

import java.util.List;

/**
 * Main activity of the application.
 * This activity serves as the central hub, providing navigation and
controlling the display of different fragments.
 * It implements a drawer layout for side navigation and a bottom
```



```
navigation bar for primary navigation.
*/
public class MainActivity extends AppCompatActivity implements
NavigationView.OnNavigationItemSelectedListener {
    private DrawerLayout drawerLayout; // Layout for the navigation
drawer.
    private GetName name; // Utility class to get the user's
name.
    private GetEmail email; // Utility class to get the user's
email.
    private TextView nameT; // TextView to display the user's name
in the navigation drawer.
    private TextView EmailT; // TextView to display the user's
email in the navigation drawer.
    public static ImageView profilePic; // ImageView to display the user's
profile picture.
    private static final String CHANNEL_ID = "notification_channel"; //
Constant for notification channel ID (currently unused).

    /**
     * Called when the activity is created.
     * Initializes the layout, toolbar, navigation drawer, bottom
navigation, and loads the initial fragment.
     */
    * @param savedInstanceState If the activity is being re-initialized
after
    * previously being shut down then this Bundle contains the data it
most
    * recently supplied in {@link #onSaveInstanceState}. Otherwise it is
null.
    */
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Set up the toolbar.
        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);

        // Set up the navigation drawer.
        drawerLayout = findViewById(R.id.drawer_layout);
        NavigationView navigationView = findViewById(R.id.nav_view);
        navigationView.setNavigationItemSelectedListener(this);
        ActionBarDrawerToggle toggle = new ActionBarDrawerToggle(this,
drawerLayout, toolbar, R.string.open_nav,
        R.string.close_nav);
        drawerLayout.addDrawerListener(toggle);
        toggle.syncState();

        // Disable night mode.

AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_NO);
```



```
// Load the home fragment as the initial fragment.

getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container, new HomeFragment()).commit();

// Check for intent data to display a specific fragment.
if (getIntent().getSerializableExtra("isP") != null && (boolean)
getIntent().getSerializableExtra("isP")) {
    // Get the passed data (Room object).
    Rooms selectedRoom = (Rooms)
getIntent().getSerializableExtra("Room");

    // Only proceed with fragment transaction if the selectedRoom
is not null.
    if (selectedRoom != null) {
        // Create the fragment and pass the data.
        DailyScheduleFragment dailyScheduleFragment = new
DailyScheduleFragment();
        Bundle bundle = new Bundle();
        bundle.putSerializable("Room", selectedRoom); // Ensure
Rooms class implements Serializable.
        dailyScheduleFragment.setArguments(bundle); // Attach the
bundle to the fragment.

        // Perform the fragment transaction.
        FragmentManager fragmentManager =
getSupportFragmentManager();
        FragmentTransaction transaction =
fragmentManager.beginTransaction();
        transaction.replace(R.id.fragment_container,
dailyScheduleFragment); // Replace with the target fragment.
        transaction.addToBackStack(null); // Allows the user to
navigate back to the previous fragment.
        transaction.commit(); // Apply the transaction.
    }
} else if (savedInstanceState == null) {
    // If no specific fragment is requested, load the home
fragment.

getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container, new HomeFragment()).commit();
    navigationView.setCheckedItem(R.id.nav_home);
}

// Initialize utility classes to get user data.
name = new GetName();
name.ToolText(name -> {
    // Set the toolbar title with the user's name.
    toolbar.setTitle("Welcome " + name.toString());
    nameT.setText(name.toString());
});
email = new GetEmail();
email.ToolText(email -> {
    EmailT.setText(email);
```



```
});

// Get references to views in the navigation drawer header.
View headerView = navigationView.getHeaderView(0); // Access the
first (and only) header view.
nameT = headerView.findViewById(R.id.name);
EmailT = headerView.findViewById(R.id.email22);
profilePic = headerView.findViewById(R.id.profile_picture);

// Load the user's profile picture.
GetProfilePic profilePic1 = new GetProfilePic();
profilePic1.getProfilePic(new GetProfilePic.ToolTextCallback() {
    @Override
    public void onProfilePicFetched(Bitmap profilePic1) {
        // If the profile picture is fetched successfully, set it
in the ImageView.
        if (profilePic1 != null) {
            profilePic.setImageBitmap(profilePic1);
        }
    }
});

// Set up the bottom navigation view.
BottomNavigationView bottomNavigationView =
findViewById(R.id.bottomNavigationView);
bottomNavigationView.setOnNavigationItemSelectedListener(new
BottomNavigationView.OnNavigationItemSelectedListener() {
    @Override
    public boolean onNavigationItemSelected(@NonNull MenuItem
item) {
        int itemId = item.getItemId();
        if (itemId == R.id.nav_home) {
getSupportFragmentManager().beginTransaction().replace(R.id.fragment_conta
iner, new HomeFragment()).commit();
        } else if (itemId == R.id.map) {
getSupportFragmentManager().beginTransaction().replace(R.id.fragment_conta
iner, new MapFragment()).commit();
        } else if (itemId == R.id.rooms) {
getSupportFragmentManager().beginTransaction().replace(R.id.fragment_conta
iner, new MainRoomsFragment()).commit();
        }
        return true;
    }
});

// Get references to menu items that might be shown/hidden based
on user role
Menu menu = navigationView.getMenu();
MenuItem mR = menu.findItem(R.id.nav_manage_rooms);
MenuItem mN = menu.findItem(R.id.nav_SystemNoti);
```



```
// Check the user's staff status and show/hide admin-related menu
items.

GetIsStaff getIsStaff = new GetIsStaff();
getIsStaff.checkIfStaff(isStaff -> {
    if (isStaff) {
        // Perform actions for staff.
        mR.setVisible(true); // Show the menu item for staff.
        mN.setVisible(true);
        Log.d("MainActivity", "The user is a staff member. Grant
access to staff features.");
    } else {
        // Perform actions for non-staff or error.
        mR.setVisible(false); // Hide the menu item for non-staff.
        mN.setVisible(false);
        Log.d("MainActivity", "The user is not a staff member.
Access denied or an error occurred.");
    }
});

//set the alarms for the user's bookings.
GetFirstHours getFirstHours = new GetFirstHours();
if (getFirstHours == null) {
    Toast.makeText(MainActivity.this, "No bookings",
Toast.LENGTH_SHORT).show();
}
getFirstHours.GetFirstHours(new GetFirstHours.HoursCallback() {
    @Override
    public void onSuccess(List<Integer> firstHours) {
        // Handle success
        Log.d("Success", "First hours: " + firstHours);
        // Further logic, like setting alarms
        for (int hour : firstHours) {
            SetNoti.convertAndSet(MainActivity.this, hour); //
Set alarms as needed
        }
    }

    @Override
    public void onFailure(Exception e) {
        // Handle failure
        Log.e("Error", "Error loading first hours", e);
        Toast.makeText(MainActivity.this, "Error loading first
hours", Toast.LENGTH_SHORT).show();
    }
});
}

/**
 * Handles navigation drawer item clicks.
 * Loads the selected fragment and closes the drawer.
 *
 * @param item The menu item that was selected.
 * @return true if the item was handled, false otherwise.
 */
```



```
@Override
public boolean onNavigationItemSelected(@NonNull MenuItem item) {
    int itemId = item.getItemId();
    if (itemId == R.id.nav_home) {

        getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container, new HomeFragment()).commit();
    } else if (itemId == R.id.nav_account) {

        getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container, new AccountFragment()).commit();
    } else if (itemId == R.id.nav_SystemNoti) {

        getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container, new ManageSchoolNotificationsFragment()).commit();
    } else if (itemId == R.id.nav_manage_rooms) {

        getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container, new ManageRoomsFragment()).commit();
    } else if (itemId == R.id.nav_logout) {
        // Handle logout: show a toast, clear login details, and start
        the LoginActivity.
        Toast.makeText(this, "Logout!", Toast.LENGTH_SHORT).show();
        new PrefManager(this).Forgetlog();
        Intent intent = new Intent(MainActivity.this,
        LoginActivity.class);
        startActivity(intent);
    }

    drawerLayout.closeDrawer(GravityCompat.START);
    return true;
}

/**
 * Handles back button presses.
 * Closes the navigation drawer if it's open, otherwise behaves like
 the default back button.
 */
@Override
public void onBackPressed() {
    if (drawerLayout.isDrawerOpen(GravityCompat.START)) {
        drawerLayout.closeDrawer(GravityCompat.START);
    } else {
        super.onBackPressed();
    }
}
}
```



MyRoomsActivity

```
package com.example.projectarieleyalyotamyona;

import android.os.Bundle;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.projectarieleyalyotamyona.Adapters.UserRoomsAdapter;
import com.example.projectarieleyalyotamyona.Data_Modules.User;
import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * Activity for displaying the user's booked rooms.
 * This activity retrieves and displays a list of rooms that the current
 * user has booked,
 * using a RecyclerView and a custom adapter.
 */
public class MyRoomsActivity extends AppCompatActivity {

    private RecyclerView recyclerView; // RecyclerView to display
    the list of booked rooms.
    private UserRoomsAdapter adapter; // Adapter for the RecyclerView.
    private List<User.Booking> userRooms; // List to store the user's
    booked rooms.

    /**
     * Called when the activity is created.
     * Initializes the RecyclerView, adapter, and loads the user's booked
     * rooms from Firestore.
     *
     * @param savedInstanceState If the activity is being re-initialized
     * after
     * previously being shut down then this Bundle contains the data it
     * most
     * recently supplied in {@link #onSaveInstanceState}. Otherwise it is
     * null.
     */
}
```



```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_my_rooms);

    // Handle Edge-To-Edge compatibility

    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.recycler_view)
    , (v, insets) -> {
        Insets systemInsets =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemInsets.left, systemInsets.top,
systemInsets.right, systemInsets.bottom);
        return WindowInsetsCompat.CONSUMED;
    });

    // Initialize RecyclerView
    recyclerView = findViewById(R.id.recycler_view);
    recyclerView.setLayoutManager(new LinearLayoutManager(this));

    // Initialize and populate the room list
    userRooms = new ArrayList<>();
    adapter = new UserRoomsAdapter(MyRoomsActivity.this, userRooms);
    recyclerView.setAdapter(adapter);

    // Populate user room list with data
    populateUserRoomList();
}

/**
 * Retrieves the user's booked rooms from Firestore and populates the
 * RecyclerView.
 * It fetches the data from the "users" collection, using the current
 * user's ID.
 */
private void populateUserRoomList() {
    FirebaseAuth mAuth = FirebaseAuth.getInstance();
    FirebaseUser currentUser = mAuth.getCurrentUser();
    String userId = currentUser != null ? currentUser.getId() : "";

    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("users").document(userId).get()
        .addOnSuccessListener(new
OnSuccessListener<DocumentSnapshot>() {
            @Override
            public void onSuccess(DocumentSnapshot
documentSnapshot) {
                // Get the bookedRooms data, which might be a
                List<Map<String, Object>>
                List<Map<String, Object>> bookingsData =
                (List<Map<String, Object>>) documentSnapshot.get("bookedRooms");

                if (bookingsData != null) {
                    userRooms.clear();
                }
            }
        })
    .addOnFailureListener(new OnFailureListener() {
        @Override
        public void onFailure(@NonNull Exception e) {
            // Handle failure
        }
    });
}
```



```
for (Map<String, Object> bookingData :
bookingsData) {
    // Manually create User.Booking object
    from Map
    String roomNumber = (String)
    bookingData.get("roomNumber");
    String day = (String)
    bookingData.get("day");
    List<String> hourStatuses = (List<String>)
    bookingData.get("hourStatuses");

    // Assuming User.Booking has a constructor
    that takes these values
    User.Booking booking = new
    User.Booking(roomNumber, day, hourStatuses);
    userRooms.add(booking);
}
adapter.notifyDataSetChanged();
}
})
.addOnFailureListener(new OnFailureListener() {
    @Override
    public void onFailure(@NonNull Exception e) {
        Toast.makeText(MyRoomsActivity.this, "Error
loading rooms", Toast.LENGTH_SHORT).show();
    }
});
}

/**
 * Called when the activity is resumed.
 * Clears the current room list and repopulates it to ensure the
 * displayed data is up-to-date.
 */
@Override
protected void onResume() {
    super.onResume();
    // Clear the list and repopulate on resume
    userRooms.clear();
    populateUserRoomList();
}
}
```



RegisterActivity

```
package com.example.projectarieleyalyotamyona;

import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AutoCompleteTextView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ProgressBar;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import com.example.projectarieleyalyotamyona.Data_Modules.User;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

/**
 * Activity for registering new users (students and teachers).
 * This activity handles user input, validates it, checks for teacher
 * status,
 * and creates a new user account in Firebase Authentication and
 * Firestore.
 */
public class RegisterActivity extends AppCompatActivity {
    private EditText emailTextView, passwordTextView, nameEditT,
    TeachPassT, Class1; // Input fields.
    private Button Btn; // Register button.
    private ProgressBar progressBar; // Progress indicator.
    private FirebaseAuth mAuth; // Firebase Authentication instance.
    private FirebaseFirestore database; // Firebase Firestore instance.

    private String[] classarr = { // Array of valid class names.
        "ז'1", "ז'2", "ז'3", "ז'4", "ז'5", "ז'6", "ז'7",
        "ח'1", "ח'2", "ח'3", "ח'4", "ח'5", "ח'6", "ח'7",
        "ט'1", "ט'2", "ט'3", "ט'4", "ט'5", "ט'6", "ט'7",
        "י'1", "י'2", "י'3", "י'4", "י'5", "י'6", "י'7",
        "יא'1", "יא'2", "יא'3", "יא'4", "יא'5", "יא'6", "יא'7",
        "יב'1", "יב'2", "יב'3", "יב'4", "יב'5", "יב'6", "יב'7"
    };

    private boolean IsT = false; // Flag to indicate if the user is a
    teacher.

    /**
```



```
* Called when the activity is created.
* Initializes the views, Firebase Authentication, and sets up the
AutoCompleteTextView for class selection.
*
* @param savedInstanceState If the activity is being re-initialized
after
* previously being shut down then this Bundle contains the data it
most
* recently supplied in {@link #onSaveInstanceState}. Otherwise it is
null.
*/
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_register);

    // Initialize FirebaseAuth and FirebaseDatabase
    mAuth = FirebaseAuth.getInstance();

    // Initialize views
    emailTextView = findViewById(R.id.email);
    passwordTextView = findViewById(R.id.passwd);
    Btn = findViewById(R.id.btnregister);
    progressbar = findViewById(R.id.progressbar);
    nameEditT = findViewById(R.id.name);
    TeachPassT = findViewById(R.id.TechPass);
    Class1 = findViewById(R.id.ACTV);

    // Set up AutoCompleteTextView for class selection.
    AutoCompleteTextView simpleAutoCompleteTextView =
findViewById(R.id.ACTV);
    ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
android.R.layout.simple_list_item_1, classarr);
    simpleAutoCompleteTextView.setAdapter(adapter);
    simpleAutoCompleteTextView.setThreshold(1); // Start suggesting
after 1 character.

    // Set click listener on the register button to initiate the
registration process.
    Btn.setOnClickListener(v -> {
        checkTeachPass(TeachPassT.getText().toString());
    });
}

/**
 * Validates the user input fields (email, password, name, and class).
 * Displays a Toast message for each validation error.
 *
 * @return true if all fields are valid, false otherwise.
 */
private boolean validateFields() {
    String email = emailTextView.getText().toString();
    String password = passwordTextView.getText().toString();
    String name = nameEditT.getText().toString();
}
```



```
String class1 = Class1.getText().toString();

    if (TextUtils.isEmpty(email)) {
        Toast.makeText(getApplicationContext(), "Please enter email!",
Toast.LENGTH_LONG).show();
        return false;
    }
    if (!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches())
{
        Toast.makeText(getApplicationContext(), "Please enter a valid
email!", Toast.LENGTH_LONG).show();
        return false;
    }
    if (TextUtils.isEmpty(password)) {
        Toast.makeText(getApplicationContext(), "Please enter
password!", Toast.LENGTH_LONG).show();
        return false;
    }
    if (password.length() < 4) {
        Toast.makeText(getApplicationContext(), "Password must be at
least 4 characters long!", Toast.LENGTH_LONG).show();
        return false;
    }
    if (TextUtils.isEmpty(name)) {
        Toast.makeText(getApplicationContext(), "Please enter name!",
Toast.LENGTH_LONG).show();
        return false;
    }
    // Class validation logic:
    if (!IsClassValid(classarr, class1) && !IsT) {
        Toast.makeText(getApplicationContext(), "Please select a valid
class!", Toast.LENGTH_LONG).show();
        return false;
    }
    if (IsClassValid(classarr, class1) && IsT) {
        Toast.makeText(getApplicationContext(), "Teachers should not
pick a class", Toast.LENGTH_LONG).show();
        Class1.setText(""); // Clear the class input.
        return false;
    }
    if (!IsClassValid(classarr, class1) && IsT) {
        Class1.setText("");
        return true;
    }

    return true;
}

/**
 * Checks if the provided class name is valid.
 *
 * @param arr      Array of valid class names.
 * @param inputClass The class name to validate.
 * @return true if the class name is valid, false otherwise.
 */
```



```
*/
private boolean IsClassValid(String[] arr, String inputClass) {
    for (String cls : arr) {
        if (cls.equals(inputClass)) {
            return true;
        }
    }
    return false;
}

/**
 * Checks if the user is a teacher based on the provided teacher
 * password.
 * It retrieves the stored teacher password from Firestore and
 * compares it with the input.
 *
 * @param TeachPassT The teacher password entered by the user.
 */
public void checkTeachPass(String TeachPassT) {
    FirebaseFirestore db = FirebaseFirestore.getInstance();

    db.collection("SharedData").document("Yoana").get()
        .addOnSuccessListener(document -> {
            if (document.exists()) {
                String storedTeachPass =
document.getString("TeachPass");

                if (TeachPassT != null &&
TeachPassT.equals(storedTeachPass)) {
                    IsT = true; // User is a teacher.

                } else {
                    IsT = false; // User is not a teacher.
                }
                //after checking the teacher pass, validate the
fields and register the user.
                if (validateFields()) {
                    registerNewUser(IsT);
                }
            } else {
                Toast.makeText(this, "Document does not exist!",
Toast.LENGTH_SHORT).show();
            }
        })
        .addOnFailureListener(e -> {
            Toast.makeText(this, "Error fetching document: " +
e.getMessage(), Toast.LENGTH_SHORT).show();
        });
}

/**
 * Registers a new user with Firebase Authentication and stores user
 * data in Firestore.
 */
}
```



```
* @param IsT Indicates whether the user is a teacher or a student.
*/
private void registerNewUser(boolean IsT) {
    progressBar.setVisibility(View.VISIBLE); // Show progress bar.

    String email = emailTextView.getText().toString();
    String password = passwordTextView.getText().toString();
    String name = nameEditT.getText().toString();
    String class1 = Class1.getText().toString();

    // Create a User object (password is NOT stored in Firestore).
    User user = new User(email, name, class1, IsT, false);

    // Create the user with email and password using Firebase
    Authentication.
    mAuth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(new
    OnCompleteListener<AuthResult>() {
        @Override
        public void onComplete(@NonNull Task<AuthResult> task)
    {
        progressBar.setVisibility(View.GONE); // Hide
    progress bar.

        if (task.isSuccessful()) {
            // Registration successful.
            Toast.makeText(getApplicationContext(),
                IsT ? "Registration successful! You
are registered as a teacher." :
                "Registration successful! You
are registered as a student.",
                Toast.LENGTH_LONG).show();

            // Get the current Firebase User.
            FirebaseUser userCurrent =
mAuth.getCurrentUser();

            database = FirebaseFirestore.getInstance();

            // Store the user data in Firestore.

            database.collection("users").document(userCurrent.getUid()).set(user);

            // Start the LoginActivity.
            Intent intent = new
            Intent(RegisterActivity.this, LoginActivity.class);
            startActivity(intent);
        } else {
            // Registration failed.
            Toast.makeText(getApplicationContext(),
                "Registration failed! Please try again
later.",
                Toast.LENGTH_LONG).show();
        }
    }
}
```



5 יחידות לימוד מדעי המחשב - (luz) - אריאל חלף

```
    } ) ;  
}  
}
```



Utilities

GetDay

```
package com.example.projectarieleyalyotamyona.Utilities;

import java.util.Calendar;
import java.util.TimeZone;

/**
 * Utility class for getting the current day of the week.
 * This class provides methods to retrieve the current day as both an
 * integer
 * and a string, using the Israel time zone.
 */
public class GetDay {

    /**
     * Gets the current day of the week as an integer.
     * <p>
     * The Calendar class returns the day of the week as an integer, where
     * {@link Calendar#SUNDAY} is 1, {@link Calendar#MONDAY} is 2, and so
     on.
     * This method uses the Israel time zone (GMT+2) to ensure correct day
     calculation.
     *
     * @return An integer representing the current day of the week (1 for
     Sunday, 2 for Monday, etc.).
     */
    public static int getCurrentDay() {
        // Set calendar to Israel's GMT+2/3
        Calendar calendar =
Calendar.getInstance(TimeZone.getTimeZone("GMT+2"));

        // Get the current day of the week
        int dayOfWeek = calendar.get(Calendar.DAY_OF_WEEK);

        // Return the day
        return dayOfWeek;
    }

    /**
     * Gets the current day of the week as a string.
     * <p>
     * This method uses the {@link #getCurrentDay()} method to get the day
     as an integer
     * and then converts it to the corresponding day name string. It also
     uses
     * the Israel time zone (GMT+2) to ensure correct day name.
     *
     * @return A string representing the current day of the week (e.g.,
     "Sunday", "Monday").
     * Returns "Unknown" if the day is not recognized.
     */
}
```



```
*/  
public static String getCurrentDayName() {  
    // Set calendar to Israel's GMT+2/3  
    Calendar calendar =  
Calendar.getInstance(TimeZone.getTimeZone("GMT+2"));  
  
    // Get the current day of the week  
    int dayOfWeek = calendar.get(Calendar.DAY_OF_WEEK);  
  
    // Return the day name  
    switch (dayOfWeek) {  
        case Calendar.SUNDAY:  
            return "Sunday";  
        case Calendar.MONDAY:  
            return "Monday";  
        case Calendar.TUESDAY:  
            return "Tuesday";  
        case Calendar.WEDNESDAY:  
            return "Wednesday";  
        case Calendar.THURSDAY:  
            return "Thursday";  
        case Calendar.FRIDAY:  
            return "Friday";  
        case Calendar.SATURDAY:  
            return "Saturday";  
        default:  
            return "Unknown";  
    }  
}
```



GetEmail

```
package com.example.projectarieleyalyotamyona.Utilities;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

/**
 * Utility class for retrieving the user's email from Firestore.
 * This class provides a method to fetch the email of the currently
 * logged-in user.
 */
public class GetEmail {
    private String email;

    /**
     * Callback interface for retrieving the email.
     * Implement this interface to handle the result of the email
     * retrieval operation.
     */
    public interface ToolTextCallback {
        /**
         * Called when the email is successfully fetched or if an error
         * occurs.
         *
         * @param email The user's email address, or "Not logged in" if
         * the user is not logged in or an error occurred.
         */
        void onToolTextFetched(String email);
    }

    /**
     * Fetches the email of the currently logged-in user from Firestore.
     *
     * <p>
     * This method retrieves the user's email from the "users" collection
     * in Firestore,
     * using the user's UID obtained from Firebase Authentication. If the
     * user is not
     * logged in or if an error occurs during the retrieval process, the
     * callback
     * will be called with the value "Not logged in".
     * </p>
     *
     * @param callback The callback object to be notified of the result.
     */
    public void ToolText(GetEmail.ToolTextCallback callback) {
        FirebaseAuth mAuth = FirebaseAuth.getInstance();
        FirebaseUser userCurrent = mAuth.getCurrentUser();

        // If the user is not logged in, return a default value
        if (userCurrent == null) {
            callback.onToolTextFetched("Not logged in");
        }
    }
}
```



```
        return;
    }

    FirebaseFirestore db = FirebaseFirestore.getInstance();

    db.collection("users")
        .document(userCurrent.getId())
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                email = documentSnapshot.getString("email");
                callback.onToolTextFetched(email != null ? email :
                "Not logged in"); // Return the user's email
            } else {
                callback.onToolTextFetched("Not logged in"); // If
                the document does not exist
            }
        })
        .addOnFailureListener(e -> {
            callback.onToolTextFetched("Not logged in"); // In
            case of error
        });
    }
}
```



GetFirstHours

```
package com.example.projectarieleyalyotamyona.Utilities;

import androidx.annotation.NonNull;

import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * Utility class for retrieving the first booked hour for the current day.
 * This class provides a method to fetch the first hour a room is booked
 * for a specific user.
 */
public class GetFirstHours {

    /**
     * Callback interface for retrieving the first booked hours.
     * Implement this interface to handle the result of the operation.
     */
    public interface HoursCallback {
        /**
         * Called when the first booked hours are successfully fetched.
         *
         * @param firstHours A list of integers representing the first
         hours.
         */
        void onSuccess(List<Integer> firstHours);

        /**
         * Called when an error occurs during the fetching process.
         *
         * @param e The exception that occurred.
         */
        void onFailure(Exception e);
    }

    /**
     * Retrieves the first booked hours for the current day for the
     logged-in user.
     *
     * <p>
     * This method fetches the booking data from Firestore for the current
     user,
     * determines the current day, and extracts the first hour from each
     booking
     */
}
```



```
* on that day. It then returns these first hours via a callback.
* </p>
*
* @param callback The callback object to be notified of the result.
*/
public static void GetFirstHours(final HoursCallback callback) {
    FirebaseAuth mAuth = FirebaseAuth.getInstance();
    FirebaseUser currentUser = mAuth.getCurrentUser();
    String userId = currentUser != null ? currentUser.getId() : "";
    String day = GetDay.getCurrentDayName(); // Get the current day
    List<Integer> firstHours = new ArrayList<>();

    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("users").document(userId).get()
        .addOnSuccessListener(new
OnSuccessListener<DocumentSnapshot>() {
            @Override
            public void onSuccess(DocumentSnapshot
documentSnapshot) {
                List<Map<String, Object>> bookingsData =
(List<Map<String, Object>>) documentSnapshot.get("bookedRooms");
                if(bookingsData != null) { //check if bookingsData
is not null
                    for (Map<String, Object> bookingData :
bookingsData) {
                        if (bookingData.get("day").equals(day)) {
                            List<String> hours = (List<String>)
bookingData.get("hourStatuses");
                            if (hours != null && !hours.isEmpty())
{
                                try {
                                    firstHours.add(Integer.parseInt(hours.get(0))); // Simply add the first
hour
                                } catch (NumberFormatException e)
{
                                    callback.onFailure(e);
                                    return;
                                }
                            }
                        }
                    }
                }
                // Callback to notify success
                callback.onSuccess(firstHours);
            }
        })
        .addOnFailureListener(new OnFailureListener() {
            @Override
            public void onFailure(@NonNull Exception e) {
                // Callback to notify failure
                callback.onFailure(e);
            }
        })
    }
```



5 יחידות לימוד מדעי המחשב - (luz) - אריאל חלף

```
    } ) ;  
}  
}
```



GetHour

```
package com.example.projectarieleyalyotamyona.Utilities;

import java.util.Calendar;
import java.util.TimeZone;

/**
 * Utility class for getting the current hour and converting it to a
 * school schedule hour.
 * This class provides methods to retrieve the current hour in 24-hour
 * format,
 * format it as a user-friendly string, and determine the corresponding
 * school
 * schedule hour based on predefined time ranges. The timezone used is
 * Asia/Jerusalem.
 */
public class GetHour {

    /**
     * Gets the current hour in 24-hour format, using the Asia/Jerusalem
     * time zone.
     *
     * @return The current hour (0-23) in the Asia/Jerusalem time zone.
     */
    public static int getCurrentHour() {
        Calendar calendar =
        Calendar.getInstance(TimeZone.getTimeZone("Asia/Jerusalem"));
        return calendar.get(Calendar.HOUR_OF_DAY); // Returns the hour (0-
        23)
    }

    /**
     * Gets the current hour formatted as a user-friendly string,
     * using the {@link #getScheduleHour()} method.
     *
     * @return A string representing the current hour. Returns "Hour X"
     if the
     * current time is within school hours, or "Not within school hours"
     * if it is outside of the defined school hours.
     */
    public static String getFormattedHour() {
        int scheduleHour = getScheduleHour();

        if (scheduleHour == -1) {
            return "Not within school hours";
        }
        return "Hour " + scheduleHour;
    }

    /**
     * Gets the current school schedule hour, using the Asia/Jerusalem
     * time zone.
     * This method retrieves the current hour and minute and then uses the

```



```
* {@link #convertToScheduleHour(int, int)} method to determine the
* corresponding school schedule hour.
*
* @return The current school schedule hour (1-9), or -1 if the
current
* time is not within any defined school hour.
*/
public static int getScheduleHour() {
    Calendar calendar =
Calendar.getInstance(TimeZone.getTimeZone("Asia/Jerusalem"));
    int currentHour = calendar.get(Calendar.HOUR_OF_DAY);
    int currentMinute = calendar.get(Calendar.MINUTE);
    return convertToScheduleHour(currentHour, currentMinute);
}

/**
 * Converts the current time (hour and minute) to a specific school
schedule hour.
 * This method uses a predefined array of time ranges to map the
current time
 * to a school hour. The time ranges are defined in minutes since
midnight.
 *
 * @param currentHour The current hour (0-23).
 * @param currentMinute The current minute (0-59).
 * @return The corresponding school schedule hour (1-9), or -1 if the
* current time is not within any defined school hour.
*/
private static int convertToScheduleHour(int currentHour, int
currentMinute) {
    // Define the start and end times for each hour in minutes since
midnight
    int[][] hourRanges = {
        {495, 540}, // Hour 1: 8:15 AM - 9:00 AM
        {545, 590}, // Hour 2: 9:05 AM - 9:50 AM
        {600, 645}, // Hour 3: 10:00 AM - 10:45 AM
        {650, 695}, // Hour 4: 10:50 AM - 11:35 AM
        {705, 750}, // Hour 5: 11:45 AM - 12:30 PM
        {750, 790}, // Hour 6: 12:30 PM - 1:10 PM
        {790, 830}, // Hour 7: 1:10 PM - 1:50 PM
        {835, 875}, // Hour 8: 1:55 PM - 2:35 PM
        {880, 920} // Hour 9: 2:40 PM - 3:20 PM
    };

    // Calculate the total minutes since midnight for the current time
    int currentTotalMinutes = currentHour * 60 + currentMinute;

    // Determine the schedule hour based on the ranges
    for (int i = 0; i < hourRanges.length; i++) {
        if (currentTotalMinutes >= hourRanges[i][0] &&
currentTotalMinutes <= hourRanges[i][1]) {
            return i + 1; // Hours are 1-based
        }
    }
}
```



```
// If the time is not within any range, return -1  
return -1;  
}  
}
```



GetIsStaff

```
package com.example.projectarieleyalyotamyona.Utilities;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

/**
 * Utility class to determine if a user is a staff member.
 * This class provides a method to check the 'staff' field in the user's
 * Firestore document.
 */
public class GetIsStaff {

    /**
     * Callback interface for retrieving the staff status.
     * Implement this interface to handle the result of the is-staff
     check.
     */
    public interface ToolTextCallback {
        /**
         * Called when the staff status is successfully fetched or if an
         error occurs.
         *
         * @param isStaff A boolean indicating whether the user is a staff
         member.
         *
         * Returns false if the user is not logged in, the document
         does not exist, or an error occurred.
         */
        void onToolTextFetched(boolean isStaff);
    }

    /**
     * Checks if the current user is a staff member by querying the
     Firestore 'users' collection.
     *
     * <p>
     * This method retrieves the user's document from Firestore using
     their UID.
     * It then checks the value of the 'staff' field. If the user is not
     logged in,
     * the document doesn't exist, or an error occurs, the callback is
     called with 'false'.
     * </p>
     *
     * @param callback The callback object to be notified of the result.
     */
    public void checkIfStaff(ToolTextCallback callback) {
        FirebaseAuth mAuth = FirebaseAuth.getInstance();
        FirebaseUser currentUser = mAuth.getCurrentUser();

        // If the user is not logged in, return false
        if (currentUser == null) {
```



```
        callback.onToolTextFetched(false);
        return;
    }

    FirebaseFirestore db = FirebaseFirestore.getInstance();

    db.collection("users")
        .document(currentUser.getId())
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                Boolean isStaff =
documentSnapshot.getBoolean("staff");
                // Check if the value is non-null to avoid
NullPointerException
                callback.onToolTextFetched(isStaff != null ?
isStaff : false);
            } else {
                callback.onToolTextFetched(false); // Document
does not exist
            }
        })
        .addOnFailureListener(e -> {
            callback.onToolTextFetched(false); // Handle failure
gracefully
        });
    }

    /**
     * Returns a string representation of this object.
     *
     * @return A string representation of this object.
     */
    @Override
    public String toString() {
        return "GetIsStaff{}";
    }
}
```



GetIsT

```
package com.example.projectarieleyalyotamyona.Utilities;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

/**
 * Utility class to check if a user is a teacher.
 * This class provides a method to check the 'isT' field in the user's
 * Firestore document,
 * indicating whether the user is a teacher.
 */
public class GetIsT {

    /**
     * Method to check if the user is a teacher.
     * This method retrieves the user's document from Firestore and checks
     * the value of the 'isT' field.
     * The result is returned through a callback interface.
     *
     * @param callback The callback interface to return the result.
     */
    public void checkIfTeacher(FirestoreCallback callback) {
        FirebaseUser currentUser =
        FirebaseAuth.getInstance().getCurrentUser();

        if (currentUser != null) {
            // Retrieve the user document from Firestore
            FirebaseFirestore db = FirebaseFirestore.getInstance();
            db.collection("users").document(currentUser.getId())
                .get()
                .addOnSuccessListener(documentSnapshot -> {
                    boolean isTeacher = documentSnapshot.exists()
                        &&
                        Boolean.TRUE.equals(documentSnapshot.getBoolean("isT"));
                    callback.onResult(isTeacher); // Return the result
                    // via callback
                })
                .addOnFailureListener(e -> {
                    callback.onResult(false); // Return false in case
                    // of failure
                });
        } else {
            callback.onResult(false); // Return false if no current user
        }
    }

    /**
     * Callback interface for Firestore operation.
     * This interface defines the method to be called when the result of
     * the Firestore query is available.
     */
}
```



```
public interface FirestoreCallback {  
    /**  
     * Called when the result of the Firestore query is available.  
     *  
     * @param isTeacher True if the user is a teacher, false  
    otherwise.  
     */  
    void onResult(boolean isTeacher);  
}
```



GetName

```
package com.example.projectarieleyalyotamyona.Utilities;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

/**
 * Utility class to retrieve the user's name from Firestore.
 * This class provides a method to fetch the name of the currently logged-
in user.
 */
public class GetName {
    private String Name;

    /**
     * Callback interface for retrieving the user's name.
     * Implement this interface to handle the result of the name retrieval
operation.
     */
    public interface ToolTextCallback {
        /**
         * Called when the user's name is successfully fetched or if an
error occurs.
         *
         * @param name The user's name, or "Hello" if the user is not
logged in or an error occurred.
         */
        void onToolTextFetched(String name);
    }

    /**
     * Fetches the name of the currently logged-in user from Firestore.
     *
     * <p>
     * This method retrieves the user's name from the "users" collection
in Firestore,
     * using the user's UID obtained from Firebase Authentication. If the
user is not
     * logged in or if an error occurs during the retrieval process, the
callback
     * will be called with the default value "Hello".
     * </p>
     *
     * @param callback The callback object to be notified of the result.
     */
    public void ToolText(ToolTextCallback callback) {
        FirebaseAuth mAuth = FirebaseAuth.getInstance();
        FirebaseUser userCurrent = mAuth.getCurrentUser();

        // If the user is not logged in, return a default value
        if (userCurrent == null) {
            callback.onToolTextFetched("Hello");
        }
    }
}
```



```
        return;
    }

    FirebaseFirestore db = FirebaseFirestore.getInstance();

    db.collection("users")
        .document(userCurrent.getUid())
        .get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                Name = documentSnapshot.getString("name");
                callback.onToolTextFetched(Name != null ? Name :
"Hello"); // Return the user's name
            } else {
                callback.onToolTextFetched("Hello"); // If the
document does not exist
            }
        })
        .addOnFailureListener(e -> {
            callback.onToolTextFetched("Hello"); // In case of
error
        });
    }

    /**
     * Returns a string representation of the GetName object.
     *
     * @return A string representation of the object, including the user's
name.
     */
    @Override
    public String toString() {
        return "GetName{" +
            "Name='" + Name + '\'' +
            '}';
    }
}
```



GetProfilePic

```
package com.example.projectarieleyalyotamyona.Utilities;

import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.util.Base64;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

/**
 * Utility class to retrieve the user's profile picture from Firestore.
 * The profile picture is assumed to be stored as a Base64 encoded string
 * in the Firestore database.
 */
public class GetProfilePic {
    private Bitmap profilePic;

    /**
     * Callback interface for retrieving the profile picture.
     * Implement this interface to handle the result of the profile
     * picture retrieval operation.
     */
    public interface ToolTextCallback {
        /**
         * Called when the user's profile picture is successfully fetched
         * or if an error occurs.
         *
         * @param profilePic The user's profile picture as a Bitmap, or
         * null if the user is not logged in,
         * the profile picture is not found, or an error occurred.
         */
        void onProfilePicFetched(Bitmap profilePic);
    }

    /**
     * Fetches the profile picture of the currently logged-in user from
     * Firestore.
     *
     * <p>
     * This method retrieves the user's profile picture from the "users"
     * collection in Firestore,
     * using the user's UID obtained from Firebase Authentication. The
     * profile picture is stored
     * as a Base64 encoded string. This method decodes the Base64 string
     * back into a Bitmap.
     * If the user is not logged in or if an error occurs during the
     * retrieval process, the
     * callback will be called with a null Bitmap.
     * </p>
     *
     * @param callback The callback object to be notified of the result.
     */
}
```



```
*/  
public void getProfilePic(ToolTextCallback callback) {  
    FirebaseAuth mAuth = FirebaseAuth.getInstance();  
    FirebaseUser userCurrent = mAuth.getCurrentUser();  
  
    // If the user is not logged in, return null for profile picture  
    if (userCurrent == null) {  
        callback.onProfilePicFetched(null);  
        return;  
    }  
  
    FirebaseFirestore db = FirebaseFirestore.getInstance();  
  
    db.collection("users")  
        .document(userCurrent.getId())  
        .get()  
        .addOnSuccessListener(documentSnapshot -> {  
            if (documentSnapshot.exists()) {  
                String base64String =  
documentSnapshot.getString("profilePicture"); // Assuming profilePic is  
stored as Base64  
                if (base64String != null) {  
                    profilePic =  
decodeBase64ToImage(base64String); // Decode the Base64 string to Bitmap  
                }  
                callback.onProfilePicFetched(profilePic); //  
Return the profile picture  
            } else {  
                callback.onProfilePicFetched(null); // If document  
doesn't exist  
            }  
        })  
        .addOnFailureListener(e -> {  
            callback.onProfilePicFetched(null); // In case of  
failure  
        });  
    }  
  
    /**  
     * Decodes a Base64 encoded string into a Bitmap image.  
     *  
     * @param base64String The Base64 encoded string representing the  
image.  
     * @return The decoded Bitmap image, or null if the decoding fails.  
     */  
    private Bitmap decodeBase64ToImage(String base64String) {  
        try{  
            byte[] decodedBytes = Base64.decode(base64String,  
Base64.DEFAULT);  
            return BitmapFactory.decodeByteArray(decodedBytes, 0,  
decodedBytes.length);  
        } catch (Exception e){  
            return null;  
        }  
    }  
}
```



```
}  
}
```



More

PrefManager

```
package com.example.projectarieleyalyotamyona.Utilities;

import android.content.Context;
import android.content.SharedPreferences;

/**
 * Utility class for managing user login details using Shared Preferences.
 * This class provides methods to save, retrieve, and clear login
 * information
 * such as email and password. It also provides a method to check if a
 * user
 * is currently logged out.
 */
public class PrefManager {

    Context context;

    /**
     * Constructor for PrefManager.
     *
     * @param context The application context.
     */
    public PrefManager(Context context) {
        this.context = context;
    }

    /**
     * Saves the user's login details (email and password) to Shared
     * Preferences.
     *
     * @param email The user's email address.
     * @param password The user's password.
     */
    public void saveLoginDetails(String email, String password) {
        SharedPreferences sharedPreferences =
            context.getSharedPreferences("LoginDetails", Context.MODE_PRIVATE);
        SharedPreferences.Editor editor = sharedPreferences.edit();
        editor.putString("Email", email);
        editor.putString("Password", password);
        editor.commit();
    }

    /**
     * Clears the user's login details (email and password) from Shared
     * Preferences, effectively logging the user out.
     */
    public void Forgetlog() {
        SharedPreferences sharedPreferences =
            context.getSharedPreferences("LoginDetails", Context.MODE_PRIVATE);
    }
}
```



```
        SharedPreferences.Editor editor = sharedPreferences.edit();
        editor.clear();
        editor.commit();
    }

    /**
     * Retrieves the user's email address from Shared Preferences.
     *
     * @return The user's email address, or an empty string if not found.
     */
    public String getEmail() {
        SharedPreferences sharedPreferences =
context.getSharedPreferences("LoginDetails", Context.MODE_PRIVATE);
        return sharedPreferences.getString("Email", "");
    }

    /**
     * Checks if the user is logged out by verifying if either the email
     * or password is empty in Shared Preferences.
     *
     * @return True if the user is logged out (either email or password is
     * empty), false otherwise.
     */
    public boolean isUserLoggedOut() {
        SharedPreferences sharedPreferences =
context.getSharedPreferences("LoginDetails", Context.MODE_PRIVATE);
        boolean isEmptyEmail = sharedPreferences.getString("Email",
        "").isEmpty();
        boolean isEmptyPassword = sharedPreferences.getString("Password",
        "").isEmpty();
        return isEmptyEmail || isEmptyPassword;
    }
}
```



NotificationReceiver

```
package com.example.projectarieleyalyotamyona;

import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.util.Log;

import androidx.core.app.NotificationCompat;

/**
 * BroadcastReceiver class responsible for handling received
 * notifications.
 * This class is triggered when a notification is sent to the application.
 */
public class NotificationReceiver extends BroadcastReceiver {

    /**
     * Called when the BroadcastReceiver receives an Intent broadcast.
     * This method handles the incoming notification intent, retrieves the
     * notification
     * title and message, and displays the notification.
     *
     * @param context The Context in which the receiver is running.
     * @param intent The Intent being received.
     */
    @Override
    public void onReceive(Context context, Intent intent) {
        Log.d("NotificationReceiver", "Notification received");

        // Retrieve the title and message from the Intent
        String title = intent.getStringExtra("notification_title");
        String message = intent.getStringExtra("notification_message");

        // Check if title or message is missing
        if (title == null || message == null) {
            Log.e("NotificationReceiver", "Title or message is missing
from intent");
            return;
        }

        // Get the NotificationManager system service
        NotificationManager notificationManager = (NotificationManager)
context.getSystemService(Context.NOTIFICATION_SERVICE);

        // Build the notification
        NotificationCompat.Builder builder = new
NotificationCompat.Builder(context, "notify_channel_id")
            .setSmallIcon(R.mipmap.ic_notification_foreground) //
Replace with your notification icon
            .setContentTitle(title)
```



```
.setContentText(message)
.setPriority(NotificationCompat.PRIORITY_HIGH)
.setAutoCancel(true);

// Check if NotificationManager is initialized before using it
if (notificationManager != null) {
    // Notify with ID 1 (you can use different IDs for different
notifications)
    notificationManager.notify(1, builder.build());
} else {
    Log.e("NotificationReceiver", "NotificationManager is null");
}
}
```



SetNoti

```
package com.example.projectarieleyalyotamyona;

import android.app.AlarmManager;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;
import android.os.Build;
import android.util.Log;

import com.example.projectarieleyalyotamyona.NotificationReceiver;

import java.util.Calendar;

/**
 * Utility class for setting up scheduled notifications for class
 * reminders.
 * This class provides methods to calculate the notification time based on
 * the
 * first hour of class and schedule the notification using AlarmManager.
 */
public class SetNoti {
    private static final String TAG = "SetNoti";

    /**
     * Converts the first hour of class to a specific time and schedules a
     * notification.
     *
     * <p>
     * This method takes the first hour of the class as input, calculates
     * the corresponding
     * time (hour and minute), adjusts it to be 10 minutes earlier, and
     * then calls
     * {@link #scheduleNotification\(Context, String, String, int, int\)} to
     * schedule
     * the notification.
     * </p>
     *
     * @param context The Context in which to schedule the notification.
     * @param firsthour The first hour of the class (1-9).
     * @throws IllegalArgumentException If the provided firsthour is
     * invalid.
     */
    public static void convertAndSet(Context context, int firsthour) {
        int hour;
        int min;

        // Determine the start time of the first class
        switch (firsthour) {
            case 1:
                hour = 8;
```



```
        min = 15;
        break;
    case 2:
        hour = 9;
        min = 5;
        break;
    case 3:
        hour = 10;
        min = 0;
        break;
    case 4:
        hour = 10;
        min = 50;
        break;
    case 5:
        hour = 11;
        min = 45;
        break;
    case 6:
        hour = 12;
        min = 30;
        break;
    case 7:
        hour = 13;
        min = 10;
        break;
    case 8:
        hour = 13;
        min = 55;
        break;
    case 9:
        hour = 14;
        min = 40;
        break;
    default:
        throw new IllegalArgumentException("Invalid period index:
" + firsthour);
    }

    Log.d(TAG, "Converting firsthour " + firsthour + " to time: " +
hour + ":" + min);

    // Adjust the time by 10 minutes earlier
    int adjustedMinute = min - 10;
    if (adjustedMinute < 0) {
        hour -= 1;
        adjustedMinute += 60;
    }

    Log.d(TAG, "Adjusted time: " + hour + ":" + adjustedMinute);

    // Schedule the notification
    scheduleNotification(context, "Class reminder", "Your class starts
soon at: " + hour + " " + min, hour, adjustedMinute);
```



```
}

/**
 * Schedules a notification using AlarmManager.
 *
 * <p>
 * This method creates a notification channel (for Android O and
above),
 * builds a PendingIntent to launch the {@link NotificationReceiver},
 * sets the notification time using Calendar, and schedules the alarm.
 * </p>
 *
 * @param context The Context in which to schedule the notification.
 * @param title The title of the notification.
 * @param message The message body of the notification.
 * @param hour The hour at which to display the notification (24-
hour format).
 * @param minute The minute at which to display the notification.
 */
private static void scheduleNotification(Context context, String
title, String message, int hour, int minute) {
    // Create the notification channel (necessary for Android O and
above)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        CharSequence name = "MyNotificationChannel";
        String description = "Channel for scheduled notifications";
        int importance = NotificationManager.IMPORTANCE_DEFAULT;
        NotificationChannel channel = new
NotificationChannel("notify_channel_id", name, importance);
        channel.setDescription(description);

        NotificationManager notificationManager =
(NotificationManager)
context.getSystemService(Context.NOTIFICATION_SERVICE);
        if (notificationManager != null) {
            notificationManager.createNotificationChannel(channel);
        }
    }

    // Create an Intent for the NotificationReceiver
    Intent intent = new Intent(context, NotificationReceiver.class);
    intent.putExtra("notification_title", title); // Changed key to
match NotificationReceiver
    intent.putExtra("notification_message", message); // Changed key
to match NotificationReceiver

    // Create a PendingIntent for the AlarmManager
    PendingIntent pendingIntent = PendingIntent.getBroadcast(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT |
PendingIntent.FLAG_IMMUTABLE
    );

    // Set the time for the notification
    Calendar calendar = Calendar.getInstance();
```



```
calendar.set(Calendar.HOUR_OF_DAY, hour);
calendar.set(Calendar.MINUTE, minute);
calendar.set(Calendar.SECOND, 0);

// If the scheduled time is in the past, set it for the next day
if (calendar.getTimeInMillis() < System.currentTimeMillis()) {
    calendar.add(Calendar.DAY_OF_YEAR, 1);
}

// Schedule the alarm
AlarmManager alarmManager = (AlarmManager)
context.getSystemService(Context.ALARM_SERVICE);
if (alarmManager != null) {
    alarmManager.setExact(AlarmManager.RTC_WAKEUP,
calendar.getTimeInMillis(), pendingIntent);
    Log.d(TAG, "Notification scheduled for: " +
calendar.getTime().toString());
}
}
```



RoomAdapterInterface

```
package com.example.projectarieleyalyotamyona.Utilities;

/**
 * Interface for handling item click events in a room adapter.
 * This interface defines a callback method to be invoked when an item in
 * a room adapter is clicked.
 */
public interface RoomAdapterInterface {

    /**
     * Called when an item in the room adapter is clicked.
     *
     * @param position The position of the clicked item in the adapter's
     * data set.
     */
    void OnItemClick(int position);
}
```

סוף (סוף סוף).